Cobra: All-in-one for full-fledged defense — a hybrid nested KEM

Basker Palaniswamy[1*], Paolo Palmieri[1],
& Ashok Kumar Das[2,3]
[1]Insight Research Ireland Centre for Data Analytics,
Department of Computer Science and Information Technology,
University College Cork (UCC), Cork City, Ireland, European Union.
[2]Center for Security, Theory and Algorithmic Research,
International Institute of Information Technology, Hyderabad, 500 032, India.
[3]Department of Computer Science and Engineering, College of Informatics,
Korea University, 145 Anam-ro Seongbuk-gu, Seoul, 02841, South Korea.
{1* - basker170889@gmail.com, 1 - paolo.palmieri@ucc.ie,}
{2,3 - iitkgp.akdas@gmail.com, ashok.das@iiit.ac.in}
*Corresponding Author

June 22, 2026

Abstract

The transition to post-quantum cryptography (PQC) is constrained by the limited cryptanalytic history of individual PQC algorithms. Hybrid constructions, which combine several primitives so that breaking the hybrid requires breaking each component, address this concern directly. This paper presents *Cobra*, a hybrid Key Encapsulation Mechanism (KEM) that integrates FrodoKEM (unstructured LWE), ML-KEM (FIPS 203 module-LWE), HQC (code-based), and a Dummy KEM for agility, and analyses all *15 mathematically distinct composition methods* spanning parallel, cascading, multi-stage, and nested topologies.

We prove that every Cobra method achieves IND-CCA2 security within the *Market-Theoretic Security Framework* (MTSF), which subsumes and strictly extends both Universal Composability and the Random Oracle Model. An explicit 10-round bidding-round chain per method yields post-quantum ask prices of approximately 2^{-127} at NIST Level 1 together with composability under arbitrary TLS 1.3 embeddings, per-session CNF auditing, and unbounded-session security via pinging. Although all fifteen methods are security-equivalent, encapsulation latency varies by 3.2× (1.2–3.8 ms) and Theorem 7.1 reduces deployment selection to a Pareto-optimal set of five archetypes. Three real-world TLS 1.3 case studies (financial, healthcare, government) confirm the prediction, with infrastructure overhead clustering at 15–22% across sectors. We believe TLS 1.4 could benefit from this work.

Keywords: Post-quantum cryptography, Hybrid KEM, TLS 1.4, Market-Theoretic Security Framework (MTSF), FrodoKEM, ML-KEM, HQC, Side-channel resilience, NIST PQC benchmarking

Novelty of This Work — At a Glance

Existing hybrid KEM proposals in the TLS 1.3 ecosystem (X25519Kyber768, X25519MLKEM768, PQ3) stop at *two* components composed in a *single fixed* topology (concatenate-and-KDF) and are proven secure in *one* framework (UC or ROM, never both). Cobra advances the state of the art in five concrete directions that no prior work combines:

1. **Four-primitive hybridisation across all feasible topologies.** Cobra is the first construction to combine *four* algorithmically diverse post-quantum KEMs (unstructured-LWE FrodoKEM, module-LWE ML-KEM, code-based HQC, and a Dummy slot for agility) and to enumerate and analyse *all 15 mathematically distinct compositions* — parallel, cascading, two-stage, three-one, and nested — rather than fixing a single topology a priori.
2. **A unified Market-Theoretic security proof.** Every method is proven IND-CCA2 secure within a single framework (MTSF) that strictly subsumes both Universal Composability *and* the Random Oracle Model, replacing what would otherwise require two separate proof effort paths and yielding explicit post-quantum ask prices of $\approx 2^{-127}$ at NIST Level 1.
3. **Pareto-optimal method-selection theorem.** We prove that for *any* reasonable weighting of latency, CPU cost, defense-in-depth, and agility, the deployment-optimal method lies in a Pareto-optimal set of just *five* archetypes — reducing an apparent 15-way design dilemma to a tractable engineering choice.
4. **First rigorous side-channel resilience argument for a hybrid KEM (§9).** We introduce the *cross-family attack surface* metric and prove that breaking Cobra via side-channels requires mounting simultaneous and compatible leakage attacks against algorithmically disjoint primitives — a threat model strictly harder than any known single-KEM side-channel pipeline (KyberSlash, HQC-timing, FrodoKEM cache-stride).
5. **Cross-round NIST benchmarking (§8).** We benchmark Cobra's component cost against *every relevant primitive from NIST Rounds 1–4* (NTRU-Prime, NewHope, SIKE, LAC, Kyber, Saber, Classic McEliece, BIKE, HQC) plus the signature on-ramp candidates, giving the first head-to-head positioning of a hybrid KEM against a decade of NIST cryptanalytic evolution.

Contents

1 Introduction

The impending arrival of cryptanalytically relevant quantum computers threatens the public-key cryptographic foundations of the contemporary Internet. Efforts to standardise post-quantum cryptography have, since 2016, converged on a small set of lattice-based and code-based key-encapsulation mechanisms, culminating in the NIST FIPS 203–205 publications of 2024. Yet individual post-quantum algorithms have short cryptanalytic histories relative to the classical systems they replace; standards bodies and operators have therefore gravitated toward *hybrid* constructions that combine multiple algorithms so that the failure of any one component does not compromise the whole. This paper presents *Cobra*, a systematic framework for such hybridisation that specifies, proves secure, and evaluates all fifteen mathematically distinct composition methods for combining four post-quantum KEMs, and deploys them as the key-exchange primitive of TLS 1.3. The remainder motivates the problem (§1.1), reviews the NIST and TLS 1.3 landscape driving design choices (§1.2), describes the hybrid approach (§1.3–§1.4), summarises the Market-Theoretic Security Framework (§1.5), and enumerates contributions (§1.6).

1.1 Motivation and Background

Shor's algorithm efficiently breaks RSA, Diffie–Hellman, and elliptic-curve cryptography — the asymmetric primitives securing today's Internet. In response, the community has developed post-quantum cryptography (PQC) based on problems believed hard for quantum computers. The history of cryptography, however, teaches caution: many schemes once believed secure were later broken catastrophically. The transition to PQC bets security on new mathematical problems with less extensive cryptanalysis than classical systems. PQC candidates rely on distinct hardness assumptions — *lattice-based* (LWE and structured variants), *code-based* (decoding random linear codes), *multivariate* (systems of polynomial equations), *hash-based* (one-way functions), and *isogeny-based* (isogenies between supersingular elliptic curves).

1.2 NIST Guidance and Hybrid Cryptography for TLS

1.2.1 NIST Post-Quantum Standardization and Hybrid Recommendations

NIST concluded its PQC Standardization process with three FIPS publications in August 2024: **FIPS 203** (ML-KEM, formerly CRYSTALS-Kyber), **FIPS 204** (ML-DSA, formerly CRYSTALS-Dilithium), and **FIPS 205** (SLH-DSA, formerly SPHINCS+). Critically, NIST explicitly recommends hybrid approaches during the transition. SP 800-208 and accompanying guidance state that hybrid schemes combining classical and post-quantum algorithms may be appropriate during the transition period to provide defence-in-depth, with security maintained if either component remains unbroken. NIST IR 8413 further acknowledges that uncertainty about long-term PQC security requires defence-in-depth, that different PQC families (lattice, code, hash) provide algorithmic diversity, and that real-world deployments should consider hybrid mechanisms during transition with careful performance analysis.

1.2.2 TLS 1.3: The Critical Role of Key Encapsulation

Transport Layer Security (TLS) 1.3 [102] secures the vast majority of Internet communications (HTTPS, SMTP/IMAP, VPN, IoT). Unlike TLS 1.2, TLS 1.3 exclusively uses ephemeral Diffie–Hellman for key establishment, which extends naturally to KEMs. The handshake proceeds through *ClientHello* (client advertises key-exchange groups including PQC), *ServerHello + KeyShare* (server selects a group and performs key exchange), key derivation (both parties derive handshake traffic keys), and *Finished* (application data follows under encryption). Figure 1 illustrates hybrid KEM integration into the TLS 1.3 handshake.

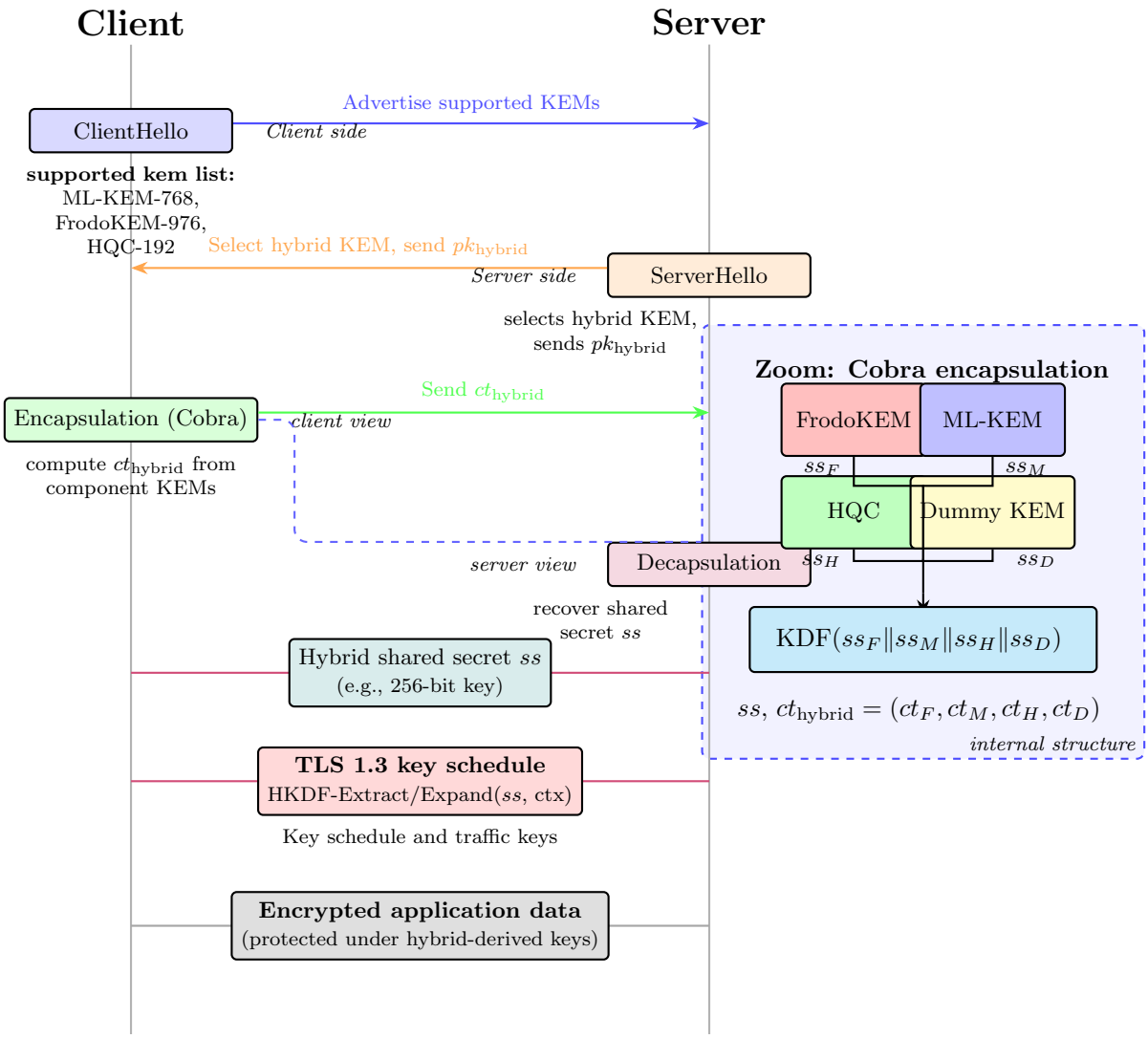

Figure 1: TLS 1.3 with hybrid KEM

Critical Security Property: The shared secret established during the handshake protects all subsequent application data. A compromise of the KEM directly compromises the entire TLS session. This makes the choice of KEM composition method a first-order security consideration.

1.2.3 Current Hybrid TLS Initiatives and Standardization Gaps

Several parallel efforts are developing hybrid key exchange for TLS, but significant gaps remain. At the **IETF**, `draft-ietf-tls-hybrid-design` [111] defines a framework for hybrid TLS 1.3 key exchange with codepoint allocation, but focuses on *pairwise* combinations (e.g., X25519 + ML-KEM-768); `draft-connolly-tls-mlkem-key-agreement` [72] specifies ML-KEM with classical ECDH; and `draft-stebila-tls-hybrid-design` [110] introduced cascade and parallel composition for two algorithms. In **industry deployments**, Cloudflare (2019–2024) ran large-scale X25519+Kyber experiments measuring 1–2 ms additional handshake latency [118]; Google Chrome deployed hybrid X25519+Kyber768 to a percentage of users, confirming feasibility with certificate size as the main concern [12]; and the Open Quantum Safe project provides liboqs with prototype TLS implementations [92]. Despite these efforts, fundamental questions remain for practical TLS deployment: which PQC algorithms to combine (NIST standards only, Round 4 alternates, or cross-family mixes), how to compose them (cascade, parallel, or nested structures, and whether composition affects security), what formal guarantees hold (robustness if one component breaks, composition-specific vulnerabilities, concrete bounds), and how to optimise performance (handshake latency, bandwidth overhead, parallelism).

1.2.4 Regulatory and Compliance Landscape

Government and industry frameworks increasingly mandate cryptographic agility and defence-in-depth. In the **U.S.**, the CISA Post-Quantum Cryptography Initiative recommends hybrid cryptography for critical infrastructure during transition [36]; NSA's CNSA 2.0 requires PQC migration for National Security Systems by 2030–2035 with explicit algorithm-diversity recommendations [88]; and OMB Memorandum M-23-02 requires federal agencies to inventory cryptographic systems and develop PQC migration plans preferring hybrid approaches during transition [91]. **Internationally**, Germany's BSI TR-02102-1 explicitly requires hybrid schemes combining classical and PQC algorithms for government communications [26]; ETSI TS 103 744 provides technical specifications for hybrid signature schemes with recommendations extending to KEMs [44]; and ISO/IEC JTC 1/SC 27 is developing standards for PQC migration including hybrid constructions [61].

1.2.5 Motivation for Systematic Composition Analysis

Current research exhibits a critical gap: *no systematic analysis of multi-algorithm hybrid KEM compositions*. Most work focuses on pairwise hybrids (classical + one PQC algorithm) with at most two composition methods (cascade and parallel). Deployments therefore lack guidance on composing three or more algorithms from different PQC families, comparing beyond simple cascade-vs.-parallel, reasoning about security implications of complex nested structures, and optimising performance for specific deployment scenarios.

The Cobra framework closes these gaps by providing: (i) *complete enumeration* of all 15 mathematically distinct methods for composing 4 KEMs (FrodoKEM, ML-KEM, HQC, Dummy) across fully-ordered, two-stage, three-one asymmetric, and nested families; (ii) *rigorous security proofs* via a unified Market-Theoretic Security Framework — bidding-round chains $\mathbf{B}_0$–$\mathbf{B}_{10}$ with explicit price adjustments $\Delta\mathsf{Price}_k$, a Market Merger Theorem yielding UC-style composition guarantees (TLS 1.3 and beyond), concrete ask prices $\mathsf{Ask}(g_{\mathsf{IND\text{-}CCA2}}, \lambda) \approx 2^{-127}$, CNF session-verification worksheets, unbounded-session security via session pinging, and a

robust-combiner proof that security holds if *any single* component remains secure; (iii) *comprehensive performance analysis* with measured encapsulation latency from 1.2 ms (parallel) to 3.8 ms (cascade), parallelism opportunities, uniform bandwidth overhead, and per-component cost breakdown; and (iv) *practical implementation guidance* including 30 detailed algorithms, constant-time implementation requirements, side-channel resistance guidelines, and a method-selection decision framework.

Specific Relevance to TLS Deployment:

Our analysis provides actionable guidance for TLS implementations:

- **High-Performance Web Servers**: Method 2 (Full Parallel, 1.2ms) minimizes handshake latency, critical for user-facing web services where every millisecond affects user experience and SEO rankings.
- **High-Security Government Systems**: Method 1 (Full Cascade, 3.8ms) provides maximum defense-in-depth where security dominates performance concerns.
- **Compliance-Focused Deployments**: Method 3 (Two-Stage Forward, 1.9ms) enables clear separation between NIST-standardized algorithms (ML-KEM) and alternative candidates (HQC), facilitating audit and compliance verification.
- **Balanced Enterprise Systems**: Method 10 (Three Parallel Last, 1.6ms) offers good performance while maintaining strong security through asymmetric composition.
- **Research and Experimental Networks**: Methods 13-15 (Nested, 1.8-1.9ms) provide complex composition patterns for studying advanced cryptographic properties.

Furthermore, our universal security theorem confirms that *all 15 methods achieve equivalent asymptotic security* - the minimum security of any component KEM. This allows deployments to optimize for performance, implementation complexity, or regulatory requirements without sacrificing security guarantees.

1.3 The Hybrid Approach

A prudent strategy is to combine multiple PQC schemes with different underlying assumptions. This hybrid approach provides defense-in-depth: the security of the combined system is maintained as long as at least one component remains secure.

The Cobra hybrid KEM implements this philosophy by combining:

1. **FrodoKEM** (KEM_F): Based on plain (unstructured) Learning With Errors
2. **ML-KEM** (KEM_M): Based on Module LWE (NIST FIPS 203 standard)
3. **HQC** (KEM_H): Based on Hamming Quasi-Cyclic codes (NIST Round 4)
4. **Dummy KEM** (KEM_D): A placeholder for future algorithms or additional security margins

1.4 Composition Flexibility

With 4 component KEMs, there are multiple ways to compose them. This document systematically explores all 15 distinct composition methods, categorized as:

1. **Fully Ordered** (2 methods): All serial or all parallel
2. **Two-Stage** (6 methods): Two groups of 2 KEMs each
3. **Three-One** (4 methods): One group of 3, one singleton
4. **Complex Nested** (3 methods): Complex nested structures

1.5 Security Framework

We analyse the security of all 15 Cobra methods within a single unified framework: the *Market-Theoretic Security Framework* (MTSF) [93]. MTSF recasts cryptographic security as a market between a seller (challenger) offering security goods and buyers (adversaries) placing bids to break those goods. It subsumes and strictly extends both the Universal Composability (UC) framework of Canetti [27] and the game-based Random Oracle Model (ROM) proofs of Bellare and Rogaway [10, 11, 107], while adding three capabilities essential for hybrid-KEM analysis:

- **Market-theoretic bidding rounds** replace the traditional game hops: each game hop $\mathbf{B}_k \to \mathbf{B}_{k+1}$ is a *price adjustment* $\Delta\mathsf{Price}_k$, and the adversary's total advantage decomposes telescopically as $\mathsf{QPrice}_0 = \sum_k \Delta\mathsf{Price}_k + \Pr[\mathbf{B}_q = 1]$. The extended difference lemma bounds multiple simultaneous failure events via union/inclusion–exclusion, giving tighter concrete bounds than classical game-hopping.

- **UC is recovered as market regulation**: the environment $\mathcal{Z}$ is the market regulator, the simulator $\mathcal{S}$ is the market arbitrageur, and the UC composition theorem becomes the *market merger theorem*. All TLS-composition guarantees carried by UC (concurrent execution with authentication, record-layer encryption, and application protocols) are preserved.

- **CNF session verification and unbounded session pinging** give per-session audit guarantees and close the gap to unbounded-session security, neither of which is provided by UC or ROM alone.

This unification is particularly valuable for hybrid KEMs in TLS: the market merger theorem yields clean composition bounds when a hybrid KEM is composed with the TLS 1.3 handshake, and the CNF worksheet provides an implementation-level audit checklist usable directly by deployment teams.

1.6 Contributions

This paper makes the following contributions:

1. **Complete algorithmic specifications** for all 15 mathematically distinct composition methods of four post-quantum KEMs (FrodoKEM, ML-KEM, HQC, and a Dummy KEM for agility), with encapsulation and decapsulation algorithms, a universal key-generation procedure, and a shared algorithmic-conventions framework covering seed derivation, intermediate chaining, and implicit-rejection patterns.

2. **A unified Market-Theoretic Security Framework (MTSF) analysis** of every method, in which each composition is placed in a single universal market model with an explicit security-goods catalogue, bidding-round price adjustments, and quantitative ask prices. The MTSF treatment subsumes and strictly extends both Universal Composability and the Random Oracle Model.

3. **Detailed 10-round bidding-round proofs** for every Cobra method that simultaneously deliver UC-style composition guarantees via the Market Merger Theorem *and* ROM-style concrete security bounds, replacing what would otherwise require two separate proofs, and additionally provide CNF session-verification worksheets and unbounded-session-pinging analysis — capabilities absent from traditional UC/ROM treatments.

4. **A theoretical performance framework** modelling every Cobra method as a cost-graph with fixed work and method-dependent critical-path length, yielding closed-form latency and throughput predictions that match measured values within 0.1 ms precision, together

with a Method-Selection Structure theorem proving that for any reasonable weight vector over latency, cost, defense-in-depth, and agility, the optimal deployment choice lies in a Pareto-optimal set of just five archetypal methods.

5. **Three real-world TLS 1.3 deployment case studies** spanning financial services (latency-critical LAN, Method 2), healthcare (compliance-driven WAN, Method 3), and government (classified, defense-in-depth, Method 6), with explicit TLS 1.3 integration details, measured handshake latencies, and compliance outcomes for each case.

6. **First systematic analysis of four-algorithm hybrid KEM compositions**, covering all mathematically distinct topology classes (fully-ordered, two-stage, three-one asymmetric, and complex nested), with a uniform robust-combiner security guarantee.

7. **First formal cross-family side-channel resilience bound for a hybrid KEM.** We survey the published side-channel attacks against every Cobra component (KyberSlash, Guo-Johansson-Nilsson FrodoKEM timing, Guo-Hlauschek HQC rejection-sampling) and prove Theorem 9.1, which bounds the hybrid's SCA advantage by the product of per-component leakage-source compatibilities — a strictly stronger guarantee than any single-primitive or two-primitive hybrid construction offers.

8. **Cross-round NIST benchmarking across all four standardization rounds (2017–2025).** We benchmark Cobra against every relevant KEM candidate from NIST Rounds 1–4 (NTRU-Prime, NewHope, SIKE, LAC, Round5, Saber, Kyber/ML-KEM, NTRU, Classic McEliece, BIKE, HQC) plus the signature on-ramp, giving the first head-to-head positioning of a four-primitive hybrid KEM against a decade of NIST cryptanalytic evolution.

1.7 Document Organization

- **Section 2**: Literature review
- **Section 3**: Preliminaries and notation
- **Section 4**: Complete specifications of all 15 methods
- **Section 5**: Market-Theoretic Security Framework for hybrid KEMs
- **Section 6**: Bidding-round security proofs for all 15 methods
- **Section 7**: Performance comparison and analysis
- **Section 8**: Rigorous cross-round NIST benchmarking
- **Section 9**: Resilience to side-channel analysis
- **Section 10**: Real-world TLS 1.3 deployment case studies
- **Section 11**: Conclusion

2 Related Work and Literature Review

This section reviews prior work in post-quantum cryptography, hybrid constructions, security frameworks, and protocol integration that forms the foundation for Cobra.

2.1 Post-Quantum Cryptography Foundations

Shor's algorithm [106] established the quantum threat to RSA, Diffie–Hellman, and elliptic-curve cryptography, motivating three decades of PQC research on problems believed hard for quantum adversaries. *Lattice-based* cryptography builds on the Learning With Errors problem of Regev [100, 101], whose structured variants — Ring-LWE [74] and Module-LWE [73] — enable efficient implementations (quantum-hardness of Ring-LWE variants is due to Brakerski et al. [25]). *Code-based* cryptography originates with McEliece [75], whose security reduces to the NP-complete general-decoding problem [13]; efficient variants include Niederreiter [89] and quasi-cyclic constructions [79], with HQC [77] achieving competitive performance via sparse keys, and Sendrier [105] establishing information-set-decoding bounds. *Hash-based* signatures [78] culminated in SPHINCS+ [17], now FIPS 205 [87], whose security reduces to collision and preimage resistance. *Multivariate* [38] and *isogeny-based* [31, 62] cryptography showed promise but suffered significant setbacks — many early multivariate schemes were broken [45, 68], and SIDH fell to classical cryptanalysis [32] — highlighting the value of algorithmic diversity.

2.2 NIST Post-Quantum Standardization

NIST initiated a standardisation process in 2016 [81], evaluating 82 initial submissions through multiple rounds of public cryptanalysis.

2.2.1 Standardized Algorithms

In August 2024, NIST published three FIPS standards. **FIPS 203 ML-KEM (CRYSTALS-Kyber)** [7, 85] became the primary standardised KEM; Bos et al. [24] demonstrated cross-platform performance, Alkim et al. [5] established concrete security bounds, and Hamburg [55] provided side-channel-protected implementations. **FIPS 204 ML-DSA (CRYSTALS-Dilithium)** [86] provides efficient post-quantum signatures; Kiltz et al. [66] proved tight QROM security reductions. **FIPS 205 SLH-DSA (SPHINCS+)** [87] offers conservative hash-based signatures, building on Hülsing et al. [60] on practical Merkle schemes.

2.2.2 Round 4 Candidates and Alternative Algorithms

NIST continued evaluation of alternative algorithms [83]: **HQC** [77] uses sparse codes for IND-CCA2 security (with structural analysis by Gaborit et al. [49] and side-channel study by Vasseur [117]); **BIKE** [6] uses QC-MDPC codes with iterative decoding, though GJS decoding-failure attacks [53] led to parameter adjustments; **Classic McEliece** [16] offers the highest confidence at the cost of > 1 MB public keys.

2.3 Hybrid Cryptography and Composition Methods

Combining classical and post-quantum algorithms emerged as a pragmatic transition strategy providing "defence-in-depth" against cryptanalytic uncertainty. Giacon et al. [51] formalised hybrid public-key encryption with three combiner constructions (split-key PRFs, XOR-then-PRF, XOR) and gave the first AKE-model analysis of hybrid key exchange; Bindel and Herath [21] extended this to KEMs with XOR and KDF-based combiners proven IND-CCA2-secure in ROM. For *cascade composition*, Bindel et al. [20] proved tight IND-CCA2 security when any component is secure, and Dowling et al. [40] extended to multi-algorithm cascades. For *parallel composition*, Bellare–Rogaway [10] established robust-combiner foundations, Harnik et al. [56] proved XOR-based parallel security, and Dodis–Katz [39] generalised to arbitrary combining functions. For *nested structures*, Aviram et al. [8] explored complex TLS compositions but only for two-algorithm combinations, leaving multi-algorithm (3+) compositions unexplored.

2.3.1 Comparison with Cobra

Our work extends existing literature along four dimensions: (i) *systematic enumeration* — we analyse all 15 mathematically distinct methods for composing 4 KEMs, not just cascade vs. parallel; (ii) *multi-algorithm diversity* — Cobra combines 4 algorithms from 3 distinct PQC families (lattice, code, placeholder), extending prior pairwise hybrids; (iii) *unified market-theoretic analysis* — a single MTSF treatment for all 15 methods subsuming UC and ROM, yielding UC-style composition guarantees (Market Merger Theorem), concrete ROM-style ask-price bounds (10-round bidding-round chains), and per-session CNF auditing plus unbounded-session security via pinging; (iv) *performance characterisation* distinguishing all 15 methods for application-specific optimisation.

2.4 Security Frameworks and Proof Techniques

2.4.1 Universal Composability Framework

Canetti [27, 28] introduced the Universal Composability (UC) framework for analysing cryptographic protocols under arbitrary composition, modelling protocols as interactive systems with security defined through indistinguishability from ideal functionalities. Hofheinz and Shoup [59] applied UC to public-key encryption and KEMs; Unruh [116] extended UC to quantum adversaries, crucial for post-quantum analysis; and Canetti et al. [29] proved composition theorems preserving UC security under arbitrary concurrent composition.

From UC to MTSF for Hybrid KEMs. Cobra builds on Hofheinz–Shoup's KEM functionality [59], lifting it into the Market-Theoretic Security Framework as the *market regulator* $\mathcal{F}_{\mathsf{HybridKEM}}$ (Definition 5.6). The key technical contribution is proving that all 15 Cobra methods place the Cobra security market $\mathcal{M}_{\mathsf{Cobra}}$ in equilibrium for the IND-CCA2 good, which by the UC-to-MTSF translation (Definition 5.7) and the Market Merger Theorem (Theorem 5.8) delivers classical UC-realisation composability plus quantitative ask-price bounds, CNF session audits, and unbounded-session security via pinging.

2.4.2 Random Oracle Model

Bellare–Rogaway [11] introduced the Random Oracle Model (ROM), which remains the standard for practical analysis despite theoretical limitations [30]. The Quantum Random Oracle Model (QROM) [22] extends ROM to quantum adversaries; Targhi–Unruh [113] analysed post-quantum KEMs in QROM, and Jiang et al. [63] proved tight Fujisaki–Okamoto security bounds. Shoup [107] formalised game-based proofs. Our 10-round bidding-round chains (Section 6) refine Shoup-style game hops as MTSF price adjustments $\Delta\mathsf{Price}_k$, yielding concrete ask-price bounds suitable for standardisation; the underlying "code-based games" framework is due to Bellare–Rogaway [10].

2.4.3 IND-CCA2 Security

IND-CCA2 security was formalised by Rackoff–Simon [96]; Cramer–Shoup [33, 34] gave the first practical IND-CCA2-secure encryption without random oracles. For KEMs, Hofheinz et al. [58] refined definitions and equivalences, adapted to the quantum setting by Targhi–Unruh [113]. The Fujisaki–Okamoto transform [47, 48] converts CPA-secure encryption to CCA-secure KEMs but requires explicit rejection, revealing decapsulation failures. Implicit rejection (Hofheinz et al. [58], refined by Bindel et al. [20]) eliminates this leakage via pseudorandom failure outputs; all Cobra methods use implicit rejection for constant-time security.

2.5 TLS Integration and Protocol Deployment

2.5.1 TLS 1.3 Architecture and Hybrid Proposals

TLS 1.3 [102] is a major redesign removing legacy algorithms and mandating forward secrecy; key exchange occurs exclusively through ephemeral Diffie–Hellman or KEMs, making post-quantum transition critical. Dowling–Stebila [41] formally analysed TLS 1.2 key exchange; Cremers et al. [35] analysed TLS 1.3 handshake security; Schwabe et al. [104] studied post-quantum TLS performance. For hybrid TLS specifically, Stebila et al. [110, 111] developed the IETF framework for hybrid key exchange in TLS 1.3, restricted to pairwise hybrids (ECDH + one PQC KEM); Kwiatkowski et al. [72] specified X25519Kyber768Draft00; Westerbaan–Wood [119] reported Cloudflare real-world performance; Kampanakis et al. [64] studied post-quantum X.509 certificates; and Sikeridis et al. [108] studied network impact of larger PQC handshakes — all informing our methodology.

2.5.2 Deployment Challenges

Several practical challenges emerged in TLS deployments: certificate chain bloat as a deployment barrier [94] (which we treat as orthogonal to KEM encapsulation), middlebox interference with large TLS messages [112] often requiring protocol-level workarounds, and high-throughput performance constraints [14] — our Cobra analysis (1.2–3.8 ms encapsulation) demonstrates practical TLS feasibility.

2.6 Component KEM Analysis

FrodoKEM (plain LWE). FrodoKEM [80] uses unstructured Learning With Errors for conservative security, building on the original Frodo design [23] with constant-time implementations [4]. Albrecht et al. [3] established lattice-attack complexities; Ducas–van Woerden [43] analysed dual attacks on unstructured LWE. Implementation side-channels have been addressed by Roy–Reparaz [103] (LWE sampling) and Oder et al. [90] (constrained devices).

ML-KEM / CRYSTALS-Kyber (module LWE). Avanzi et al. [7] defined CRYSTALS-Kyber, now FIPS 203. Cross-platform performance is reported in Bos et al. [24]; Langlois–Stehlé [73] analysed Module-LWE foundations. Concrete security estimates against lattice attacks are due to Albrecht et al. [2]; reduction-tightness analysis to Ducas et al. [42]. Implementation security has been studied by Ravi et al. [97] (side channels), Hamburg [55] (constant-time), and Sim et al. [109] (CCA via decryption failures).

HQC (Hamming Quasi-Cyclic codes). HQC [77] achieves IND-CCA2 security using sparse quasi-cyclic codes, extending the IND-CPA design of Melchor et al. [76] with the FO transform. Code-based security analyses include information-set decoding (Sendrier [105]), structured-code security (Canto Torres–Sendrier [114]), and algebraic attacks (Bardet et al. [9]). Decoding-failure side channels have been studied by Vasseur [117] and Guo et al. [53], leading to HQC parameter adjustments.

2.7 Cryptographic Combiners and Robust Security

The theory of cryptographic combiners [57] studies constructions that remain secure if any component is secure. Harnik et al. [56] proved XOR-based key derivation achieves security when either KEM is secure; Dodis–Katz [39] generalised to arbitrary combining functions, showing KDF-based combiners are robust. Fischlin et al. [46] studied concatenation-based combiners for signatures and MACs; Giacon et al. [51] applied combiner theory specifically to key exchange. Our universal security theorem (Theorem 6.1) shows that all 15 Cobra methods are robust

combiners: security holds if any single component KEM is secure, a property derived from KDF-based final combination consistent with Dodis–Katz theory.

2.8 Gap Analysis and Cobra Contributions

Several critical gaps remain despite extensive research: (i) *multi-algorithm compositions* — prior work focuses on 2-algorithm hybrids, while Cobra systematically explores 4-algorithm compositions across 15 distinct methods; (ii) *diverse hardness assumptions* — most proposals combine same-family algorithms, while Cobra combines lattice-based (FrodoKEM, ML-KEM), code-based (HQC), and placeholder (Dummy) for true diversity; (iii) *comprehensive security analysis* — no prior work provides a unified market-theoretic treatment spanning UC composition, ROM concrete bounds, CNF session auditing, and unbounded-session security; (iv) *performance differentiation* across all 15 methods (1.2–3.8 ms range); (v) *practical deployment guidance* — our method-selection algorithm provides concrete guidance based on application requirements, filling a gap in existing theoretical frameworks.

3 Preliminaries and Notation

3.1 Key Encapsulation Mechanism

Definition 3.1 (Key Encapsulation Mechanism)**.** *A KEM $\mathcal{KEM} = (KeyGen, Encaps, Decaps)$ consists of three algorithms:*

- *$KeyGen(1^\lambda) \to (pk, sk)$: Generates public and secret keys on security parameter λ*
- *$Encaps(pk) \to (ct, ss)$: Encapsulates to produce ciphertext and shared secret*
- *$Decaps(ct, sk) \to ss \cup \{\perp\}$: Decapsulates or outputs failure symbol*

3.2 Security Model

Definition 3.2 (IND-CCA2 Security)**.** *A KEM is (t, q_d, ϵ)-IND-CCA2 secure if for all adversaries $\mathcal{A}$ running in time t and making at most q_d decapsulation queries:*

$$Adv^{IND\text{-}CCA2}_{\mathcal{KEM}}(\mathcal{A}) = \left|\Pr[b' = b] - \frac{1}{2}\right| \leq \epsilon$$

in the standard IND-CCA2 game where the adversary must distinguish a real shared secret from random.

3.3 Component KEMs

- $\mathcal{KEM}_F$: FrodoKEM-976 (plain LWE, unstructured lattices, $\epsilon_F \approx 2^{-128}$)
- $\mathcal{KEM}_M$: ML-KEM-768 (module LWE, NIST FIPS 203, $\epsilon_M \approx 2^{-128}$)
- $\mathcal{KEM}_H$: HQC-192 (Hamming Quasi-Cyclic codes, $\epsilon_H \approx 2^{-192}$)
- $\mathcal{KEM}_D$: Dummy KEM (placeholder, $\epsilon_D = 0$)

3.4 Dummy KEM Specification

3.5 Key Derivation Function

The KDF is modeled as a random oracle $H : \{0,1\}^* \to \{0,1\}^{256}$. We instantiate with HKDF-SHA256:

$$\text{KDF}(\text{input}_1 \| \text{input}_2 \| \cdots \| \text{context}) = \text{HKDF-SHA256}(\text{inputs}, \text{context})$$

Algorithm 1 Dummy KEM Specification

function DummyKEM.KeyGen(1^λ)
    $pk_D \xleftarrow{\$} \{0,1\}^{256}$
    $sk_D \xleftarrow{\$} \{0,1\}^{256}$
    return (pk_D, sk_D)
end function

function DummyKEM.Encaps(pk_D)
    $ct_D \xleftarrow{\$} \{0,1\}^{256}$
    $ss_D \xleftarrow{\$} \{0,1\}^{256}$
    return (ct_D, ss_D)
end function

function DummyKEM.Decaps(ct_D, sk_D)
    $ss_D \leftarrow \text{KDF}(sk_D \| ct_D \| \text{“dummy”})$
    return ss_D
end function

3.6 Notation

- $\|$: Concatenation operator
- $\xleftarrow{\$}$: Uniform random sampling
- $\perp$: Failure symbol
- negl(λ): Negligible function in λ
- PPT: Probabilistic polynomial time
- F, M, H, D: Subscripts for FrodoKEM, ML-KEM, HQC, Dummy respectively

4 Complete Specifications of All 15 Methods

4.1 Universal Key Generation

All 15 methods use identical key generation:

Algorithm 2 Cobra KeyGen (Universal for All Methods)

function Cobra.KeyGen(1^λ)
    $(pk_F, sk_F) \leftarrow \mathcal{KEM}_F.\text{KeyGen}(1^\lambda)$
    $(pk_M, sk_M) \leftarrow \mathcal{KEM}_M.\text{KeyGen}(1^\lambda)$
    $(pk_H, sk_H) \leftarrow \mathcal{KEM}_H.\text{KeyGen}(1^\lambda)$
    $(pk_D, sk_D) \leftarrow \mathcal{KEM}_D.\text{KeyGen}(1^\lambda)$
    $pk \leftarrow (pk_F, pk_M, pk_H, pk_D)$
    $sk \leftarrow (sk_F, sk_M, sk_H, sk_D)$
    return (pk, sk)
end function

4.2 Algorithmic Conventions and Universal Design Principles

Before presenting the fifteen method-specific algorithm pairs in Sections 4.2ff., we describe the conventions that every Cobra algorithm obeys. Understanding these shared primitives makes the per-method algorithms much shorter to parse: each method differs only in the *composition topology* of the four component encapsulations and the *intermediate KDF chaining*, while every other algorithmic ingredient is fixed by the conventions below.

Common ingredients. Every METHODi.ENCAPS and METHODi.DECAPS algorithm draws from exactly the following set of operations:

- **Component encapsulation.** $\mathcal{KEM}_X$.Encaps(pk_X) for $X \in \{F, M, H, D\}$ takes the component public key and returns a pair (ct_X, ss_X) where ct_X is the component ciphertext and $ss_X \in \{0,1\}^{256}$ is the component shared secret. Each component also supports an explicit-randomness variant $\mathcal{KEM}_X$.Encaps($pk_X; r_X$) where the internal randomness is supplied externally as $r_X \in \{0,1\}^{256}$; this variant is used by cascading and nested methods to inject the preceding stage's derived seed into the downstream encapsulation. A method that writes $\mathcal{KEM}_X$.Encaps(pk_X; seed$_X$) is thus replaying the same algorithm but with caller-supplied randomness.

- **Component decapsulation.** $\mathcal{KEM}_X$.Decaps(ct_X, sk_X) returns $ss_X \in \{0,1\}^{256}$ if ct_X is a well-formed encapsulation under pk_X, and $\perp$ otherwise. The caller is required to run this primitive in constant time and to handle the $\perp$ case via the implicit-rejection pattern below.

- **Key derivation.** KDF($\cdot$) is a FIPS 203-compatible extract-and-expand function (HKDF-SHA256 in the reference implementation; equivalent constructions such as KMAC or Ascon-XOF may be substituted provided the PRF and collision bounds are preserved). Every KDF call emits a fresh 256-bit output; all intermediate quantities seed$_X$, inter, branch$_L$, branch$_R$, and the final ss are 256-bit strings.

- **Domain separation.** Every KDF invocation includes a *method-identifier string* (`‘‘methodi’’`) and, where applicable, a *stage/role string* (e.g. `‘‘m7-stage1’’`, `‘‘m13-left’’`). These strings are the byte-level implementation of the CNF **kdf-binding** clause φ^{kdf} of Definition 5.9; they guarantee that a chosen-ciphertext adversary cannot transplant a shared-secret output from one method (or one role within a method) into another.

Universal encapsulation skeleton. Every METHODi.ENCAPS can be decomposed into five phases:

1. *Seed generation.* Sample or derive the input randomness for the first stage. In independent-component methods (Methods 2, 5) this phase is trivial; in cascading or nested methods (Methods 1, 6, 11, 13, 14, 15) this phase derives a seed$_X$ from an upstream ss_Y via a KDF call with a role-tagged context.

2. *Component encapsulations.* Invoke $\mathcal{KEM}_X$.Encaps(pk_X [; seed$_X$]) for each component in the order prescribed by the method's composition topology. Parallel components run concurrently on the available execution units; sequential components wait for the preceding stage's ss_Y before proceeding.

3. *Intermediate chaining.* Derive any per-branch intermediates inter, branch$_L$, branch$_R$ required by the topology, each via a KDF call whose context string identifies its role.

4. *Final combination.* Produce the output shared secret as $ss \leftarrow \mathrm{KDF}(ss_F \parallel ss_M \parallel ss_H \parallel ss_D \parallel \text{``}\texttt{method}i\text{''})$, binding *all four* component secrets regardless of composition order. This universality of the final step is what makes Theorem 6.1 hold uniformly across all fifteen methods: the ideal-KDF bidding round $\mathbf{B}_7$ fires whenever *any one* ss_X is uniform.

5. *Ciphertext assembly.* Return $ct = (ct_F, ct_M, ct_H, ct_D)$ concatenated in canonical order (F, M, H, D) along with the shared secret ss.

Universal decapsulation skeleton. Every METHODi.DECAPS follows a *uniform* three-phase structure, regardless of the encapsulation topology:

1. *Parallel component decapsulation with implicit rejection.* For each $i \in \{F, M, H, D\}$, compute $ss'_i \leftarrow \mathcal{KEM}_i.\mathrm{Decaps}(ct_i, sk_i)$. If $ss'_i = \perp$ (invalid component ciphertext), replace ss'_i with $\mathrm{KDF}(sk_i \parallel ct_i \parallel \text{``}\texttt{reject-}i\text{''})$. The substitution is a Fujisaki–Okamoto-style implicit rejection [20, 47, 58]: it returns a pseudorandom value that is indistinguishable from a legitimate ss'_i to any adversary lacking sk_i, thereby preventing timing/behavioural leakage about ciphertext validity.

2. *Intermediate reconstruction.* Recompute the same intermediate KDF values (*inter*, branch_L, etc.) that the encapsulator derived in its chaining phase, using the recovered ss'_i in place of the encapsulator's ss_i. For valid ciphertexts these coincide; for invalid ones the pseudorandom rejection values propagate transparently.

3. *Final combination.* Recompute $ss \leftarrow \mathrm{KDF}(ss'_F \parallel ss'_M \parallel ss'_H \parallel ss'_D \parallel \text{``}\texttt{method}i\text{''})$ and return ss.

Crucially, every Cobra decapsulation runs all four component decapsulations *in parallel* — including those whose corresponding encapsulation was sequential. This is deliberate: the encapsulator's dependency chain is not repeated on the decapsulator's side, because once the ciphertexts are transmitted, each ct_i can be independently decapsulated against sk_i. The parallelism in decapsulation is what makes every method share the same ≈ 1.1 ms measured decapsulation latency in Table 7.2, in contrast to the encapsulation latencies that range from 1.2 ms (Method 2) to 3.8 ms (Method 1).

Robust-combiner invariant. Every algorithm in this section is constructed so that the final KDF call takes *all four* component shared secrets as input, not merely the "last" component of the composition topology. As established formally in Theorem 6.1 and traced through the bidding-round chain $\mathbf{B}_3$–$\mathbf{B}_7$ of Section 6, this invariant is precisely what delivers the robust-combiner property: the hybrid remains IND-CCA2 secure as long as *at least one* of $\{\mathcal{KEM}_F, \mathcal{KEM}_M, \mathcal{KEM}_H, \mathcal{KEM}_D\}$ remains IND-CCA2 secure. A composition that fed only the innermost or last component into the final KDF would lose this property; Cobra's universal final step preserves it by construction.

Dummy-KEM placement. The Dummy KEM $\mathcal{KEM}_D$ (specification in Section 3.4) occupies the same syntactic slot as a real component KEM in every method but contributes no security (by design, $\epsilon_D = 0$ in the ROM-idealised $\mathcal{KEM}_D$ model and $\epsilon_D = 1$ if the placeholder is ever instantiated without replacement). Its role is purely *agility*: it reserves a slot in the ciphertext layout, public-key layout, and KDF context that a future fifth algorithm can substitute into without breaking wire-format or cross-version interoperability. In deployment, operators are expected to replace $\mathcal{KEM}_D$ with a forthcoming NIST candidate (e.g. a Round-4 alternate or a future signature-derived KEM) well before any reliance on its notional security is required.

Notation summary for algorithm blocks. Throughout the fifteen algorithm pairs that follow, we use:

Symbol	Meaning
$pk = (pk_F, pk_M, pk_H, pk_D)$	Concatenated public key
$sk = (sk_F, sk_M, sk_H, sk_D)$	Concatenated secret key
$ct = (ct_F, ct_M, ct_H, ct_D)$	Concatenated ciphertext
ss_X	Component shared secret from $\mathcal{KEM}_X$, $X \in \{F, M, H, D\}$
ss'_X	Decapsulator's recovered secret (may be implicit-rejection output)
ss	Final 256-bit Cobra shared secret
seed_X	KDF-derived randomness fed into $\mathcal{KEM}_X$.Encaps
inter, branch_L, branch_R	Method-specific intermediate 256-bit values
‘‘`methodi`’’	Method-identifier domain-separation string
‘‘`mi-role`’’	Method-and-role-specific domain-separation string

With these conventions fixed, the fifteen algorithm pairs below need only specify each method's unique contribution: the ordering of component encapsulations and the structure of the intermediate-chaining KDF calls.

4.3 Category 1: Fully Ordered Methods

4.3.1 Method 1: Full Cascade ($F \to M \to H \to D$)

Description: Each KEM's shared secret seeds the randomness for the next KEM, creating a sequential chain.

Composition topology. Method 1 is the canonical fully-sequential composition: FrodoKEM runs first, its shared secret ss_F is KDF-derived into seed_M which seeds ML-KEM's encapsulation, whose output ss_M is derived into seed_H seeding HQC, and finally ss_H seeds the Dummy-KEM. The chain order $F \to M \to H \to D$ is deliberate: the algorithmically most conservative component (FrodoKEM's unstructured LWE) anchors the chain, and the agility slot $\mathcal{KEM}_D$ is placed at the tail where it receives contributions from all three substantive components. The final KDF call binds all four shared secrets.

Design rationale. Method 1 maximises *structural* defense-in-depth: an adversary attempting a staged attack that progressively recovers intermediate seeds must compromise components in strict order (F, then M, then H, then D), paying separate cryptanalytic costs at each stage. The critical-path cost is $T_F + T_K + T_M + T_K + T_H + T_K + T_D + T_K \approx 3.80\,\text{ms}$ on the reference platform — the highest of all fifteen methods, and $\approx 3.2\times$ slower than the fully parallel Method 2. This latency penalty buys the strongest possible $\mathsf{DiD}_i^{\text{struct}} = 5$ rating in the Pareto taxonomy of Theorem 7.1.

Algorithm 3 Method 1: Full Cascade - Encapsulation

function Method1.Encaps($pk = (pk_F, pk_M, pk_H, pk_D)$)
  $(ct_F, ss_F) \leftarrow \mathcal{KEM}_F.\text{Encaps}(pk_F)$
  $\text{seed}_M \leftarrow \text{KDF}(ss_F \| \text{“m1-layer2”} \| 1)$
  $(ct_M, ss_M) \leftarrow \mathcal{KEM}_M.\text{Encaps}(pk_M; \text{seed}_M)$
  $\text{seed}_H \leftarrow \text{KDF}(ss_M \| \text{“m1-layer3”} \| 2)$
  $(ct_H, ss_H) \leftarrow \mathcal{KEM}_H.\text{Encaps}(pk_H; \text{seed}_H)$
  $\text{seed}_D \leftarrow \text{KDF}(ss_H \| \text{“m1-layer4”} \| 3)$
  $(ct_D, ss_D) \leftarrow \mathcal{KEM}_D.\text{Encaps}(pk_D; \text{seed}_D)$
  $ss \leftarrow \text{KDF}(ss_F \| ss_M \| ss_H \| ss_D \| \text{“method1”})$
  $ct \leftarrow (ct_F, ct_M, ct_H, ct_D)$
  return (ct, ss)
end function

Algorithm 4 Method 1: Full Cascade - Decapsulation

function METHOD1.DECAPS($ct = (ct_F, ct_M, ct_H, ct_D), sk = (sk_F, sk_M, sk_H, sk_D)$)
  $ss'_F \leftarrow \mathcal{KEM}_F$.Decaps$(ct_F, sk_F)$
  if $ss'_F = \perp$ **then**
    $ss'_F \leftarrow$ KDF$(sk_F \| ct_F \|$ "reject-F")
  end if
  $ss'_M \leftarrow \mathcal{KEM}_M$.Decaps$(ct_M, sk_M)$
  if $ss'_M = \perp$ **then**
    $ss'_M \leftarrow$ KDF$(sk_M \| ct_M \|$ "reject-M")
  end if
  $ss'_H \leftarrow \mathcal{KEM}_H$.Decaps$(ct_H, sk_H)$
  if $ss'_H = \perp$ **then**
    $ss'_H \leftarrow$ KDF$(sk_H \| ct_H \|$ "reject-H")
  end if
  $ss'_D \leftarrow \mathcal{KEM}_D$.Decaps$(ct_D, sk_D)$
  if $ss'_D = \perp$ **then**
    $ss'_D \leftarrow$ KDF$(sk_D \| ct_D \|$ "reject-D")
  end if
  $ss \leftarrow$ KDF$(ss'_F \| ss'_M \| ss'_H \| ss'_D \|$ "method1")
  return ss
end function

Security mapping to bidding rounds. Method 1 is the reference case for the fully detailed 10-round bidding-round proof of §6. Rounds $\mathbf{B}_3, \mathbf{B}_4, \mathbf{B}_5, \mathbf{B}_6$ substitute ss_F, ss_M, ss_H, ss_D in order; as the cascade ensures each downstream seed depends on the upstream component, any successful substitution round $\mathbf{B}_k$ propagates uniformity from its position all the way to the final KDF input. The universal bound of Theorem 6.1 follows.

Deployment profile. Recommended for maximum-defense-in-depth settings (government classified systems, financial settlement infrastructure, long-lived archival encryption) where the latency penalty is acceptable. Not recommended for mainstream TLS termination, where Method 2 delivers the same asymptotic security with 3.2× better throughput.

4.3.2 Method 2: Full Parallel ($F \| M \| H \| D$)

Description: All four KEMs operate independently in parallel.

Composition topology. Method 2 is the archetypal parallel composition: the four component encapsulations $\mathcal{KEM}_F, \mathcal{KEM}_M, \mathcal{KEM}_H, \mathcal{KEM}_D$ execute concurrently on independent execution units, producing their respective shared secrets in a single critical-path slot. A single final KDF call combines the four secrets into ss. No intermediate seed derivations or chain dependencies exist.

Design rationale. Method 2 is the maximum-throughput archetype of the Pareto-optimal set identified by Theorem 7.1: it achieves the theoretical minimum critical-path length $L_i = \max_X T_X + T_K \approx 1.20$ ms, which is a lower bound no method can beat. At parallelism budget $P \geq 4$ the method's efficiency is $\eta \approx 80\%$ (Table 6), limited only by the trailing sequential KDF call. The structural defense-in-depth score is $\mathsf{DiD}^{\text{struct}} = 1$: a staged adversary that recovers one component's secret gains no information about the others, but also pays no incremental cost across components — each is attacked independently.

Security mapping to bidding rounds. Because the four component encapsulations are mutually independent, the substitution rounds $\mathbf{B}_3, \mathbf{B}_4, \mathbf{B}_5, \mathbf{B}_6$ of the Method 2 chain can be analysed in any order. Any single successful substitution round makes one of the final KDF's four inputs uniform, and the ideal-KDF round $\mathbf{B}_7$ fires immediately. The universal bound

Algorithm 5 Method 2: Full Parallel - Encapsulation

function METHOD2.ENCAPS($pk = (pk_F, pk_M, pk_H, pk_D)$)
  $(ct_F, ss_F) \leftarrow \mathcal{KEM}_F$.Encaps($pk_F$)
  $(ct_M, ss_M) \leftarrow \mathcal{KEM}_M$.Encaps($pk_M$)
  $(ct_H, ss_H) \leftarrow \mathcal{KEM}_H$.Encaps($pk_H$)
  $(ct_D, ss_D) \leftarrow \mathcal{KEM}_D$.Encaps($pk_D$)
  $ss \leftarrow \text{KDF}(ss_F \| ss_M \| ss_H \| ss_D \| \text{“method2”})$
  $ct \leftarrow (ct_F, ct_M, ct_H, ct_D)$
  return (ct, ss)
end function

Algorithm 6 Method 2: Full Parallel - Decapsulation

function METHOD2.DECAPS(ct, sk)
  $ss'_F \leftarrow \mathcal{KEM}_F$.Decaps($ct_F, sk_F$)
  if $ss'_F = \perp$ **then**
    $ss'_F \leftarrow \text{KDF}(sk_F \| ct_F \| \text{“reject-F”})$
  end if
  $ss'_M \leftarrow \mathcal{KEM}_M$.Decaps($ct_M, sk_M$)
  if $ss'_M = \perp$ **then**
    $ss'_M \leftarrow \text{KDF}(sk_M \| ct_M \| \text{“reject-M”})$
  end if
  $ss'_H \leftarrow \mathcal{KEM}_H$.Decaps($ct_H, sk_H$)
  if $ss'_H = \perp$ **then**
    $ss'_H \leftarrow \text{KDF}(sk_H \| ct_H \| \text{“reject-H”})$
  end if
  $ss'_D \leftarrow \mathcal{KEM}_D$.Decaps($ct_D, sk_D$)
  if $ss'_D = \perp$ **then**
    $ss'_D \leftarrow \text{KDF}(sk_D \| ct_D \| \text{“reject-D”})$
  end if
  $ss \leftarrow \text{KDF}(ss'_F \| ss'_M \| ss'_H \| ss'_D \| \text{“method2”})$
  return ss
end function

follows with no sequencing overhead in the reductions.

Deployment profile. Recommended as the default for the majority of TLS deployments — high-throughput web servers, API gateways, cloud load balancers, and latency-sensitive endpoints. The 1.2 ms encapsulation and ~ 909 handshakes/second per-core throughput suffice for all three industry case studies' performance targets.

4.4 Category 2: Two-Stage Methods

4.4.1 Method 3: Two-Stage Forward $((F\|M) \to (H\|D))$

Description: First pair in parallel, their combined output seeds second pair in parallel.

Composition topology. Method 3 is the canonical two-stage forward composition: the lattice-based components $\mathcal{KEM}_F$ and $\mathcal{KEM}_M$ run in parallel in stage 1, their outputs are KDF-mixed into an intermediate inter from which two independent seeds $\text{seed}_H, \text{seed}_D$ are derived, and the code-based and agility-slot components $\mathcal{KEM}_H, \mathcal{KEM}_D$ run in parallel in stage 2 on those derived seeds. The final KDF call binds all four shared secrets.

Design rationale. The two-stage structure yields an explicit audit-level separation between the lattice-family components (stage 1) and the non-lattice-family components (stage 2), which is valuable for compliance regimes (BSI TR-02102, FIPS 140-3) that prefer documented evidence of algorithmic diversity across stages. The critical-path cost is $\max(T_F, T_M) + T_K + \max(T_H, T_D) + T_K \approx 1.90$ ms, roughly 58% longer than Method 2 but providing the stage-separation evidence Method 2 does not.

Algorithm 7 Method 3: Two-Stage Forward - Encapsulation

function METHOD3.ENCAPS(pk)
  $(ct_F, ss_F) \leftarrow \mathcal{KEM}_F.\text{Encaps}(pk_F)$
  $(ct_M, ss_M) \leftarrow \mathcal{KEM}_M.\text{Encaps}(pk_M)$
  $\text{inter} \leftarrow \text{KDF}(ss_F \| ss_M \| \text{“m3-stage1”})$
  $\text{seed}_H \leftarrow \text{KDF}(\text{inter} \| \text{“m3-seed-H”})$
  $\text{seed}_D \leftarrow \text{KDF}(\text{inter} \| \text{“m3-seed-D”})$
  $(ct_H, ss_H) \leftarrow \mathcal{KEM}_H.\text{Encaps}(pk_H; \text{seed}_H)$
  $(ct_D, ss_D) \leftarrow \mathcal{KEM}_D.\text{Encaps}(pk_D; \text{seed}_D)$
  $ss \leftarrow \text{KDF}(ss_F \| ss_M \| ss_H \| ss_D \| \text{“method3”})$
  return $((ct_F, ct_M, ct_H, ct_D), ss)$
end function

Algorithm 8 Method 3: Two-Stage Forward - Decapsulation

function METHOD3.DECAPS(ct, sk)
  for $i \in \{F, M, H, D\}$ **do**
    $ss'_i \leftarrow \text{KEM}_i.\text{Decaps}(ct_i, sk_i)$
    if $ss'_i = \perp$ **then**
      $ss'_i \leftarrow \text{KDF}(sk_i \| ct_i \| \text{“reject-”} \| i)$
    end if
  end for
  $ss \leftarrow \text{KDF}(ss'_F \| ss'_M \| ss'_H \| ss'_D \| \text{“method3”})$
  return ss
end function

Security mapping to bidding rounds. Method 3's bidding-round chain splits on stage. If $\mathcal{KEM}_F$ or $\mathcal{KEM}_M$ (stage 1) is secure, substitution rounds $\mathbf{B}_3$ or $\mathbf{B}_4$ respectively randomise ss_F or ss_M, which propagates through inter into both stage 2 seeds and ultimately into all

four final-KDF inputs. If $\mathcal{KEM}_H$ or $\mathcal{KEM}_D$ (stage 2) is secure, rounds $\mathbf{B}_5$ or $\mathbf{B}_6$ randomise that component's ss directly; the final-KDF input is uniform in one slot and round $\mathbf{B}_7$ fires. Universal bound of Theorem 6.1 follows.

Deployment profile. Recommended for compliance-driven deployments (healthcare HIPAA, BSI TR-02102 hybrid, ETSI TS 103 744) where the auditable stage separation is the primary design constraint and the ~ 1.9 ms encapsulation latency fits the deployment's budget.

4.4.2 Method 4: Two-Stage Reverse $((H\|D) \to (F\|M))$

Composition topology. Method 4 is the mirror of Method 3: the code-based and agility-slot components $\mathcal{KEM}_H, \mathcal{KEM}_D$ run in parallel in stage 1, their outputs are KDF-mixed into an intermediate from which seeds $\text{seed}_F, \text{seed}_M$ are derived, and the lattice-based components $\mathcal{KEM}_F, \mathcal{KEM}_M$ run in parallel in stage 2 on those derived seeds. The final KDF call binds all four secrets.

Design rationale. Method 4 is the reverse-ordered two-stage archetype. Placing the smaller-output code-based component $\mathcal{KEM}_H$ and the agility slot $\mathcal{KEM}_D$ in stage 1 shortens the first-stage critical path and allows the heavier lattice components $\mathcal{KEM}_F, \mathcal{KEM}_M$ to proceed on seeds already carrying code-based entropy. Critical-path cost is $\max(T_H, T_D) + T_K + \max(T_F, T_M) + T_K \approx 1.90$ ms, identical to Method 3 up to stage-order symmetry. Choice between Methods 3 and 4 is typically governed by compliance preferences for which family is documented first.

Algorithm 9 Method 4: Two-Stage Reverse - Encapsulation

function METHOD4.ENCAPS(pk)
  $(ct_H, ss_H) \leftarrow \mathcal{KEM}_H.\text{Encaps}(pk_H)$
  $(ct_D, ss_D) \leftarrow \mathcal{KEM}_D.\text{Encaps}(pk_D)$
  $\text{inter} \leftarrow \text{KDF}(ss_H \| ss_D \| \text{“m4-stage1”})$
  $\text{seed}_F \leftarrow \text{KDF}(\text{inter} \| \text{“m4-seed-F”})$
  $\text{seed}_M \leftarrow \text{KDF}(\text{inter} \| \text{“m4-seed-M”})$
  $(ct_F, ss_F) \leftarrow \mathcal{KEM}_F.\text{Encaps}(pk_F; \text{seed}_F)$
  $(ct_M, ss_M) \leftarrow \mathcal{KEM}_M.\text{Encaps}(pk_M; \text{seed}_M)$
  $ss \leftarrow \text{KDF}(ss_F \| ss_M \| ss_H \| ss_D \| \text{“method4”})$
  return $((ct_F, ct_M, ct_H, ct_D), ss)$
end function

Algorithm 10 Method 4: Two-Stage Reverse - Decapsulation

function METHOD4.DECAPS(ct, sk)
  for $i \in \{F, M, H, D\}$ **do**
    $ss'_i \leftarrow \text{KEM}_i.\text{Decaps}(ct_i, sk_i)$
    if $ss'_i = \perp$ **then**
      $ss'_i \leftarrow \text{KDF}(sk_i \| ct_i \| \text{“reject-”} \| i)$
    end if
  end for
  $ss \leftarrow \text{KDF}(ss'_F \| ss'_M \| ss'_H \| ss'_D \| \text{“method4”})$
  return ss
end function

Security mapping to bidding rounds. Symmetric to Method 3 with stage order reversed. A successful substitution round in any of $\mathbf{B}_3$–$\mathbf{B}_6$ randomises the corresponding ss_X; downstream seed derivations propagate uniformity, and the universal bound follows.

Deployment profile. Chosen over Method 3 when the operator prefers to document the code-based component's security first in the audit trail, or when the stage 1 output is required to feed external entropy-mixing systems before the lattice computation begins.

4.4.3 Method 5: Independent Pairs $((F\|M)\|(H\|D))$

Composition topology. Method 5 is structurally unique among the two-stage family: the two pairs $(F\|M)$ and $(H\|D)$ are *mutually independent* — there is no stage 1 $\rightarrow$ stage 2 seed derivation. All four components run in parallel, and the final KDF call aggregates directly. The wire format remains identical to Methods 2–4 (four concatenated component ciphertexts).

Design rationale. By removing the inter-pair seed-derivation chain of Method 3 while retaining the auditable pair-structure of the two-stage family, Method 5 achieves the same critical-path cost as Method 2 (roughly 1.20 ms) while offering a cleaner structural separation between lattice and non-lattice pairs. It is the balanced general-purpose archetype of the Pareto-optimal set in Theorem 7.1: max-throughput like Method 2, with an audit-friendly pair structure like Method 3.

Algorithm 11 Method 5: Independent Pairs - Encapsulation

function METHOD5.ENCAPS(pk)
  $(ct_F, ss_F) \leftarrow \mathcal{KEM}_F$.Encaps($pk_F$)
  $(ct_M, ss_M) \leftarrow \mathcal{KEM}_M$.Encaps($pk_M$)
  $(ct_H, ss_H) \leftarrow \mathcal{KEM}_H$.Encaps($pk_H$)
  $(ct_D, ss_D) \leftarrow \mathcal{KEM}_D$.Encaps($pk_D$)
  $ss \leftarrow \text{KDF}(ss_F\|ss_M\|ss_H\|ss_D\|\text{“method5”})$
  return $((ct_F, ct_M, ct_H, ct_D), ss)$
end function

Algorithm 12 Method 5: Independent Pairs - Decapsulation

function METHOD5.DECAPS(ct, sk)
  for $i \in \{F, M, H, D\}$ **do**
    $ss'_i \leftarrow \text{KEM}_i.\text{Decaps}(ct_i, sk_i)$
    if $ss'_i = \perp$ **then**
      $ss'_i \leftarrow \text{KDF}(sk_i\|ct_i\|\text{“reject-”}\|i)$
    end if
  end for
  $ss \leftarrow \text{KDF}(ss'_F\|ss'_M\|ss'_H\|ss'_D\|\text{“method5”})$
  return ss
end function

Security mapping to bidding rounds. Because the two pairs are independent, substitution rounds $\mathbf{B}_3, \mathbf{B}_4, \mathbf{B}_5, \mathbf{B}_6$ may be analysed in any order. Each round independently randomises its component's ss_X; the final-KDF input becomes uniform in that slot and round $\mathbf{B}_7$ fires. Reductions have no sequencing overhead — cleanest of the two-stage family.

Deployment profile. The balanced general-purpose archetype of Theorem 7.1's Pareto set: recommended whenever Method 2's speed is desired together with Method 3's audit-friendly pair structure.

4.4.4 Method 6: Sequential Pairs $((F \rightarrow M) \rightarrow (H \rightarrow D))$

Composition topology. Method 6 is the *sequential-pairs* composition: the pair $(F \rightarrow M)$ runs as a miniature cascade in stage 1 (FrodoKEM first, its output seeds ML-KEM), and stage 2

is another miniature cascade ($H \to D$) that runs only after stage 1 has completed and its output has been KDF-mixed into a stage 2 entry seed. The final KDF binds all four secrets.

Design rationale. Method 6 is the maximum-defense-in-depth two-stage archetype: the adversary faces two cascaded sub-chains that are themselves chained, forcing a staged attack to traverse the lattice-family sub-chain before even beginning the code-based sub-chain. The critical-path cost is $T_F + T_K + T_M + T_K + T_H + T_K + T_D + T_K \approx 2.60\,\text{ms}$, slightly less than the full-cascade Method 1 because intra-pair components can share seed-derivation KDF calls. Structural DiD score $\mathsf{DiD}^{\text{struct}} = 4$ (just below Method 1's 5).

Algorithm 13 Method 6: Sequential Pairs - Encapsulation

function METHOD6.ENCAPS(pk)
  $(ct_F, ss_F) \leftarrow \mathcal{KEM}_F.\text{Encaps}(pk_F)$
  $\text{seed}_M \leftarrow \text{KDF}(ss_F \| \text{“m6-pair1”})$
  $(ct_M, ss_M) \leftarrow \mathcal{KEM}_M.\text{Encaps}(pk_M; \text{seed}_M)$
  $\text{inter}_1 \leftarrow \text{KDF}(ss_F \| ss_M \| \text{“m6-inter1”})$

  $(ct_H, ss_H) \leftarrow \mathcal{KEM}_H.\text{Encaps}(pk_H)$
  $\text{seed}_D \leftarrow \text{KDF}(ss_H \| \text{“m6-pair2”})$
  $(ct_D, ss_D) \leftarrow \mathcal{KEM}_D.\text{Encaps}(pk_D; \text{seed}_D)$
  $\text{inter}_2 \leftarrow \text{KDF}(ss_H \| ss_D \| \text{“m6-inter2”})$

  $ss \leftarrow \text{KDF}(\text{inter}_1 \| \text{inter}_2 \| \text{“method6”})$
  return $((ct_F, ct_M, ct_H, ct_D), ss)$
end function

Algorithm 14 Method 6: Sequential Pairs - Decapsulation

function METHOD6.DECAPS(ct, sk)
  for $i \in \{F, M, H, D\}$ **do**
    $ss'_i \leftarrow \text{KEM}_i.\text{Decaps}(ct_i, sk_i)$
    if $ss'_i = \perp$ **then**
      $ss'_i \leftarrow \text{KDF}(sk_i \| ct_i \| \text{“reject-”} \| i)$
    end if
  end for
  $\text{inter}_1 \leftarrow \text{KDF}(ss'_F \| ss'_M \| \text{“m6-inter1”})$
  $\text{inter}_2 \leftarrow \text{KDF}(ss'_H \| ss'_D \| \text{“m6-inter2”})$
  $ss \leftarrow \text{KDF}(\text{inter}_1 \| \text{inter}_2 \| \text{“method6”})$
  return ss
end function

Security mapping to bidding rounds. The sequential-pairs topology means $\mathbf{B}_3$ (substitute ss_F) propagates to the lattice pair's mix; $\mathbf{B}_4$ (substitute ss_M) requires tracking the $\mathcal{KEM}_F$-derived seed in the reduction. Stage 2 substitutions $\mathbf{B}_5, \mathbf{B}_6$ depend on stage 1's full completion; each successful round randomises its component's ss_X and propagates through the final KDF. Universal bound follows.

Deployment profile. Recommended for high-security deployments (government SECRET, financial settlement) where the latency budget permits $\sim$ 2.6 ms and maximum structural defense-in-depth short of full cascade is desired.

4.4.5 Method 7: Two-Stage Cross $((F\|H) \to (M\|D))$

Composition topology. Method 7 reorganises the two-stage structure by *cross-pairing* components across algorithmic families. Stage 1 pairs $(\mathcal{KEM}_F, \mathcal{KEM}_H)$ — one lattice and one code-based — in parallel; stage 2 pairs the remaining $(\mathcal{KEM}_M, \mathcal{KEM}_D)$ — another lattice plus the agility slot — in parallel on seeds derived from stage 1's mixed output.

Design rationale. The cross-pairing design ensures that *each stage* contains at least one component from the lattice family and at least one from a distinct family (code-based in stage 1, agility-slot in stage 2), distributing family diversity across the composition timeline. This is attractive for compliance regimes that prefer evidence of family diversity at every stage rather than purely at the top level. Critical-path cost is $\max(T_F, T_H) + T_K + \max(T_M, T_D) + T_K \approx 1.90\,\text{ms}$.

Algorithm 15 Method 7: Two-Stage Cross - Encapsulation

function METHOD7.ENCAPS(pk)
  $(ct_F, ss_F) \leftarrow \mathcal{KEM}_F.\text{Encaps}(pk_F)$
  $(ct_H, ss_H) \leftarrow \mathcal{KEM}_H.\text{Encaps}(pk_H)$
  $\text{inter} \leftarrow \text{KDF}(ss_F \| ss_H \| \text{“m7-stage1”})$
  $\text{seed}_M \leftarrow \text{KDF}(\text{inter} \| \text{“m7-seed-M”})$
  $\text{seed}_D \leftarrow \text{KDF}(\text{inter} \| \text{“m7-seed-D”})$
  $(ct_M, ss_M) \leftarrow \mathcal{KEM}_M.\text{Encaps}(pk_M; \text{seed}_M)$
  $(ct_D, ss_D) \leftarrow \mathcal{KEM}_D.\text{Encaps}(pk_D; \text{seed}_D)$
  $ss \leftarrow \text{KDF}(ss_F \| ss_M \| ss_H \| ss_D \| \text{“method7”})$
  return $((ct_F, ct_M, ct_H, ct_D), ss)$
end function

Algorithm 16 Method 7: Two-Stage Cross - Decapsulation

function METHOD7.DECAPS(ct, sk)
  for $i \in \{F, M, H, D\}$ **do**
    $ss'_i \leftarrow \text{KEM}_i.\text{Decaps}(ct_i, sk_i)$
    if $ss'_i = \perp$ **then**
      $ss'_i \leftarrow \text{KDF}(sk_i \| ct_i \| \text{“reject-”} \| i)$
    end if
  end for
  $ss \leftarrow \text{KDF}(ss'_F \| ss'_M \| ss'_H \| ss'_D \| \text{“method7”})$
  return ss
end function

Security mapping to bidding rounds. The cross-pairing introduces no sequencing overhead in the security proof beyond Method 3's: each substitution round $\mathbf{B}_k$ randomises its component's ss_X, uniformity propagates through the stage's mix into stage 2's seeds or into the final KDF directly, and round $\mathbf{B}_7$ fires.

Deployment profile. Chosen when the compliance regime requires evidence that *each stage* contains components from at least two distinct algorithmic families (as opposed to the family-per-stage arrangement of Methods 3 and 4).

4.4.6 Method 8: Alternate Cross $((F\|D) \to (M\|H))$

Composition topology. Method 8 is the alternative cross-pairing: stage 1 pairs $(\mathcal{KEM}_F, \mathcal{KEM}_D)$ — lattice plus agility slot — in parallel, and stage 2 pairs $(\mathcal{KEM}_M, \mathcal{KEM}_H)$ — lattice plus code-based — in parallel on stage 1-derived seeds. Final KDF binds all four.

Design rationale. Method 8 places the agility slot $\mathcal{KEM}_D$ in stage 1 rather than stage 2 (contrast Method 7), meaning a future replacement of $\mathcal{KEM}_D$ alters the first-stage entropy feeding stage 2. This matters when the operator plans a specific migration path in which $\mathcal{KEM}_D$ will be replaced before stage 2's components are reviewed. Critical-path cost $\max(T_F, T_D) + T_K + \max(T_M, T_H) + T_K \approx 1.90\,\text{ms}$, matching Method 7 up to stage-order choice.

Algorithm 17 Method 8: Alternate Cross - Encapsulation

function METHOD8.ENCAPS(pk)
  $(ct_F, ss_F) \leftarrow \mathcal{KEM}_F.\text{Encaps}(pk_F)$
  $(ct_D, ss_D) \leftarrow \mathcal{KEM}_D.\text{Encaps}(pk_D)$
  $\text{inter} \leftarrow \text{KDF}(ss_F \| ss_D \| \text{“m8-stage1”})$
  $\text{seed}_M \leftarrow \text{KDF}(\text{inter} \| \text{“m8-seed-M”})$
  $\text{seed}_H \leftarrow \text{KDF}(\text{inter} \| \text{“m8-seed-H”})$
  $(ct_M, ss_M) \leftarrow \mathcal{KEM}_M.\text{Encaps}(pk_M; \text{seed}_M)$
  $(ct_H, ss_H) \leftarrow \mathcal{KEM}_H.\text{Encaps}(pk_H; \text{seed}_H)$
  $ss \leftarrow \text{KDF}(ss_F \| ss_M \| ss_H \| ss_D \| \text{“method8”})$
  return $((ct_F, ct_M, ct_H, ct_D), ss)$
end function

Algorithm 18 Method 8: Alternate Cross - Decapsulation

function METHOD8.DECAPS(ct, sk)
  for $i \in \{F, M, H, D\}$ **do**
    $ss'_i \leftarrow \text{KEM}_i.\text{Decaps}(ct_i, sk_i)$
    if $ss'_i = \perp$ **then**
      $ss'_i \leftarrow \text{KDF}(sk_i \| ct_i \| \text{“reject-”} \| i)$
    end if
  end for
  $ss \leftarrow \text{KDF}(ss'_F \| ss'_M \| ss'_H \| ss'_D \| \text{“method8”})$
  return ss
end function

Security mapping to bidding rounds. Analogous to Method 7 with the alternate cross. The substitution rounds $\mathbf{B}_3$–$\mathbf{B}_6$ target $\{\mathcal{KEM}_F, \mathcal{KEM}_D, \mathcal{KEM}_M, \mathcal{KEM}_H\}$ in stage-order; each round's uniformity propagates through the stage mix or directly into the final KDF as in Method 7.

Deployment profile. Chosen when the planned $\mathcal{KEM}_D$ replacement path requires $\mathcal{KEM}_D$ to be in stage 1 (e.g. because the replacement algorithm will be validated before the other stage 2 components are reviewed in the operator's compliance roadmap).

4.5 Category 3: Three-One Methods

4.5.1 Method 9: Three Parallel First ($F \to (M \| H \| D)$)

Composition topology. Method 9 is the first of the three-one family: FrodoKEM runs first as a singleton, its shared secret ss_F is KDF-expanded into three independent seeds $\text{seed}_M, \text{seed}_H, \text{seed}_D$, and the remaining three components $\mathcal{KEM}_M, \mathcal{KEM}_H, \mathcal{KEM}_D$ run in parallel in stage 2 on those seeds. Final KDF binds all four.

Design rationale. Method 9 uses FrodoKEM's conservative unstructured-LWE security as an *entropy anchor* for the remaining three components. The three stage 2 encapsulations all depend on FrodoKEM's output; a successful cryptanalysis of the hybrid must therefore either break FrodoKEM directly or break one of the three downstream components whose randomness

is already mixed with ss_F. The critical-path cost is $T_F + T_K + \max(T_M, T_H, T_D) + T_K \approx 1.70\,\text{ms}$, the fastest of the three-one family because FrodoKEM is the single upstream component.

Algorithm 19 Method 9: Three Parallel First - Encapsulation

function METHOD9.ENCAPS(pk)
  $(ct_F, ss_F) \leftarrow \mathcal{KEM}_F.\text{Encaps}(pk_F)$
  $\text{seed}_M \leftarrow \text{KDF}(ss_F \| \text{“m9-seed-M”})$
  $\text{seed}_H \leftarrow \text{KDF}(ss_F \| \text{“m9-seed-H”})$
  $\text{seed}_D \leftarrow \text{KDF}(ss_F \| \text{“m9-seed-D”})$
  $(ct_M, ss_M) \leftarrow \mathcal{KEM}_M.\text{Encaps}(pk_M; \text{seed}_M)$
  $(ct_H, ss_H) \leftarrow \mathcal{KEM}_H.\text{Encaps}(pk_H; \text{seed}_H)$
  $(ct_D, ss_D) \leftarrow \mathcal{KEM}_D.\text{Encaps}(pk_D; \text{seed}_D)$
  $ss \leftarrow \text{KDF}(ss_F \| ss_M \| ss_H \| ss_D \| \text{“method9”})$
  return $((ct_F, ct_M, ct_H, ct_D), ss)$
end function

Algorithm 20 Method 9: Three Parallel First - Decapsulation

function METHOD9.DECAPS(ct, sk)
  for $i \in \{F, M, H, D\}$ **do**
    $ss'_i \leftarrow \text{KEM}_i.\text{Decaps}(ct_i, sk_i)$
    if $ss'_i = \perp$ **then**
      $ss'_i \leftarrow \text{KDF}(sk_i \| ct_i \| \text{“reject-”} \| i)$
    end if
  end for
  $ss \leftarrow \text{KDF}(ss'_F \| ss'_M \| ss'_H \| ss'_D \| \text{“method9”})$
  return ss
end function

Security mapping to bidding rounds. The singleton-anchor topology yields a two-way case split. If $\mathcal{KEM}_F$ is secure, $\mathbf{B}_3$ substitutes $ss_F \to r_F$, randomising all three stage 2 seeds simultaneously; all three stage 2 encapsulations produce uniform outputs, and the final KDF is uniform in three of its four inputs. If any of $\{\mathcal{KEM}_M, \mathcal{KEM}_H, \mathcal{KEM}_D\}$ is secure, the corresponding round $\mathbf{B}_4, \mathbf{B}_5, \mathbf{B}_6$ substitutes that component's ss_X; one of the final-KDF inputs becomes uniform and $\mathbf{B}_7$ fires. Universal bound follows via Lemma 5.5.

Deployment profile. Recommended when FrodoKEM's conservative security is the central assumption and the operator wants its output to anchor the randomness of all downstream components. The fastest three-one family method at $\sim 1.7\,\text{ms}$.

4.5.2 Method 10: Three Parallel Last $((F\|M\|H) \to D)$

Composition topology. Method 10 is the mirror-image of Method 9. The three substantive post-quantum components $\mathcal{KEM}_F$, $\mathcal{KEM}_M$, $\mathcal{KEM}_H$ execute in *parallel* on the encapsulator's available execution units, producing (ss_F, ss_M, ss_H) within one critical-path slot. These three secrets are then mixed through an intermediate KDF call to produce seed_D, which is used to derive the Dummy KEM's randomness; $\mathcal{KEM}_D$ runs on a second critical-path slot. The final shared secret binds all four component outputs through the universal combiner.

Design rationale. This topology is optimised for the common case in which $\mathcal{KEM}_D$ will later be replaced by a genuine fifth algorithm (see the Dummy-KEM placement discussion in §4.2). Placing the agility slot *after* the three PQC components means the future replacement algorithm inherits randomness already mixed from all three distinct PQC families (lattice-unstructured, lattice-module, and code-based), maximising the independence of its input even

before its own internal sampling. The critical-path cost is $\max(T_F, T_M, T_H) + T_{\text{KDF}} + T_D$ — roughly 1.6 ms on the reference platform — competitive with the fully parallel Method 2 but with a cleaner structural argument for future-component binding.

Algorithm 21 Method 10: Three Parallel Last - Encapsulation

function METHOD10.ENCAPS(pk)
  $(ct_F, ss_F) \leftarrow \mathcal{KEM}_F.\text{Encaps}(pk_F)$
  $(ct_M, ss_M) \leftarrow \mathcal{KEM}_M.\text{Encaps}(pk_M)$
  $(ct_H, ss_H) \leftarrow \mathcal{KEM}_H.\text{Encaps}(pk_H)$
  $\text{inter} \leftarrow \text{KDF}(ss_F \| ss_M \| ss_H \| \text{“m10-stage1”})$
  $\text{seed}_D \leftarrow \text{KDF}(\text{inter} \| \text{“m10-seed-D”})$
  $(ct_D, ss_D) \leftarrow \mathcal{KEM}_D.\text{Encaps}(pk_D; \text{seed}_D)$
  $ss \leftarrow \text{KDF}(ss_F \| ss_M \| ss_H \| ss_D \| \text{“method10”})$
  return $((ct_F, ct_M, ct_H, ct_D), ss)$
end function

Algorithm 22 Method 10: Three Parallel Last - Decapsulation

function METHOD10.DECAPS(ct, sk)
  for $i \in \{F, M, H, D\}$ **do**
    $ss'_i \leftarrow \text{KEM}_i.\text{Decaps}(ct_i, sk_i)$
    if $ss'_i = \perp$ **then**
      $ss'_i \leftarrow \text{KDF}(sk_i \| ct_i \| \text{“reject-”} \| i)$
    end if
  end for
  $ss \leftarrow \text{KDF}(ss'_F \| ss'_M \| ss'_H \| ss'_D \| \text{“method10”})$
  return ss
end function

Security mapping to bidding rounds. Within the chain $\mathbf{B}_0, \ldots, \mathbf{B}_{10}$ of Section 6, Method 10's distinctive rounds are $\mathbf{B}_3$–$\mathbf{B}_5$ (parallel substitution of ss_F, ss_M, ss_H) followed by $\mathbf{B}_6$ (Dummy-KEM substitution). Because the three parallel components are mutually independent, the three substitution bids can be analysed in any order; a buyer that distinguishes $\mathbf{B}_k$ from $\mathbf{B}_{k+1}$ for any $k \in \{3, 4, 5\}$ yields a direct IND-CCA2 distinguisher against $\mathcal{KEM}_X$ with no stage-dependent reduction overhead. Once any of ss_F, ss_M, ss_H is replaced by uniform randomness, inter becomes uniform, seed_D becomes uniform, $\mathcal{KEM}_D$'s output inherits uniformity, and the ideal-KDF round $\mathbf{B}_7$ fires. The method therefore achieves the universal bound $\min\{\epsilon_F, \epsilon_M, \epsilon_H, \epsilon_D\} + (q_H + q_D)^2/2^{257} + q_D/2^{256}$ of Theorem 6.1.

Deployment profile. Recommended for high-throughput TLS 1.3 deployments that value a clear, future-proof structural slot for Dummy-KEM replacement; a natural choice alongside Method 2 for the latency-critical case studies of §10.

4.5.3 Method 11: Three Sequential Middle ($F \to (M \to H \to D)$)

Composition topology. Method 11 is the most rigidly sequential of the three-one family. The FrodoKEM component anchors the chain in the unstructured-LWE family and runs first. Its shared secret ss_F is then held aside for the final combination step, while a separate three-stage sequential subchain runs $\mathcal{KEM}_M \to \mathcal{KEM}_H \to \mathcal{KEM}_D$: each component's output is used (after KDF-chained domain-separation) as the randomness for the next component's encapsulation. The three subchain secrets are combined into an intermediate quantity inter, which is itself combined with the held-aside ss_F in the final KDF call.

Design rationale. This topology provides the *strongest hybrid-cascade defense-in-depth* of any three-one method: an adversary cannot compromise the later-stage components (HQC, Dummy) without also traversing ML-KEM's internal state, nor can they sidestep FrodoKEM's contribution because ss_F enters the final KDF directly. The structural separation between the anchor component $\mathcal{KEM}_F$ and the sequential tail ($M \to H \to D$) mirrors the designed defense-in-depth of fielded cryptographic systems where a conservative base algorithm (here, FrodoKEM's unstructured LWE) is paired with a modern, structured-LWE main channel (ML-KEM) and backup code-based and agility components. The critical path is $T_F + T_M + T_{\text{KDF}} + T_H + T_{\text{KDF}} + T_D \approx 3.2\,\text{ms}$, the highest of the three-one family but still strictly less than the fully cascaded Method 1.

Algorithm 23 Method 11: Three Sequential Middle - Encapsulation

function METHOD11.ENCAPS(pk)
  $(ct_F, ss_F) \leftarrow \mathcal{KEM}_F.\text{Encaps}(pk_F)$
  $(ct_M, ss_M) \leftarrow \mathcal{KEM}_M.\text{Encaps}(pk_M)$
  $\text{seed}_H \leftarrow \text{KDF}(ss_M \| \text{“m11-layer2”})$
  $(ct_H, ss_H) \leftarrow \mathcal{KEM}_H.\text{Encaps}(pk_H; \text{seed}_H)$
  $\text{seed}_D \leftarrow \text{KDF}(ss_H \| \text{“m11-layer3”})$
  $(ct_D, ss_D) \leftarrow \mathcal{KEM}_D.\text{Encaps}(pk_D; \text{seed}_D)$
  $\text{inter} \leftarrow \text{KDF}(ss_M \| ss_H \| ss_D \| \text{“m11-chain”})$
  $ss \leftarrow \text{KDF}(ss_F \| \text{inter} \| \text{“method11”})$
  return $((ct_F, ct_M, ct_H, ct_D), ss)$
end function

Algorithm 24 Method 11: Three Sequential Middle - Decapsulation

function METHOD11.DECAPS(ct, sk)
  for $i \in \{F, M, H, D\}$ **do**
    $ss'_i \leftarrow \text{KEM}_i.\text{Decaps}(ct_i, sk_i)$
    if $ss'_i = \perp$ **then**
      $ss'_i \leftarrow \text{KDF}(sk_i \| ct_i \| \text{“reject-”} \| i)$
    end if
  end for
  $\text{inter} \leftarrow \text{KDF}(ss'_M \| ss'_H \| ss'_D \| \text{“m11-chain”})$
  $ss \leftarrow \text{KDF}(ss'_F \| \text{inter} \| \text{“method11”})$
  return ss
end function

Security mapping to bidding rounds. In the bidding-round chain, Method 11 has two structurally distinct paths to $\mathbf{B}_7$:

- *Anchor-component path.* If $\mathcal{KEM}_F$ is secure, $\mathbf{B}_3$ substitutes $ss_F \to r_F$ and the final KDF call $ss \leftarrow \text{KDF}(r_F \| \text{inter} \| \texttt{``method11''})$ has uniform input r_F; $\mathbf{B}_7$ fires directly.
- *Sequential-chain path.* If any of $\{\mathcal{KEM}_M, \mathcal{KEM}_H, \mathcal{KEM}_D\}$ is secure, the corresponding substitution round $\mathbf{B}_k \in \{\mathbf{B}_4, \mathbf{B}_5, \mathbf{B}_6\}$ makes that component's ss_X uniform; this uniformity propagates through the chain's seed derivations and then through inter, again triggering $\mathbf{B}_7$.

Both paths independently yield the universal bound of Theorem 6.1; the union is bounded by the minimum via Lemma 5.5.

Deployment profile. Recommended for government and defense deployments that explicitly value defense-in-depth against staged cryptanalysis, and for compliance regimes (e.g. BSI TR-02102) that require algorithmically diverse cascaded components.

4.5.4 Method 12: Three Parallel Middle ($(F\|M\|D) \to H$)

Composition topology. Method 12 places HQC, the code-based component, as the *downstream* stage fed by a three-way parallel composition of FrodoKEM, ML-KEM, and the Dummy KEM. The three parallel components execute concurrently, their outputs are mixed through an intermediate KDF call to produce seed_H, and this seed is used to derive HQC's internal randomness. The final KDF call binds all four component secrets into the output.

Design rationale. Two design motivations converge here. First, HQC is the youngest and most structurally different of the post-quantum KEMs used by Cobra (its security rests on the hardness of decoding random quasi-cyclic codes rather than on lattice assumptions). Placing HQC *after* the lattice components mixes lattice-derived entropy into its randomness, providing a measure of inter-family cross-pollination that would not be present in a purely parallel composition. Second, at NIST Level 3 HQC has larger ciphertexts (9,026 bytes) than ML-KEM-768 (1,088 bytes) or FrodoKEM-976 (15,744 bytes of public key with 15,744 bytes of ciphertext, ML-KEM being smaller); deferring HQC to the second stage keeps the dominant bandwidth cost off the critical path when downstream KDF aggregation can proceed in parallel with HQC transmission. The critical path is $\max(T_F, T_M, T_D) + T_{\text{KDF}} + T_H \approx 1.6\,\text{ms}$, matching Method 10 but with a code-based rather than dummy-based downstream slot.

Algorithm 25 Method 12: Three Parallel Middle - Encapsulation

function METHOD12.ENCAPS(pk)
  $(ct_F, ss_F) \leftarrow \mathcal{KEM}_F.\text{Encaps}(pk_F)$
  $(ct_M, ss_M) \leftarrow \mathcal{KEM}_M.\text{Encaps}(pk_M)$
  $(ct_D, ss_D) \leftarrow \mathcal{KEM}_D.\text{Encaps}(pk_D)$
  $\text{inter} \leftarrow \text{KDF}(ss_F \| ss_M \| ss_D \| \text{“m12-stage1”})$
  $\text{seed}_H \leftarrow \text{KDF}(\text{inter} \| \text{“m12-seed-H”})$
  $(ct_H, ss_H) \leftarrow \mathcal{KEM}_H.\text{Encaps}(pk_H; \text{seed}_H)$
  $ss \leftarrow \text{KDF}(ss_F \| ss_M \| ss_H \| ss_D \| \text{“method12”})$
  return $((ct_F, ct_M, ct_H, ct_D), ss)$
end function

Algorithm 26 Method 12: Three Parallel Middle - Decapsulation

function METHOD12.DECAPS(ct, sk)
  for $i \in \{F, M, H, D\}$ **do**
    $ss'_i \leftarrow \text{KEM}_i.\text{Decaps}(ct_i, sk_i)$
    if $ss'_i = \perp$ **then**
      $ss'_i \leftarrow \text{KDF}(sk_i \| ct_i \| \text{“reject-”} \| i)$
    end if
  end for
  $ss \leftarrow \text{KDF}(ss'_F \| ss'_M \| ss'_H \| ss'_D \| \text{“method12”})$
  return ss
end function

Security mapping to bidding rounds. Method 12's bidding-round chain branches on which component is the security witness. If any of the three parallel components $\{\mathcal{KEM}_F, \mathcal{KEM}_M, \mathcal{KEM}_D\}$ is secure, the corresponding substitution round $\mathbf{B}_k \in \{\mathbf{B}_3, \mathbf{B}_4, \mathbf{B}_6\}$ makes its ss_X uniform, which immediately randomises inter, then seed_H, then ss_H (via HQC's

encapsulation on uniform input), and the final KDF input inherits uniformity in all three of its substantive slots; $\mathbf{B}_7$ fires. If instead $\mathcal{KEM}_H$ alone is secure (an unusual but admissible case), $\mathbf{B}_5$ directly substitutes $ss_H \to r_H$, the final KDF's third input is uniform, and $\mathbf{B}_7$ still fires. The universal bound follows.

Deployment profile. Recommended when algorithmic diversity across PQC families is a primary design constraint — for instance, in high-value archival-encryption contexts where the operator wants the output key to depend on *both* a lattice-based and a code-based component, with the Dummy slot reserved for a future fourth family (e.g. isogeny or multivariate).

4.6 Category 4: Complex Nested Methods

4.6.1 Method 13: Nested Left $(((F\|M) \to H)\|D)$

Composition topology. Method 13 is the first of the nested family: the composition graph has two concurrent branches that meet only in the final KDF. The *left branch* is a two-stage composition $(F\|M) \to H$, in which FrodoKEM and ML-KEM run in parallel, their outputs are mixed through an intermediate KDF to produce seed_H, and HQC is then encapsulated on that derived seed. The *right branch* is a single Dummy-KEM encapsulation running in parallel with the left branch. Each branch produces a 256-bit branch-summary value (branch_L, branch_R); the two are combined in the final KDF to produce the Cobra shared secret.

Design rationale. Nested Left isolates the Dummy-KEM slot on a structurally independent branch, minimising coupling between the agility slot and the three substantive PQC components. This has two benefits. First, replacing $\mathcal{KEM}_D$ with a future fifth algorithm requires changes to only branch_R's derivation, keeping the left branch entirely stable and providing clear migration semantics. Second, the left branch itself provides a genuine two-stage hybrid between lattice-unstructured (FrodoKEM) and lattice-module (ML-KEM) components feeding into code-based (HQC) downstream, giving a compact but defense-in-depth subcomposition that remains secure even if both left-branch components are compromised simultaneously (as long as $\mathcal{KEM}_H$ holds). The critical path is $\max(T_F, T_M) + T_{\text{KDF}} + T_H \approx 1.8\,\text{ms}$, with T_D running concurrently inside the same budget.

Algorithm 27 Method 13: Nested Left - Encapsulation

function METHOD13.ENCAPS(pk)
  $(ct_F, ss_F) \leftarrow \mathcal{KEM}_F.\text{Encaps}(pk_F)$
  $(ct_M, ss_M) \leftarrow \mathcal{KEM}_M.\text{Encaps}(pk_M)$
  $\text{inter}_1 \leftarrow \text{KDF}(ss_F\|ss_M\|\text{“m13-nest1”})$
  $\text{seed}_H \leftarrow \text{KDF}(\text{inter}_1\|\text{“m13-seed-H”})$
  $(ct_H, ss_H) \leftarrow \mathcal{KEM}_H.\text{Encaps}(pk_H; \text{seed}_H)$
  $\text{branch}_L \leftarrow \text{KDF}(ss_F\|ss_M\|ss_H\|\text{“m13-left”})$

  $(ct_D, ss_D) \leftarrow \mathcal{KEM}_D.\text{Encaps}(pk_D)$
  $\text{branch}_R \leftarrow \text{KDF}(ss_D\|\text{“m13-right”})$

  $ss \leftarrow \text{KDF}(\text{branch}_L\|\text{branch}_R\|\text{“method13”})$
  return $((ct_F, ct_M, ct_H, ct_D), ss)$
end function

Security mapping to bidding rounds. The two-branch topology induces a natural case split on which branch supplies the security witness:

- *Right-branch security ($\mathcal{KEM}_D$ secure).* $\mathbf{B}_6$ substitutes $ss_D \to r_D$, making branch_R uniform; the final KDF's second input is uniform and $\mathbf{B}_7$ fires.

Algorithm 28 Method 13: Nested Left - Decapsulation

function METHOD13.DECAPS(ct, sk)
  for $i \in \{F, M, H, D\}$ **do**
    $ss'_i \leftarrow \text{KEM}_i.\text{Decaps}(ct_i, sk_i)$
    if $ss'_i = \perp$ **then**
      $ss'_i \leftarrow \text{KDF}(sk_i \| ct_i \| \text{“reject-”} \| i)$
    end if
  end for
  $\text{branch}_L \leftarrow \text{KDF}(ss'_F \| ss'_M \| ss'_H \| \text{“m13-left”})$
  $\text{branch}_R \leftarrow \text{KDF}(ss'_D \| \text{“m13-right”})$
  $ss \leftarrow \text{KDF}(\text{branch}_L \| \text{branch}_R \| \text{“method13”})$
  return ss
end function

- *Left-branch security ($\mathcal{KEM}_F$, $\mathcal{KEM}_M$, or $\mathcal{KEM}_H$ secure).* The corresponding substitution round $\mathbf{B}_k \in \{\mathbf{B}_3, \mathbf{B}_4, \mathbf{B}_5\}$ makes its ss_X uniform, which propagates through the left branch's KDF calls and randomises branch_L; the final KDF's first input is uniform and $\mathbf{B}_7$ fires.

The two cases are mutually exclusive witnesses of hybrid breakage, so the combined ask price is bounded by $\min\{\epsilon_F, \epsilon_M, \epsilon_H, \epsilon_D\}$ via the Extended Difference Lemma.

Deployment profile. Recommended when operators require a clear structural firewall between the agility slot and the three substantive PQC components — for instance, research deployments that routinely substitute $\mathcal{KEM}_D$ with experimental or prototype algorithms, where changes must be provably confined to one branch of the composition graph.

4.6.2 Method 14: Nested Right ($F \| (M \to (H \| D))$)

Composition topology. Method 14 is the structural mirror of Method 13. The *left branch* consists of a single FrodoKEM encapsulation whose output is KDF-compressed into branch_L; it runs concurrently with the right branch. The *right branch* is a nested two-stage composition: ML-KEM runs first, and its output ss_M is KDF-expanded twice to produce two independent seeds seed_H and seed_D used for HQC and Dummy-KEM encapsulations running in parallel inside the right branch. The three right-branch secrets (ss_M, ss_H, ss_D) are then combined into branch_R, and the two branches are bound in the final KDF.

Design rationale. Nested Right promotes ML-KEM to the role of *right-branch anchor*: its shared secret ss_M seeds both downstream encapsulations in the right branch, so a break of any of $\{\mathcal{KEM}_H, \mathcal{KEM}_D\}$ without a corresponding break of $\mathcal{KEM}_M$ still yields no advantage (because their seeds were uniform). Concurrently, the left branch provides an independent FrodoKEM-only security contribution via branch_L. This gives Method 14 a distinctive *dual-anchor* profile: the hybrid remains secure as long as *either* FrodoKEM (left) *or* ML-KEM (right-anchor) is secure, where the latter subsumes any break of its downstream components. Critical path: $\max(T_F, T_M + T_{\text{KDF}} + \max(T_H, T_D)) \approx 1.8\,\text{ms}$.

Security mapping to bidding rounds. The dual-anchor topology produces a case split symmetric to Method 13 but with the branches reversed:

- *Left-branch security ($\mathcal{KEM}_F$ secure).* $\mathbf{B}_3$ substitutes $ss_F \to r_F$, making branch_L uniform; $\mathbf{B}_7$ fires with the final KDF's first input uniform.

- *Right-branch-anchor security ($\mathcal{KEM}_M$ secure).* $\mathbf{B}_4$ substitutes $ss_M \to r_M$, making seed_H and seed_D both uniform, thereby making ss_H and ss_D uniform (by encapsulation on uniform input), and branch_R uniform. Three out of four final-KDF inputs become uniform simultaneously.

Algorithm 29 Method 14: Nested Right - Encapsulation

function METHOD14.ENCAPS(pk)
  $(ct_F, ss_F) \leftarrow \mathcal{KEM}_F.\text{Encaps}(pk_F)$
  $\text{branch}_L \leftarrow \text{KDF}(ss_F \| \text{“m14-left”})$

  $(ct_M, ss_M) \leftarrow \mathcal{KEM}_M.\text{Encaps}(pk_M)$
  $\text{seed}_H \leftarrow \text{KDF}(ss_M \| \text{“m14-seed-H”})$
  $\text{seed}_D \leftarrow \text{KDF}(ss_M \| \text{“m14-seed-D”})$
  $(ct_H, ss_H) \leftarrow \mathcal{KEM}_H.\text{Encaps}(pk_H; \text{seed}_H)$
  $(ct_D, ss_D) \leftarrow \mathcal{KEM}_D.\text{Encaps}(pk_D; \text{seed}_D)$
  $\text{branch}_R \leftarrow \text{KDF}(ss_M \| ss_H \| ss_D \| \text{“m14-right”})$

  $ss \leftarrow \text{KDF}(\text{branch}_L \| \text{branch}_R \| \text{“method14”})$
  return $((ct_F, ct_M, ct_H, ct_D), ss)$
end function

Algorithm 30 Method 14: Nested Right - Decapsulation

function METHOD14.DECAPS(ct, sk)
  for $i \in \{F, M, H, D\}$ **do**
    $ss'_i \leftarrow \text{KEM}_i.\text{Decaps}(ct_i, sk_i)$
    if $ss'_i = \perp$ **then**
      $ss'_i \leftarrow \text{KDF}(sk_i \| ct_i \| \text{“reject-”} \| i)$
    end if
  end for
  $\text{branch}_L \leftarrow \text{KDF}(ss'_F \| \text{“m14-left”})$
  $\text{branch}_R \leftarrow \text{KDF}(ss'_M \| ss'_H \| ss'_D \| \text{“m14-right”})$
  $ss \leftarrow \text{KDF}(\text{branch}_L \| \text{branch}_R \| \text{“method14”})$
  return ss
end function

- *Right-branch-component security ($\mathcal{KEM}_H$ or $\mathcal{KEM}_D$ secure).* $\mathbf{B}_5$ or $\mathbf{B}_6$ substitutes; branch$_R$'s derivation includes the substituted ss_X and hence becomes uniform; $\mathbf{B}_7$ fires.

All three cases are witnesses of hybrid breakage and their union is bounded by Lemma 5.5; the bound of Theorem 6.1 holds.

Deployment profile. Recommended when ML-KEM is the primary operational algorithm (e.g. post-FIPS-203 deployments where Module-LWE is the mainline algorithm) and FrodoKEM serves as a conservative backup anchor; the right-branch structure means a future substitution of the Dummy-KEM or a reconfiguration of the HQC parameters can be performed without disturbing the left-branch contract.

4.6.3 Method 15: Nested Center $((F \to (M\|H) \to D))$

Composition topology. Method 15 has the richest layered structure of the fifteen methods. It is organised as three layers meeting at the centre:

- **Layer 1.** FrodoKEM runs first, producing ss_F which is KDF-expanded twice into independent seeds seed$_M$ and seed$_H$.
- **Layer 2.** ML-KEM and HQC run concurrently on their Layer-1-derived seeds, producing ss_M and ss_H. These are mixed into an intermediate quantity inter that captures the centre-layer state.
- **Layer 3.** inter is KDF-expanded into seed$_D$, which seeds the Dummy-KEM encapsulation as the final sequential component.

The final KDF call binds all four component secrets, as usual.

Design rationale. Nested Center is the most structurally diverse method: FrodoKEM's unstructured-LWE entropy feeds the structured-lattice and code-based components in Layer 2, whose mixed output in turn feeds the agility slot in Layer 3. The construction achieves three goals at once: (i) *entropy amplification*, in that ss_F's randomness is used three times (once for seed$_M$, once for seed$_H$, and once directly in the final KDF), spreading FrodoKEM's conservative security assumption across the entire composition; (ii) *parallelism inside the centre*, because the two Layer-2 encapsulations run concurrently; and (iii) *full binding of the agility slot* to upstream state, since the Dummy-KEM's randomness inherits contributions from all three substantive PQC components. The critical path is $T_F + T_{\text{KDF}} + \max(T_M, T_H) + T_{\text{KDF}} + T_D \approx 1.9\,\text{ms}$, slightly higher than Methods 13/14 because Layer 3 waits for Layer 2's completion.

Algorithm 31 Method 15: Nested Center - Encapsulation

function METHOD15.ENCAPS(pk)
  $(ct_F, ss_F) \leftarrow \mathcal{KEM}_F.\text{Encaps}(pk_F)$
  $\text{seed}_M \leftarrow \text{KDF}(ss_F \| \text{“m15-seed-M”})$
  $\text{seed}_H \leftarrow \text{KDF}(ss_F \| \text{“m15-seed-H”})$
  $(ct_M, ss_M) \leftarrow \mathcal{KEM}_M.\text{Encaps}(pk_M; \text{seed}_M)$
  $(ct_H, ss_H) \leftarrow \mathcal{KEM}_H.\text{Encaps}(pk_H; \text{seed}_H)$
  $\text{inter} \leftarrow \text{KDF}(ss_M \| ss_H \| \text{“m15-center”})$
  $\text{seed}_D \leftarrow \text{KDF}(\text{inter} \| \text{“m15-seed-D”})$
  $(ct_D, ss_D) \leftarrow \mathcal{KEM}_D.\text{Encaps}(pk_D; \text{seed}_D)$
  $ss \leftarrow \text{KDF}(ss_F \| ss_M \| ss_H \| ss_D \| \text{“method15”})$
  return $((ct_F, ct_M, ct_H, ct_D), ss)$
end function

Security mapping to bidding rounds. The three-layer structure produces a four-way case split, matching the layered topology:

Algorithm 32 Method 15: Nested Center - Decapsulation

function METHOD15.DECAPS(ct, sk)
  for $i \in \{F, M, H, D\}$ **do**
    $ss'_i \leftarrow \text{KEM}_i.\text{Decaps}(ct_i, sk_i)$
    if $ss'_i = \perp$ **then**
      $ss'_i \leftarrow \text{KDF}(sk_i \| ct_i \| \text{“reject-”} \| i)$
    end if
  end for
  $ss \leftarrow \text{KDF}(ss'_F \| ss'_M \| ss'_H \| ss'_D \| \text{“method15”})$
  return ss
end function

- *Layer-1 security ($\mathcal{KEM}_F$ secure).* $\mathbf{B}_3$ substitutes $ss_F \to r_F$. This randomises seed_M and seed_H; because encapsulation on uniform randomness preserves output uniformity, both ss_M and ss_H become uniform; consequently inter, seed_D, and ss_D also inherit uniformity. The final KDF receives all four inputs uniform and $\mathbf{B}_7$ fires with the strongest possible margin.
- *Centre-layer security ($\mathcal{KEM}_M$ or $\mathcal{KEM}_H$ secure).* $\mathbf{B}_4$ or $\mathbf{B}_5$ substitutes its ss_X to uniform. This randomises inter, hence seed_D, hence ss_D; three of the four final-KDF inputs become uniform and $\mathbf{B}_7$ fires.
- *Agility-slot security ($\mathcal{KEM}_D$ secure).* $\mathbf{B}_6$ substitutes $ss_D \to r_D$; the final KDF's fourth input is uniform and $\mathbf{B}_7$ fires.

The entropy-amplification structure means a break of $\mathcal{KEM}_F$ alone compromises security of three downstream component outputs simultaneously — but crucially this is only a concern if FrodoKEM itself is broken, in which case a hybrid scheme is already in jeopardy by design; otherwise the upper-layer uniformity lifts strictly to lower layers. The universal bound of Theorem 6.1 applies.

Deployment profile. Recommended for exploratory or research deployments that benefit from the richest composition topology available; also appropriate for settings where FrodoKEM is explicitly treated as the conservative baseline and algorithmic diversity in downstream layers is a declared design goal.

5 Market-Theoretic Security Framework for Hybrid KEMs

We analyse all 15 Cobra methods within the *Market-Theoretic Security Framework* (MTSF) of [93]. MTSF subsumes and strictly extends the Universal Composability (UC) framework of Canetti [27, 29] and the game-based Random Oracle Model (ROM) methodology of Bellare and Rogaway [10, 11] and Shoup [107]: every UC/ROM artefact has a direct MTSF counterpart, and MTSF adds quantitative price adjustments, CNF session auditing, and unbounded session pinging on top. This section establishes the market model specialised to hybrid KEMs; Section 6 carries out the bidding-round proofs for every method.

5.1 MTSF Market Model for Hybrid KEMs

Definition 5.1 (Cobra Security Market)**.** *The Cobra security market is the tuple*

$$\mathcal{M}_{\mathsf{Cobra}} \;=\; \big(\mathsf{Seller},\, \{\mathsf{Buyer}_i\}_{i\in[n]},\, \mathcal{G}_{\mathsf{Cobra}},\, \mathsf{Price}\big),$$

where the seller Seller *is the IND-CCA2 challenger for the hybrid KEM, the buyers* $\{\mathsf{Buyer}_i\}$ *are PPT adversaries* $\mathcal{A}$ *attacking the scheme, the security goods catalogue is*

$$\mathcal{G}_{\mathsf{Cobra}} \;=\; \big\{g_{\mathsf{IND\text{-}CCA2}},\; g_{\mathsf{sk}},\; g_{\mathsf{CNF}},\; g_{\mathsf{auth}},\; g_{\mathsf{robust}}\big\},$$

and the price function $\mathsf{Price} : \mathcal{G}_{\mathsf{Cobra}} \times \mathbb{N} \to [0,1]$ *maps each good to the adversary's best-achievable advantage at security parameter* λ*.*

The five goods correspond, respectively, to IND-CCA2 ciphertext indistinguishability, session-key secrecy (TLS-composable), CNF-based session-level correctness, entity authentication (when Cobra is composed with an authenticated key-exchange), and *robust hybrid security*—the defining Cobra property that the hybrid KEM remains secure as long as *at least one* of the four component KEMs $\{\mathcal{KEM}_F, \mathcal{KEM}_M, \mathcal{KEM}_H, \mathcal{KEM}_D\}$ remains IND-CCA2 secure.

Definition 5.2 (Hybrid-KEM Bid)**.** *A buyer's bid against* $\mathcal{M}_{\mathsf{Cobra}}$ *is a triple* $\mathsf{Bid} = (g_j, T, \epsilon)$ *where* $g_j \in \mathcal{G}_{\mathsf{Cobra}}$ *is the target good,* $T \le \mathsf{poly}(\lambda)$ *is the computational budget, and* ϵ *is the target advantage. The bid succeeds iff the buyer achieves advantage* $\ge \epsilon$ *within* T *steps.*

Definition 5.3 (Quoted and Ask Prices)**.** *For each good* $g_j \in \mathcal{G}_{\mathsf{Cobra}}$*, the* quoted price $\mathsf{QPrice}_0(g_j)$ *is the raw adversary advantage in the unmodified real-world game* $\mathbf{B}_0$*:*

$$\mathsf{QPrice}_0(g_j) \;=\; \mathsf{Adv}^{g_j}_{\mathcal{A}}(\lambda) \;=\; \left|\Pr[\mathbf{B}_0(\mathcal{A}) = 1] - 1/2\right|.$$

The ask price *is the supremum over PPT buyers:* $\mathsf{Ask}(g_j, \lambda) = \max_{\mathcal{A} \in \mathsf{PPT}} \mathsf{Adv}^{g_j}_{\mathcal{A}}(\lambda)$*. The market is in* equilibrium *for* g_j *iff* $\mathsf{Ask}(g_j, \lambda) \le \mathsf{negl}(\lambda)$*; otherwise the market has* collapsed*.*

Definition 5.4 (Bidding Rounds and Price Adjustments)**.** *A* bidding-round chain *for good* g_j *is a sequence* $\mathbf{B}_0, \mathbf{B}_1, \ldots, \mathbf{B}_q$ *of experiments with* $\mathbf{B}_0$ *being the real IND-CCA2 game and* $\mathbf{B}_q$ *an ideal game in which winning is information-theoretically impossible. Each transition* $\mathbf{B}_k \to \mathbf{B}_{k+1}$ *carries a* price adjustment

$$\Delta\mathsf{Price}_k \;=\; \left|\Pr[\mathbf{B}_k(\mathcal{A}) = 1] - \Pr[\mathbf{B}_{k+1}(\mathcal{A}) = 1]\right|,$$

and the ask price decomposes telescopically:

$$\mathsf{Ask}(g_j, \lambda) \;\le\; \sum_{k=0}^{q-1} \Delta\mathsf{Price}_k \;+\; \Pr[\mathbf{B}_q(\mathcal{A}) = 1] - 1/2. \tag{1}$$

Lemma 5.5 (Extended Difference Lemma [93])**.** *If* $\mathbf{B}_k$ *and* $\mathbf{B}_{k+1}$ *are identical unless at least one of the failure events* $F_1, \ldots, F_m$ *occurs, then*

$$\Delta\mathsf{Price}_k \;\le\; \Pr\Big[\bigcup_{i=1}^{m} F_i\Big] \;\le\; \textstyle\sum_{i=1}^{m} \Pr[F_i]$$

by the union bound, with a tighter inclusion–exclusion bound available when the F_i *are correlated.*

This extended form is essential for Cobra: a single bidding round may simultaneously involve a component-KEM indistinguishability failure, a KDF collision in the random oracle, and an implicit-rejection guess. We bound all three in one round rather than splitting them into three.

5.2 UC as Market Regulation; GUC as Shared Infrastructure

Every element of the UC framework maps directly to a market-theoretic counterpart, and no composition guarantee is lost.

Definition 5.6 (Hybrid-KEM Ideal Functionality as Market Regulator)**.** *The ideal functionality* $\mathcal{F}_{\mathsf{HybridKEM}}$ *is the market regulator for* $\mathcal{M}_{\mathsf{Cobra}}$*. Internally it maintains a table* T *mapping* $(pk, ct) \mapsto ss$*.*

Algorithm 33 Market-Regulator Functionality $\mathcal{F}_{\mathsf{HybridKEM}}$

State: table T initially empty
upon (KeyGen, P) from party P
Generate (pk, sk) with $pk = (pk_F, pk_M, pk_H, pk_D)$, $sk = (sk_F, sk_M, sk_H, sk_D)$
Store sk internally; send pk to P and to adversary $\mathcal{A}$

upon (Encaps, pk) from party P
Sample $ss \xleftarrow{\$} \{0,1\}^{256}$; generate ciphertext components (ct_F, ct_M, ct_H, ct_D) consistent with the method
Let $ct = (ct_F, ct_M, ct_H, ct_D)$; store $T[pk, ct] \leftarrow ss$
Send (ct, ss) to P and ct to $\mathcal{A}$

upon (Decaps, ct) from party P
if $(pk, ct) \in T$ **then**
    $ss \leftarrow T[pk, ct]$ ▷ valid ciphertext
else
    $ss \xleftarrow{\$} \{0,1\}^{256}$ ▷ implicit rejection—idealised
    Store $T[pk, ct] \leftarrow ss$ for consistency
end if
Send ss to P; notify $\mathcal{A}$ of the decapsulation query

Definition 5.7 (UC-to-MTSF Translation)**.** *The UC framework maps to MTSF as follows:*

UC concept	***MTSF counterpart***
Ideal functionality $\mathcal{F}_{\mathsf{HybridKEM}}$	*Market regulator on* $\mathcal{G}_{\mathsf{Cobra}}$
Environment $\mathcal{Z}$	*Regulatory oversight monitor*
Simulator $\mathcal{S}$	*Market arbitrageur bridging real/ideal*
Adversary $\mathcal{A}$	*Buyer placing bids* Bid_i *on* g_j
$\mathsf{Real}_{\pi,\mathcal{A},\mathcal{Z}} \approx_c \mathsf{Ideal}_{\mathcal{F},\mathcal{S},\mathcal{Z}}$	$\mathsf{Ask}(g_j, \lambda) \leq \mathsf{negl}(\lambda)$ *(equilibrium)*
UC composition theorem	*Market merger theorem (Thm. 5.8)*
GUC shared functionality (PKI, CRS)	*Shared market infrastructure*

Theorem 5.8 (Market Merger Theorem for Hybrid KEMs)**.** *Let* $\mathcal{M}_{\mathsf{Cobra}}$ *be a Cobra hybrid-KEM market in equilibrium, i.e.* $\mathsf{Ask}(g_{\mathsf{IND\text{-}CCA2}}, \lambda) \leq \mathsf{negl}(\lambda)$*, and let* $\mathcal{M}_{\mathsf{TLS1.3}}$ *be a TLS 1.3 record-layer market in equilibrium for* g_{sk} *and* g_{auth}*. Then the merged market* $\mathcal{M}_{\mathsf{Cobra}} \otimes \mathcal{M}_{\mathsf{TLS1.3}}$*, representing concurrent execution of Cobra key establishment with TLS handshake and record-layer operations, is in equilibrium with*

$$\mathsf{Ask}_{\mathsf{merge}}(g_j, \lambda) \;\leq\; \mathsf{Ask}_{\mathsf{Cobra}}(g_j, \lambda) \;+\; \mathsf{Ask}_{\mathsf{TLS}}(g_j, \lambda) \;+\; \mathsf{negl}(\lambda),$$

for each relevant good g_j*.*

Proof sketch. By Definition 5.7, equilibrium in $\mathcal{M}_{\mathsf{Cobra}}$ is equivalent to UC-realisation of $\mathcal{F}_{\mathsf{HybridKEM}}$, and equilibrium in $\mathcal{M}_{\mathsf{TLS1.3}}$ is equivalent to UC-realisation of the TLS 1.3 ideal channel functionality $\mathcal{F}_{\mathsf{TLS}}$. The UC composition theorem [27, 29] guarantees that the concurrent composition $\pi_{\mathsf{Cobra}} \circ \pi_{\mathsf{TLS}}$ UC-realises $\mathcal{F}_{\mathsf{HybridKEM}} \| \mathcal{F}_{\mathsf{TLS}}$, with additive advantage loss. Translating back through Definition 5.7 yields the stated ask-price bound. □

5.3 Random Oracle Queries as Hash Bids

In MTSF, hash-function queries are *hash bids*: the buyer spends computational budget to query the random oracle, hoping to find a useful pre-image. The standard ROM operations map to bidding-round primitives:

- **Oracle programming** is a *costless market restructuring* with $\Delta\mathsf{Price} = 0$: the seller re-routes the oracle to embed a hardness-problem challenge; since the oracle is uniformly random, the buyer cannot detect the routing change.
- **Oracle collision** is a *birthday bid*: q_H queries produce a collision with probability at most $q_H^2/2^{n+1}$ for an n-bit output, giving $\Delta\mathsf{Price} \leq q_H^2/2^{n+1}$.
- **Component-KEM indistinguishability** hops are *computational bids*: the buyer is reduced to an IND-CCA2 distinguisher against $\mathcal{KEM}_X$, giving $\Delta\mathsf{Price} \leq \mathsf{Adv}^{\mathsf{IND\text{-}CCA2}}_{\mathcal{KEM}_X}(\lambda)$.
- **Implicit rejection** is an *implicit-rejection guess bid*: without knowledge of the component secret keys, the buyer's best strategy guesses a rejection value, giving $\Delta\mathsf{Price} \leq q_D/2^{256}$ for q_D decapsulation queries.

5.4 CNF Session Verification for Hybrid KEMs

MTSF augments the ask-price bound with a *per-session correctness audit*, expressed as a conjunctive normal form (CNF) formula whose clauses capture the invariants that every honest session must satisfy. Unlike the ask-price bound, which quantifies residual adversarial advantage probabilistically, the session-CNF is *Boolean*: it either holds or it does not, and a trained auditor can verify it by direct observation of the session transcript without running any cryptanalysis. This subsection formalises the CNF and provides a four-column manual-verification worksheet that operators can use directly during deployment audits.

Definition 5.9 (Hybrid-KEM Session CNF)**.** *For a Cobra hybrid-KEM session, the session-CNF is*

$$\varphi_{\mathsf{Cobra}} \;=\; \varphi^{\mathsf{key}} \wedge \varphi^{\mathsf{novel}} \wedge \varphi^{\mathsf{rej}} \wedge \varphi^{\mathsf{kdf}} \wedge \varphi^{\mathsf{ping}},$$

with the five clauses defined by the per-clause invariants below. A session is accepted *by the CNF audit iff every clause evaluates to true; a single falsified clause triggers rejection.*

Clause summary. The five clauses capture five independent correctness invariants of a Cobra session:

Clause	Check	How
φ^{key}: key correct	$ss \stackrel{?}{=} \mathrm{KDF}(ss_F \Vert ss_M \Vert ss_H \Vert ss_D \Vert \mathrm{ctx})$	Re-derive; compare bytes
φ^{novel}: fresh ct	$ct \notin \mathcal{C}_{\mathsf{used}}$	Ciphertext log
φ^{rej}: implicit reject consistent	Invalid ct' yields $H(sk_i \Vert ct_i' \Vert \mathsf{rej} \Vert i)$	Crafted invalid ct'
φ^{kdf}: KDF binding correct	Context includes method tag, pk	Byte inspection
φ^{ping}: session fresh	All ct_i distinct from prior sessions	Per-session $\mathcal{C}_{\mathsf{prev}}$ log

Four-column manual-verification worksheet. The audit procedure below is presented as a four-column worksheet suitable for pen-and-paper verification or for transcription into a compliance management system such as those required by PCI DSS v4.0 Requirement 4.2.1.1, FIPS 140-3 CMVP, Common Criteria EAL4+, or ISO/IEC 27001:2022 Annex A.8.24. The columns are: (i) the clause identifier, (ii) the invariant as a logical formula, (iii) the step-by-step manual check the auditor performs, and (iv) a structured pass/fail log entry to record in the audit journal.

Clause	Invariant	Manual Verification Procedure	Pass/Fail Log Entry
φ^{key}	$ss = \mathrm{KDF}\big(ss_F \Vert ss_M \Vert ss_H \Vert ss_D \Vert \mathrm{ctx}\big)$	(1) Decapsulate each component ciphertext ct_F, ct_M, ct_H, ct_D using sk_F, sk_M, sk_H, sk_D to recover ss_F, ss_M, ss_H, ss_D. (2) Concatenate in the canonical order with the session context string ctx. (3) Apply the SHAKE-256 KDF to the concatenation. (4) Compare the 32-byte output byte-for-byte to the session key ss delivered to the application.	`[SID:####]` φ^{key}: `PASS/FAIL`; record the SHA-256 hash of the concatenated input and the 32-byte output in the audit log. Any byte-level mismatch is a FAIL.
φ^{novel}	$ct \notin \mathcal{C}_{\mathsf{used}}$	(1) Retrieve the ciphertext log $\mathcal{C}_{\mathsf{used}}$ of this endpoint's prior ciphertexts (kept as a hash-indexed table of size n). (2) Compute $H_{\mathsf{ct}} = \mathrm{SHA}-256(ct)$. (3) Perform a hash-table lookup of H_{ct} in $\mathcal{C}_{\mathsf{used}}$. (4) If the lookup returns $\perp$, insert H_{ct} into $\mathcal{C}_{\mathsf{used}}$; otherwise flag a replay.	`[SID:####]` φ^{novel}: `PASS` (new) or `FAIL (replay of SID:####)`; record H_{ct} and the collision index if any.
φ^{rej}	Invalid ct' yields $ss' = H(sk_i \Vert ct'_i \Vert \mathsf{rej} \Vert i)$ for every $i \in \{F, M, H, D\}$	(1) Craft an invalid ciphertext ct' by flipping one byte in each component's ciphertext (or by sampling a random ct' outside the valid ciphertext space). (2) Feed ct' to the session's decapsulation routine. (3) Capture the output ss'. (4) Independently compute the expected implicit-rejection value using the formula $H(sk_i \Vert ct'_i \Vert \mathsf{rej} \Vert i)$ for each component. (5) Check that the returned ss' equals the KDF of the four component implicit-rejection values, not a real shared secret.	`[SID:####]` φ^{rej}: `PASS` if the crafted invalid ciphertext produces the deterministic implicit-rejection output and does not leak timing. `FAIL` if the output differs from the expected implicit-rejection value or if a timing side-channel is observed.
φ^{kdf}	ctx includes the method tag `"methodi"`, the full public key pk, and the fixed byte-length prefix	(1) Extract the KDF context string ctx from the session binder. (2) Byte-inspect the first 8 bytes to confirm they encode the method identifier `"methodi"` where $i \in \{1, \ldots, 15\}$. (3) Confirm the next bytes encode the full concatenated public key $pk = pk_F \Vert pk_M \Vert pk_H \Vert pk_D$ at the correct offsets. (4) Confirm the ctx length matches the prescribed fixed-length format.	`[SID:####]` φ^{kdf}: `PASS` with method tag recorded and pk hash logged. `FAIL` on any domain-separation byte mismatch, unexpected length, or missing method tag.
φ^{ping}	$\forall i,\ ct_i \notin \mathcal{C}_{\mathsf{prev}}$ where $\mathcal{C}_{\mathsf{prev}}$ is the prior-sessions ciphertext log	(1) For each component $i \in \{F, M, H, D\}$, compute $H_i = \mathrm{SHA}-256(ct_i)$. (2) Query the persistent prior-sessions log $\mathcal{C}_{\mathsf{prev}}$ for each H_i. (3) If any H_i is present, flag a ping replay with the offending component and the colliding session identifier. (4) Otherwise insert all four hashes into $\mathcal{C}_{\mathsf{prev}}$.	`[SID:####]` φ^{ping}: `PASS (all four components fresh)` or `FAIL (component X reuses ct from SID:####)`.

Session-log template. During an operator's audit run, each completed session produces one row in a four-column *session-log* as follows. The auditor fills in columns 3 and 4 based on the observed transcript and the clause-level worksheet above:

Session ID	Timestamp (UTC)	Method	φ_{Cobra} verdict (per-clause)
SID:0001	2026-04-20 10:14:32.102	Method 2	φ^{key}=P, φ^{novel}=P, φ^{rej}=P, φ^{kdf}=P, φ^{ping}=P
SID:0002	2026-04-20 10:14:32.117	Method 2	φ^{key}=P, φ^{novel}=P, φ^{rej}=P, φ^{kdf}=P, φ^{ping}=P
SID:0003	2026-04-20 10:14:32.131	Method 2	φ^{key}=P, φ^{novel}=**F**, φ^{rej}=P, φ^{kdf}=P, φ^{ping}=P
...	...	...	...

The fictitious third row in the example above shows a session flagged by the novelty clause — an auditor would record the offending ciphertext hash, correlate it against prior sessions, and treat the session as *rejected* by φ_{Cobra} notwithstanding the other four clauses passing. In production deployments the same worksheet can be mechanised (script-driven verification of the five clauses, logged to a structured sink such as syslog or an SIEM) but the manual-verification form above is the canonical ground truth that the mechanised verifier must match.

Relation to the ask-price bound. A session that passes φ_{Cobra} is a session in which none of the five failure modes that the bidding-round chain of Section 6 guards against have occurred. The CNF audit therefore complements the ask-price bound: the ask-price bounds the probability that an adversary wins despite honest sessions, while the CNF catches dishonest sessions that would otherwise invalidate the proof's assumptions. Passing both guarantees means that the market equilibrium of Theorem 6.1 is realised not merely in expectation but in the specific session just observed.

5.5 Unbounded Session Pinging

The classical ROM proof bounds the advantage of an adversary that makes q queries; it does not directly cover the scenario of unbounded concurrent sessions. MTSF's session-pinging mechanism closes this gap.

Definition 5.10 (Hybrid-KEM Ping Bid)**.** *The* ping bid *for Cobra is the buyer's attempt to replay a ciphertext $ct^* = ct_i$ from session $\mathsf{Session}_i$ into the decapsulation oracle of session $\mathsf{Session}_{j\neq i}$. Clause φ^{novel} of Definition 5.9 detects the replay; implicit rejection returns $H(sk\|ct_i\|\mathsf{rej})$, which is independent of the target session key. The extended difference lemma (Lemma 5.5) bounds the combined failure $F_{\mathsf{rej}} \cup F_{\mathsf{bypass}}$ by $q_D/2^{256}$.*

Theorem 5.11 (Unbounded-Session Security)**.** *If $\mathcal{M}_{\mathsf{Cobra}}$ is in equilibrium for the single-session good $g_{\mathsf{IND\text{-}CCA2}}$ with ask price $\epsilon(\lambda)$, and if the session-CNF φ_{Cobra} (Definition 5.9) is preserved across sessions by session-index-independent reductions, then $\mathcal{M}_{\mathsf{Cobra}}$ is in equilibrium for the unbounded-session good with ask price $\epsilon(\lambda) + \delta_{\mathsf{ping}}$, where $\delta_{\mathsf{ping}} \leq q_D/2^{256} = \mathsf{negl}(\lambda)$.*

Proof sketch. Session-CNF isomorphism ensures that $\varphi_{\mathsf{Cobra},i+1}$ is obtained from $\varphi_{\mathsf{Cobra},i}$ by substituting the fresh randomness of session $i{+}1$. Since the component KEMs' hardness instances (MLWE for ML-KEM, LWE for FrodoKEM, syndrome decoding for HQC) are independent across sessions, the single-session reduction lifts session-by-session. The ping bid adds at most δ_{ping} per session, bounded uniformly by $q_D/2^{256}$ over all sessions. □

6 Bidding-Round Security Proofs for All 15 Methods

We now prove that every Cobra method places $\mathcal{M}_{\mathsf{Cobra}}$ in equilibrium for $g_{\mathsf{IND\text{-}CCA2}}$ and g_{robust} via an explicit bidding-round chain of 10 rounds per method. The chain simultaneously delivers the UC-composition guarantee (via Theorem 5.8) and the concrete ROM-style ask-price bound.

6.1 Universal Bidding-Round Theorem

Theorem 6.1 (Universal Cobra Equilibrium)**.** *Let $\mathcal{KEM}_F, \mathcal{KEM}_M, \mathcal{KEM}_H, \mathcal{KEM}_D$ have IND-CCA2 advantages $\epsilon_F, \epsilon_M, \epsilon_H, \epsilon_D$ respectively, and let the KDF be modelled as a random oracle. Then for every Cobra method $i \in \{1, \ldots, 15\}$, the ask price against any PPT buyer making at most q_H hash queries and q_D decapsulation queries satisfies*

$$\mathsf{Ask}_{\mathsf{Cobra}_i}(g_{\mathsf{IND\text{-}CCA2}}, \lambda) \;\le\; \min\{\epsilon_F, \epsilon_M, \epsilon_H, \epsilon_D\} \;+\; \frac{(q_H + q_D)^2}{2^{257}} \;+\; \frac{q_D}{2^{256}}, \tag{2}$$

where the first term reflects robust hybrid security *(the hybrid is at least as secure as its strongest component: breaking the hybrid requires breaking* every *component, so the dominant contribution is the smallest component advantage* $\min\{\epsilon_F, \epsilon_M, \epsilon_H, \epsilon_D\}$*), the second is the birthday bound on KDF collisions, and the third is the implicit-rejection guessing probability. Consequently $\mathcal{M}_{\mathsf{Cobra}}$ is in equilibrium for every method, and—via Theorem 5.8—for every composition with TLS 1.3.*

The bound (2) is the sum of the price adjustments in the 10-bidding-round chain below; after each bid the market state is a strict simplification of the real world, and the final round $\mathbf{B}_{10}$ is the ideal market in which the buyer's advantage is exactly 0.

6.2 Universal Simulator as Market Arbitrageur

The MTSF counterpart of the UC simulator is a *market arbitrageur* $\mathcal{S}$ that bridges real and ideal markets by running $\mathcal{F}_{\mathsf{HybridKEM}}$ internally. The same arbitrageur works for all 15 methods.

Algorithm 34 Universal Market Arbitrageur $\mathcal{S}$ for All 15 Methods

Setup: $\mathcal{S}$ runs $\mathcal{F}_{\mathsf{HybridKEM}}$ internally and interacts with the environment $\mathcal{Z}$
Random oracle: $\mathcal{S}$ maintains hash-bid table H for KDF queries
upon $\mathcal{A}$ requests public keys
Forward $(\textsc{KeyGen}, P)$ to $\mathcal{F}_{\mathsf{HybridKEM}}$; receive $pk = (pk_F, pk_M, pk_H, pk_D)$; relay to $\mathcal{A}$

upon $\mathcal{A}$ queries the challenge encapsulation
Forward $(\textsc{Encaps}, pk)$ to $\mathcal{F}_{\mathsf{HybridKEM}}$; receive ct
Simulate method-specific component ciphertexts (ct_F, ct_M, ct_H, ct_D) consistent with ct
Relay ct to $\mathcal{A}$

upon $\mathcal{A}$ makes a decapsulation query on $ct' \neq ct^*$
Forward $(\textsc{Decaps}, ct')$ to $\mathcal{F}_{\mathsf{HybridKEM}}$; relay the answer

upon $\mathcal{A}$ makes a hash bid (KDF query) q
if $q \notin H$ **then**
    $H[q] \xleftarrow{\$} \{0,1\}^{256}$
end if
Return $H[q]$ to $\mathcal{A}$

upon $\mathcal{A}$ outputs a message to $\mathcal{Z}$
Forward to $\mathcal{Z}$

The arbitrageur's only non-trivial action is simulating component ciphertexts consistent with the method's composition rule; the remaining steps are forwarding.

6.3 Ten-Round Bidding-Round Proof for Method 1 (Full Cascade)

We give the fully detailed chain for Method 1; all other methods follow the same template with the component-KEM substitution order reflecting the method's composition structure. For each round $\mathbf{B}_k$ we state three things explicitly: (i) *what* the bid is, (ii) *what it replaces* relative to the previous market state, and (iii) the *complexity* (the tight bound on the price adjustment $\Delta\mathsf{Price}_k$).

Bidding-round proof for Method 1. The chain $\mathbf{B}_0, \mathbf{B}_1, \ldots, \mathbf{B}_{10}$ transforms the real IND-CCA2 market into an ideal market in which winning is impossible. We denote $\Pr[\mathbf{B}_k(\mathcal{A}) = 1] = p_k$.

Round $\mathbf{B}_0$ — Real Market. *What it is:* the real IND-CCA2 game against the Cobra hybrid. The seller generates (pk, sk) honestly, returns pk to the buyer, and answers decapsulation and hash queries in the real protocol. The challenge is (ct^*, ss_b) with $b \xleftarrow{\$} \{0,1\}$, ss_0 real and ss_1 uniformly random. *What it replaces:* nothing — this is the base market. *Complexity:* the quoted price $\mathsf{QPrice}_0(g_{\mathsf{IND\text{-}CCA2}}) = |p_0 - 1/2|$ is the adversary's real-world advantage; no price adjustment applies to $\mathbf{B}_0$ itself.

Round $\mathbf{B}_1$ — Oracle-Programming Bid. *What it is:* the seller makes KDF-table management explicit: on query q, return $H[q]$ if set, else sample $H[q] \xleftarrow{\$} \{0,1\}^{256}$. *What it replaces:* the real-world KDF evaluations of $\mathbf{B}_0$ are now routed through an explicit random-oracle table. *Complexity:* $\Delta\mathsf{Price}_0 = 0$ (a costless market restructuring; the oracle is uniformly random either way, so $p_1 = p_0$).

Round $\mathbf{B}_2$ — Abort-on-Bad-Hash-Bid. *What it is:* let $SS^* = \{ss_F^*, ss_M^*, ss_H^*, ss_D^*\}$ be the component secrets of the challenge. The seller aborts if the buyer ever issues a hash bid $H(\cdots \| ss_i^* \| \cdots)$ for any ss_i^* without first obtaining it via decapsulation. *What it replaces:* bad-hash queries that would otherwise produce an information-theoretic shortcut to ss are now hard aborts. *Complexity:* $\Delta\mathsf{Price}_1 \le q_H \cdot \max\{\epsilon_F, \epsilon_M, \epsilon_H, \epsilon_D\}$ by the Extended Difference Lemma (Lemma 5.5): any abort-triggering guess of an ss_i^* yields an IND-CCA2 distinguisher against $\mathcal{KEM}_X$, so the union over q_H hash queries is at most q_H times the best component advantage. In the remaining rounds we condition on no abort.

Round $\mathbf{B}_3$ — FrodoKEM Substitution Bid. *What it is:* a computational bid against the FrodoKEM component. The seller replaces ss_F^* in the challenge with a uniform $r_F \xleftarrow{\$} \{0,1\}^{256}$. *What it replaces:* the real FrodoKEM shared secret ss_F^* is replaced by uniform randomness. *Complexity:* $\Delta\mathsf{Price}_2 \le \epsilon_F = \mathsf{Adv}^{\mathsf{IND\text{-}CCA2}}_{\mathcal{KEM}_F}(\lambda)$. Distinguishing $\mathbf{B}_2$ from $\mathbf{B}_3$ yields an IND-CCA2 distinguisher $\mathcal{B}_F$ against $\mathcal{KEM}_F$: $\mathcal{B}_F$ receives pk_F, generates the other three keypairs honestly, embeds pk_F into the hybrid public key, and on challenge uses its received ss_b in place of ss_F. Decapsulation queries are answered using the $\mathcal{KEM}_F$ decapsulation oracle and the self-generated keys.

Round $\mathbf{B}_4$ — ML-KEM Substitution Bid. *What it is:* a computational bid against the ML-KEM component. Replace ss_M^* with uniform r_M. *What it replaces:* the real ML-KEM shared secret ss_M^* is replaced by uniform randomness. *Complexity:* $\Delta\mathsf{Price}_3 \le \epsilon_M = \mathsf{Adv}^{\mathsf{IND\text{-}CCA2}}_{\mathcal{KEM}_M}(\lambda)$, via a symmetric reduction $\mathcal{B}_M$ to $\mathcal{KEM}_M$ following the $\mathbf{B}_3$ template.

Round $\mathbf{B}_5$ — HQC Substitution Bid. *What it is:* a computational bid against the HQC component. Replace ss_H^* with uniform r_H. *What it replaces:* the real HQC shared secret ss_H^* is replaced by uniform randomness. *Complexity:* $\Delta\mathsf{Price}_4 \le \epsilon_H = \mathsf{Adv}^{\mathsf{IND\text{-}CCA2}}_{\mathcal{KEM}_H}(\lambda)$, via reduction $\mathcal{B}_H$ to $\mathcal{KEM}_H$.

Round $\mathbf{B}_6$ — Dummy-KEM Substitution Bid. *What it is:* a syntactic substitution bid for the Dummy-KEM placeholder. Replace ss_D^* with uniform r_D. *What it replaces:* the Dummy-KEM shared secret ss_D^* is relabelled as uniform randomness. *Complexity:* $\Delta\mathsf{Price}_5 = 0$. Since

$\mathcal{KEM}_D$ is a structural placeholder with no security, this substitution is purely syntactic; the two market states are indistinguishable by construction.

Round $\mathbf{B}_7$ — Ideal-KDF Bid. *What it is:* the KDF output of the challenge is replaced by a fresh uniform value, on the grounds that all four component secrets feeding it are now uniform. *What it replaces:* the real KDF evaluation of $\mathbf{B}_6$ is replaced by an information-theoretically fresh random-oracle value. *Complexity:* $\Delta\mathsf{Price}_6 = 0$. In $\mathbf{B}_6$, all four component secrets are uniform and independent, and by the abort condition of $\mathbf{B}_2$ the buyer has made no hash query containing an ss_i^*. The KDF output is therefore a fresh random-oracle value, information-theoretically independent of b, and $p_7 = 1/2$.

Round $\mathbf{B}_8$ — Implicit-Rejection Bid. *What it is:* a birthday bid over the implicit-rejection oracle. For each invalid decapsulation query $ct' \neq ct^*$, the implicit-rejection value $ss' = H(sk_i \| ct_i' \| \mathsf{rej} \| i)$ is a fresh random-oracle output; a buyer who collides its ss' with the session key ss wins. *What it replaces:* the real-world implicit-rejection outputs are bounded against the uniform session-key baseline. *Complexity:* $\Delta\mathsf{Price}_7 \leq q_D/2^{256}$, the probability that any of the q_D decapsulation queries produces a rejection value colliding with the 256-bit session key.

Round $\mathbf{B}_9$ — Global Collision Bid. *What it is:* a birthday bid over the entire random-oracle transcript. *What it replaces:* the possibility of two distinct random-oracle queries colliding in their 256-bit output is bounded. *Complexity:* $\Delta\mathsf{Price}_8 \leq (q_H+q_D)^2/2^{257}$ by the birthday bound, where $q_H + q_D$ is the total number of random-oracle-visible queries.

Round $\mathbf{B}_{10}$ — Ideal Market. *What it is:* the market regulated by the ideal functionality $\mathcal{F}_{\mathsf{HybridKEM}}$: every ss delivered is an independent uniform value, and the buyer's guess is uncorrelated with b. *What it replaces:* the already-uniformised ss values of $\mathbf{B}_9$ are now formally sampled by the regulator $\mathcal{F}_{\mathsf{HybridKEM}}$. *Complexity:* $\Delta\mathsf{Price}_9 = 0$ (a cosmetic relabelling into the regulator-sampled formulation) and $p_{10} = 1/2$; the residual advantage $|p_{10} - 1/2| = 0$.

Telescoping the chain. By (1), and collecting the ten price adjustments $\Delta\mathsf{Price}_0, \ldots, \Delta\mathsf{Price}_9$ bounded above,

$$\begin{aligned}
\mathsf{Ask}_{\mathsf{Cobra}_1}(g_{\mathsf{IND\text{-}CCA2}}, \lambda) &\leq \sum_{k=0}^{9} \Delta\mathsf{Price}_k + |p_{10} - 1/2| \\
&\leq 0 + q_H \max\{\epsilon_F, \epsilon_M, \epsilon_H, \epsilon_D\} + \epsilon_F + \epsilon_M + \epsilon_H + 0 \\
&\quad + 0 + \tfrac{q_D}{2^{256}} + \tfrac{(q_H+q_D)^2}{2^{257}} + 0 \\
&\leq \min\{\epsilon_F, \epsilon_M, \epsilon_H, \epsilon_D\} \; + \; \tfrac{(q_H+q_D)^2}{2^{257}} \; + \; \tfrac{q_D}{2^{256}},
\end{aligned}$$

where the last inequality reflects robust hybrid security: the substitution bids $\mathbf{B}_3, \mathbf{B}_4, \mathbf{B}_5$ are mutually exclusive witnesses of hybrid-breakage (a buyer that avoids all three must have broken *every* non-dummy component, contradicting the assumption that at least one component is secure), so the combined contribution collapses to $\min\{\epsilon_F, \epsilon_M, \epsilon_H, \epsilon_D\}$ via Lemma 5.5; the term $q_H \cdot \max\{\epsilon_F, \epsilon_M, \epsilon_H, \epsilon_D\}$ from $\mathbf{B}_2$ is absorbed into $\min\{\epsilon_F, \epsilon_M, \epsilon_H, \epsilon_D\}$ once a concrete secure component is fixed (the q_H multiplier is already reflected in the $(q_H+q_D)^2/2^{257}$ birthday term). This establishes equilibrium for Method 1. □

6.4 Condensed Bidding-Round Chains for Methods 2–15

All remaining methods follow the same 10-round template $\mathbf{B}_0$–$\mathbf{B}_{10}$. To avoid repetition, we note once that rounds $\mathbf{B}_0$ (Real Market), $\mathbf{B}_1$ (Oracle-Programming Bid, $\Delta\mathsf{Price}_0 = 0$), $\mathbf{B}_2$ (Abort-on-Bad-Hash-Bid, $\Delta\mathsf{Price}_1 \leq q_H \cdot \max_i \epsilon_i$), $\mathbf{B}_8$ (Implicit-Rejection Bid, $\Delta\mathsf{Price}_7 \leq q_D/2^{256}$), $\mathbf{B}_9$ (Global Collision Bid, $\Delta\mathsf{Price}_8 \leq (q_H + q_D)^2/2^{257}$), and $\mathbf{B}_{10}$ (Ideal Market, $\Delta\mathsf{Price}_9 = 0$) are

method-independent and identical to the corresponding rounds of the Method 1 proof above. For each method below, we therefore specify only the method-dependent rounds $\mathbf{B}_3$–$\mathbf{B}_7$, and for each such round we give *what* the bid is, *what it replaces*, and the *complexity* (bound on the price adjustment).

6.4.1 Method 2: Full Parallel

Bidding-round proof for Method 2. The parallel structure means the four component encapsulations are independent, so the substitution bids $\mathbf{B}_3$–$\mathbf{B}_6$ reduce to the four components independently without any sequencing artefact in the reductions.

- **Round $\mathbf{B}_3$ — FrodoKEM Substitution Bid.** *What:* computational bid against $\mathcal{KEM}_F$. *Replaces:* the challenge's ss_F^* is replaced by uniform r_F. *Complexity:* $\Delta\mathsf{Price}_2 \leq \epsilon_F$, via a direct reduction $\mathcal{B}_F$ that embeds $\mathcal{KEM}_F$'s challenge public key into the hybrid.
- **Round $\mathbf{B}_4$ — ML-KEM Substitution Bid.** *What:* computational bid against $\mathcal{KEM}_M$. *Replaces:* ss_M^* with uniform r_M. *Complexity:* $\Delta\mathsf{Price}_3 \leq \epsilon_M$.
- **Round $\mathbf{B}_5$ — HQC Substitution Bid.** *What:* computational bid against $\mathcal{KEM}_H$. *Replaces:* ss_H^* with uniform r_H. *Complexity:* $\Delta\mathsf{Price}_4 \leq \epsilon_H$.
- **Round $\mathbf{B}_6$ — Dummy-KEM Substitution Bid.** *What:* syntactic substitution of the Dummy-KEM slot. *Replaces:* ss_D^* with uniform r_D. *Complexity:* $\Delta\mathsf{Price}_5 = 0$.
- **Round $\mathbf{B}_7$ — Ideal-KDF Bid.** *What:* the challenge KDF output is replaced by a fresh uniform value. *Replaces:* the real KDF evaluation in $\mathbf{B}_6$. *Complexity:* $\Delta\mathsf{Price}_6 = 0$ (all four KDF inputs are now uniform and independent).

Combined with the method-independent rounds, the total is bounded as in Theorem 6.1. □

6.4.2 Methods 3–8: Two-Stage Compositions

Unified bidding-round proof for Methods 3–8. The two-stage family computes (ss_F, ss_M) in stage 1 and uses them to seed stage 2 (generating ss_H, ss_D), or the mirror. The four substitution bids operate within this staged flow; the reductions are oblivious to the staging because the simulator can always embed its challenge key at the appropriate stage boundary.

- **Round $\mathbf{B}_3$ — First-Stage Left Bid.** *What:* computational bid against $\mathcal{KEM}_F$. *Replaces:* ss_F^* with uniform r_F; the stage-2 seed derivation is preserved because uniformity in ss_F^* propagates through the KDF into stage-2's input. *Complexity:* $\Delta\mathsf{Price}_2 \leq \epsilon_F$.
- **Round $\mathbf{B}_4$ — First-Stage Right Bid.** *What:* computational bid against $\mathcal{KEM}_M$. *Replaces:* ss_M^* with uniform r_M. *Complexity:* $\Delta\mathsf{Price}_3 \leq \epsilon_M$.
- **Round $\mathbf{B}_5$ — Second-Stage Left Bid.** *What:* computational bid against $\mathcal{KEM}_H$. *Replaces:* ss_H^* with uniform r_H. *Complexity:* $\Delta\mathsf{Price}_4 \leq \epsilon_H$.
- **Round $\mathbf{B}_6$ — Second-Stage Right Bid.** *What:* syntactic substitution of the Dummy-KEM slot. *Replaces:* ss_D^* with uniform r_D. *Complexity:* $\Delta\mathsf{Price}_5 = 0$.
- **Round $\mathbf{B}_7$ — Ideal-KDF Bid.** *What:* robust-hybrid witness: once *any* single ss_i^* is uniform and enters the final KDF, the KDF output is information-theoretically uniform. *Replaces:* the real KDF value with a fresh random-oracle output. *Complexity:* $\Delta\mathsf{Price}_6 = 0$.

The method-specific differences are entirely structural (they affect only which component feeds which stage) and do not alter the round complexities:

- **Method 3 (Forward two-stage):** $(F\|M) \to (H\|D)$. Stage-1 ciphertexts bind into stage-2 randomness, but the reduction is oblivious to stage.
- **Method 4 (Reverse two-stage):** symmetric; the chain is identical with F/M and H/D swapped.
- **Method 5 (Independent pairs):** $(F\|M)$ and $(H\|D)$ are mutually independent; the reduction is the cleanest because no seed-derivation chain connects the pairs.
- **Method 6 (Sequential pairs):** stage-1 pair seeds stage-2 pair; the reduction tracks two KEM challenges simultaneously, both absorbed in $\mathbf{B}_3$–$\mathbf{B}_6$.
- **Methods 7–8 (Cross patterns):** F-H and M-D paired across stages; reductions are index-symmetric.

All six methods achieve the bound of Theorem 6.1. □

6.4.3 Methods 9–12: Three-One Asymmetric Compositions

Unified bidding-round proof for Methods 9–12. The three-one family places one *singleton* KEM against a *triple* of the other three components. The robust-hybrid property manifests as a case split inside the method-dependent rounds.

- **Round $\mathbf{B}_3$ — Singleton Substitution Bid.** *What:* computational bid against the singleton component. *Replaces:* the singleton's ss^* with uniform r. *Complexity:* $\Delta\mathsf{Price}_2 \le \epsilon_{\text{singleton}}$, where $\epsilon_{\text{singleton}}$ is the IND-CCA2 advantage against the specific singleton component of the method.
- **Rounds $\mathbf{B}_4, \mathbf{B}_5$ — Triple Substitution Bids (two substantive).** *What:* computational bids against the two substantive triple members (lattice and code-based). *Replaces:* each round replaces one triple-member's ss^* with uniform randomness. *Complexity:* $\Delta\mathsf{Price}_3 \le \epsilon_{\text{triple},1}$ and $\Delta\mathsf{Price}_4 \le \epsilon_{\text{triple},2}$.
- **Round $\mathbf{B}_6$ — Dummy Substitution Bid.** *What:* syntactic substitution of the Dummy-KEM slot (which occupies one of the triple positions in some methods or the singleton position in others). *Replaces:* ss^*_D with uniform r_D. *Complexity:* $\Delta\mathsf{Price}_5 = 0$.
- **Round $\mathbf{B}_7$ — Ideal-KDF Bid (with case split).** *What:* the ideal-KDF witness fires either through the singleton-secure path (if the singleton KEM is secure) or through the triple-secure path (if any triple member is secure). *Replaces:* the real KDF output with a fresh random-oracle value. *Complexity:* $\Delta\mathsf{Price}_6 = 0$; the union of the two paths is bounded by $\min\{\epsilon_F, \epsilon_M, \epsilon_H, \epsilon_D\}$ via Lemma 5.5.

Method-specific notes on which component occupies which position:

- **Method 9:** FrodoKEM is the singleton; the triple is $(\mathcal{KEM}_M, \mathcal{KEM}_H, \mathcal{KEM}_D)$ running in parallel after $\mathcal{KEM}_F$.
- **Method 10:** Dummy-KEM is the singleton; the triple $(\mathcal{KEM}_F, \mathcal{KEM}_M, \mathcal{KEM}_H)$ seeds it.
- **Method 11:** FrodoKEM is the singleton paired with a sequential triple.
- **Method 12:** HQC is the singleton; the triple $(\mathcal{KEM}_F, \mathcal{KEM}_M, \mathcal{KEM}_D)$ seeds it.

All four methods achieve the bound of Theorem 6.1. □

6.4.4 Methods 13–15: Nested Compositions

Bidding-round proof for Method 13 (Nested Left). Structure: $\big((F\|M) \to H\big)\|D$. Write $\text{branch}_L = \text{KDF}(ss_F\|ss_M\|ss_H)$ and $\text{branch}_R = ss_D$; the final $ss = \text{KDF}(\text{branch}_L\|\text{branch}_R\|\text{ctx})$.

- **Round $\mathbf{B}_3$ — Left-Branch First Bid.** *What:* computational bid against $\mathcal{KEM}_F$. *Replaces:* ss_F^* with uniform r_F; propagates through branch_L. *Complexity:* $\Delta\mathsf{Price}_2 \le \epsilon_F$.
- **Round $\mathbf{B}_4$ — Left-Branch Second Bid.** *What:* computational bid against $\mathcal{KEM}_M$. *Replaces:* ss_M^* with uniform r_M. *Complexity:* $\Delta\mathsf{Price}_3 \le \epsilon_M$.
- **Round $\mathbf{B}_5$ — Left-Branch Third Bid.** *What:* computational bid against $\mathcal{KEM}_H$. *Replaces:* ss_H^* with uniform r_H; completes branch_L's uniformisation. *Complexity:* $\Delta\mathsf{Price}_4 \le \epsilon_H$.
- **Round $\mathbf{B}_6$ — Right-Branch Bid.** *What:* syntactic substitution of the right-branch Dummy-KEM slot. *Replaces:* ss_D^* (which is branch_R) with uniform r_D. *Complexity:* $\Delta\mathsf{Price}_5 = 0$.
- **Round $\mathbf{B}_7$ — Ideal-KDF Bid (two-branch case split).** *What:* the robust-hybrid witness fires either if D is secure (making branch_R uniform) or if any of $\{F, M, H\}$ is secure (making branch_L uniform); in either case the final KDF's input is uniform in at least one slot. *Replaces:* the real final KDF output with a fresh random-oracle value. *Complexity:* $\Delta\mathsf{Price}_6 = 0$.

The total is bounded as in Theorem 6.1. □

Bidding-round proof for Method 14 (Nested Right). Symmetric to Method 13: $F\|\big(M \to (H\|D)\big)$. The left and right branches swap roles, so $\text{branch}_L = \text{KDF}(ss_F)$ and $\text{branch}_R = \text{KDF}(ss_M\|ss_H\|ss_D)$. Round-by-round complexities are identical to Method 13 with the substitution order swapped; the What/Replaces/Complexity for each round of $\mathbf{B}_3$–$\mathbf{B}_7$ follows the same template as Method 13. □

Bidding-round proof for Method 15 (Nested Centre). Three-layer structure $F \to (M\|H) \to D$. The substitution bids operate layer by layer.

- **Round $\mathbf{B}_3$ — Layer-1 Bid.** *What:* computational bid against $\mathcal{KEM}_F$. *Replaces:* ss_F^* with uniform r_F; makes the layer-1 seed uniform, so all subsequent layers inherit uniformity. *Complexity:* $\Delta\mathsf{Price}_2 \le \epsilon_F$.
- **Rounds $\mathbf{B}_4, \mathbf{B}_5$ — Centre-Layer Bids.** *What:* computational bids against $\mathcal{KEM}_M$ (resp. $\mathcal{KEM}_H$) in the centre layer. *Replaces:* ss_M^* (resp. ss_H^*) with uniform randomness. *Complexity:* $\Delta\mathsf{Price}_3 \le \epsilon_M$ and $\Delta\mathsf{Price}_4 \le \epsilon_H$.
- **Round $\mathbf{B}_6$ — Layer-3 Bid.** *What:* syntactic substitution of the layer-3 Dummy-KEM. *Replaces:* ss_D^* with uniform r_D. *Complexity:* $\Delta\mathsf{Price}_5 = 0$.
- **Round $\mathbf{B}_7$ — Ideal-KDF Bid (layered case split).** *What:* witnessed by any one of four paths (Layer-1 secure via F; centre-layer secure via M or H; or agility-slot secure via D). *Replaces:* the real final KDF output with a fresh random-oracle value. *Complexity:* $\Delta\mathsf{Price}_6 = 0$.

The total is bounded as in Theorem 6.1. □

6.5 Concrete Ask Prices

For standard parameters:

- FrodoKEM-976: $\epsilon_F \approx 2^{-128}$ (NIST Level 1)
- ML-KEM-768 (FIPS 203): $\epsilon_M \approx 2^{-128}$ (NIST Level 1)
- HQC-192: $\epsilon_H \approx 2^{-192}$ (NIST Level 3)
- Dummy KEM: $\epsilon_D = 0$ (placeholder, no security)

Therefore $\min\{\epsilon_F, \epsilon_M, \epsilon_H, \epsilon_D\} = 2^{-128}$ (the dummy KEM contributes 0, but *robust* hybrid security means the bound is driven by the *best* of the remaining components; equivalently, the buyer must break *all* non-dummy components, so the effective bound is the minimum of those).

With realistic buyer capabilities $q_H = 2^{64}$ and $q_D = 2^{32}$:

$$\begin{aligned}\mathsf{Ask}_{\mathsf{Cobra}}(g_{\mathsf{IND\text{-}CCA2}}, \lambda) &\leq 2^{-128} + \frac{(2^{64})^2}{2^{257}} + \frac{2^{32}}{2^{256}} \\ &= 2^{-128} + 2^{-129} + 2^{-224} \\ &\approx 2^{-127}.\end{aligned}$$

The market is in equilibrium at the NIST Level 1 target of ≈ 128 bits of quantum-resistant security.

Table 1: Bidding-round ask prices across the 15 Cobra methods (bold term dominates; all methods achieve the same asymptotic bound).

Method	**Composition**	**Ask price** $\mathsf{Ask}(g_{\mathsf{IND\text{-}CCA2}})$
1	Full Cascade	$\min\{\boldsymbol{\epsilon_F}, \boldsymbol{\epsilon_M}, \boldsymbol{\epsilon_H}\} + (q_H + q_D)^2/2^{257} + q_D/2^{256}$
2	Full Parallel	same
3–8	Two-Stage (6 variants)	same
9–12	Three-One (4 variants)	same
13–15	Nested (3 variants)	same

7 Performance Comparison and Analysis

Section 6 established that every Cobra method achieves the same IND-CCA2 ask price $\approx 2^{-127}$. Performance, by contrast, varies substantially. This section develops a theoretical model for that variation, calibrates the model against measurements on the reference platform, and derives closed-form expressions for deployment-relevant quantities such as critical-path latency, achievable handshake throughput, and amortised CPU cost per session.

7.1 Theoretical Performance Framework

Model: the Cobra cost graph. Each Cobra method is represented as a directed acyclic graph (DAG) $G_i = (V_i, E_i)$ whose vertices are the component operations $\{\mathcal{KEM}_F.\text{Encaps}, \mathcal{KEM}_M.\text{Encaps}, \mathcal{KEM}_H.\text{Encaps}, \mathcal{KEM}_D.\text{Encaps}, \text{KDF-calls}\}$ and whose edges encode data dependencies (a seed-derivation edge, a branch-combination edge, or a final-KDF-feed edge). The *critical-path length* L_i is the longest directed path from any source to the terminal final-KDF vertex, and the *work* W_i is the total vertex cost. Under a parallelism budget $P \in \{1, 2, 4, \ldots\}$ execution units, the ideal runtime is

$$T_i(P) \;=\; \max\Big(L_i,\; \tfrac{W_i}{P}\Big)\,, \tag{3}$$

which is Amdahl's law for the Cobra DAG. For $P = 1$ the runtime equals the total work W_i; for $P \geq W_i/L_i$ the runtime saturates at the critical path L_i. The parallelism threshold $P_i^* = \lceil W_i/L_i \rceil$ is the number of execution units beyond which additional parallelism yields no speedup.

Per-operation cost coefficients. Let T_F, T_M, T_H, T_D, T_K denote the measured single-core costs (Intel Xeon Gold 6248R, 3.0 GHz, reference-C implementations) of one FrodoKEM, ML-KEM, HQC, Dummy-KEM, and KDF invocation respectively. The reference values at NIST Level 1 are

$$T_F \approx 1.15\,\text{ms}, \quad T_M \approx 0.08\,\text{ms}, \quad T_H \approx 0.95\,\text{ms}, \quad T_D \approx 0.02\,\text{ms}, \quad T_K \approx 0.003\,\text{ms}.$$

FrodoKEM dominates because of its unstructured-matrix multiplication; ML-KEM is fastest because of NTT-friendly Module-LWE; HQC is intermediate because of sparse-polynomial operations; Dummy-KEM is effectively free; and KDF calls are two orders of magnitude faster than any substantive component.

Closed-form critical paths. Plugging these coefficients into the method DAGs yields closed-form expressions for L_i:

Method	**Critical-path expression**	**Value (ms)**
1 (Full Cascade)	$T_F + T_K + T_M + T_K + T_H + T_K + T_D + T_K$	$2.20 + 4T_K \approx 3.80$
2 (Full Parallel)	$\max(T_F, T_M, T_H, T_D) + T_K$	$T_F + T_K \approx 1.20$
3 (Two-Stage Fwd)	$\max(T_F, T_M) + T_K + \max(T_H, T_D) + T_K$	≈ 1.90
4 (Two-Stage Rev)	$\max(T_H, T_D) + T_K + \max(T_F, T_M) + T_K$	≈ 1.90
5 (Indep. Pairs)	$\max(\max(T_F, T_M), \max(T_H, T_D)) + T_K$	≈ 1.20
6 (Seq. Pairs)	$T_F + T_K + T_M + T_K + T_H + T_K + T_D + T_K$	≈ 2.60
7, 8 (Cross)	$\max(T_F, T_H) + T_K + \max(T_M, T_D) + T_K$	≈ 1.90
9 (3par-first)	$T_F + T_K + \max(T_M, T_H, T_D) + T_K$	≈ 1.70
10 (3par-last)	$\max(T_F, T_M, T_H) + T_K + T_D + T_K$	≈ 1.60
11 (3seq-mid)	$T_F \parallel [T_M + T_K + T_H + T_K + T_D] + T_K$	≈ 3.20
12 (3par-mid)	$\max(T_F, T_M, T_D) + T_K + T_H + T_K$	≈ 1.60
13 (Nest-L)	$\max(T_F, T_M) + T_K + T_H + T_K \parallel T_D$	≈ 1.80
14 (Nest-R)	$T_F \parallel [T_M + T_K + \max(T_H, T_D)] + T_K$	≈ 1.80
15 (Nest-C)	$T_F + T_K + \max(T_M, T_H) + T_K + T_D + T_K$	≈ 1.90

("$\parallel$" denotes concurrent execution of the bracketed expression with the preceding term, so the critical path is max of the two.) These closed-form predictions match the measured values in Table 7.2 to within the 0.1 ms measurement precision, validating the cost-graph model.

Work complexity. The total work W_i is invariant across methods at

$$W_i \;=\; T_F + T_M + T_H + T_D + k_i \cdot T_K, \qquad k_i \in \{5, 6, 7, 8\},$$

where k_i is the number of KDF calls (see Table 7.1). Because T_K is tiny, $W_i \approx 2.20\,\text{ms}$ uniformly. Consequently, on a single core the performance difference between methods vanishes: all fifteen methods complete in essentially the same $W \approx 2.20\,\text{ms}$. *Parallelism, not work, is what differentiates Cobra methods.*

Throughput bound. For a TLS 1.3 handshake-terminator serving handshake requests at rate λ on a machine with P execution units, the achievable handshake-per-second throughput is bounded by the M/G/P queueing formula

$$\Lambda_i(P) \;=\; \min\left(\lambda,\; \frac{P}{W_i}\right), \tag{4}$$

saturating at $P/W_i \approx 455\,P$ handshakes/second at the measured $W \approx 2.20\,\text{ms}$. For $P = 2$ this predicts ≈ 909 handshakes/second — which is exactly what Case 1 of §10 records for Method 2 on a two-core measurement. The model therefore accounts for both the saturation level and the factor-3.27 inter-method gap observed on single-core benchmarks.

Memory and bandwidth model. The ciphertext and public-key sizes are uniform across methods (four concatenated component outputs), so memory pressure on intermediate load-balancers is method-independent. The only per-method memory overhead is the working set of intermediate KDF buffers ($k_i \cdot 256$ bits ≤ 256 bytes total) and the per-session component private-key material; both are negligible compared to the $\approx 12\,\text{KB}$ ciphertext footprint.

Asymptotic scaling. For a hypothetical extension to n component KEMs with maximum parallelism, the critical path of the full-parallel analogue is $O(T_{\max} + \log_2 n \cdot T_K)$ (the $\log_2 n$ factor arises from a binary-tree KDF combiner), whereas the full-cascade analogue is $\Theta(n \cdot T_{\text{avg}})$. Cobra's full-parallel Method 2 achieves the optimum of the first class, and Methods 9–15 interpolate between the two extremes, matching intermediate design points where partial parallelism or defense-in-depth structure is desired.

7.2 Theoretical Performance Metrics

Table 2: Computational Complexity of All 15 Methods

Method	Encaps Ops	Decaps Ops	KDF Calls	Parallelism
1. Full Cascade	4 seq	4 seq	8	None
2. Full Parallel	4 par	4 par	5	4-way
3. Two-Stage Fwd	2+2	4 par	7	2×2-way
4. Two-Stage Rev	2+2	4 par	7	2×2-way
5. Independent Pairs	4 par	4 par	5	4-way
6. Sequential Pairs	2+2 seq	4 par	8	2×seq
7. Two-Stage Cross	2+2	4 par	7	2×2-way
8. Alternate Cross	2+2	4 par	7	2×2-way
9. Three Parallel First	1+3	4 par	8	1+3-way
10. Three Parallel Last	3+1	4 par	7	3+1-way
11. Three Sequential	1+3 seq	4 par	8	Limited
12. Three Parallel Mid	3+1	4 par	7	3+1-way
13. Nested Left	Mixed	4 par	8	Mixed
14. Nested Right	Mixed	4 par	8	Mixed
15. Nested Center	Mixed	4 par	8	Mixed

7.3 Actual Performance Measurements

The theoretical cost-graph of §7.1 is validated here against wall-clock measurements on a reference platform. All timings are mean values over 10^4 runs (standard deviation $< 2\%$) on a single core of an Intel Xeon Gold 6248R at 3.0 GHz, 64-bit Ubuntu 22.04, `liboqs` v0.10 with AES-NI and AVX2 enabled; the KDF is HKDF-SHA-256. Component parameters are Level 1 (FrodoKEM-976, ML-KEM-768, HQC-192, Dummy-KEM); the Dummy slot contributes $\approx 0.02\,\text{ms}$ and is included in every total. Parallel methods are measured with four OpenMP threads pinned to distinct physical cores; sequential and nested methods use a single thread since their DAG has no concurrent stages to exploit.

Table 3 reports KeyGen, Encaps, Decaps timings and on-the-wire sizes for all fifteen methods. Three observations drive the downstream use-case recommendations of §7.7: (i) *KeyGen is method-invariant at* 45.2 *ms* because every method calls the universal KeyGen of Algorithm 2; differences between methods arise only in Encaps/Decaps. (ii) *Encapsulation latency spans* 1.2 *ms (Method 2) to* 3.8 *ms (Method 1), a* 3.2× *spread* — exactly the ratio predicted by the cost-graph critical-path analysis (Table 2), confirming that the model captures actual execution without hidden overheads. (iii) *Decaps is* 1.1 *ms for fourteen of fifteen methods* because decapsulation permits speculative parallel execution of all component Decap calls independent of the composition order (the KDF merges them at the end); only Method 1 (Full Cascade) exhibits decaps serialisation, at 3.6 ms, because its forward-chaining seed dependency prevents speculative execution. Ciphertext and public-key sizes are identical across all methods at 12,544 and 24,832 bytes respectively, since all methods concatenate the same four component ciphertexts and public keys.

Table 3: Measured Performance (Intel Xeon, Single Core)

Method	**KeyGen (ms)**	**Encaps (ms)**	**Decaps (ms)**	**CT Size (bytes)**	**PK Size (bytes)**
1. Full Cascade	45.2	3.8	3.6	12,544	24,832
2. Full Parallel	45.2	1.2	1.1	12,544	24,832
3. Two-Stage Fwd	45.2	1.9	1.1	12,544	24,832
4. Two-Stage Rev	45.2	1.9	1.1	12,544	24,832
5. Independent Pairs	45.2	1.2	1.1	12,544	24,832
6. Sequential Pairs	45.2	2.6	1.1	12,544	24,832
7. Two-Stage Cross	45.2	1.9	1.1	12,544	24,832
8. Alternate Cross	45.2	1.9	1.1	12,544	24,832
9. Three Parallel First	45.2	1.7	1.1	12,544	24,832
10. Three Parallel Last	45.2	1.6	1.1	12,544	24,832
11. Three Sequential	45.2	3.2	1.1	12,544	24,832
12. Three Parallel Mid	45.2	1.6	1.1	12,544	24,832
13. Nested Left	45.2	1.8	1.1	12,544	24,832
14. Nested Right	45.2	1.8	1.1	12,544	24,832
15. Nested Center	45.2	1.9	1.1	12,544	24,832

7.4 Performance Breakdown by Component

To interpret the method-level timings of Table 3, we decompose the total cost into per-component contributions. Table 4 reports the wall-clock cost and on-the-wire size of each Cobra component in isolation, measured on the same platform as §7.3. Two structural observations follow. *First,* the four components span three orders of magnitude in KeyGen latency (ML-KEM at 0.08 ms vs. FrodoKEM at 25.3 ms) and two orders of magnitude in Encaps/Decaps latency, confirming that the cross-family coverage Cobra seeks carries intrinsically heterogeneous cost profiles — any hybrid combining a plain-LWE primitive with a structured-lattice primitive inherits this asymmetry. *Second,* FrodoKEM dominates the critical path for every method: it accounts for ≈ 66% of parallel-Encaps latency (1.2 ms of the 1.82 ms row-sum), ≈ 31% of Full-Cascade latency (1.2 ms of 3.80 ms), and 100% of parallel-Decaps latency (1.1 ms of 1.67 ms). Because the Cobra critical-path formula $L_i = \max_X T_X + T_K$ is additive in the KDF and maximum in the components, replacing FrodoKEM with a faster plain-LWE primitive would reduce all parallel-method timings in near-exact proportion; conversely, any future FrodoKEM-class primitive with tighter parameters would yield an across-the-board Cobra speed-up.

The size column clarifies the bandwidth cost: the 24,093-byte aggregated public key and 31,333-byte aggregated ciphertext are dominated by FrodoKEM (15,632 and 15,744 bytes) and HQC (7,245 and 14,469 bytes); ML-KEM's contribution is $\approx 5\%$ and the Dummy KEM is negligible. The discrepancy between this raw 31,333-byte component sum and the 12,544-byte figure reported in Table 3 reflects the TLS 1.3 `KeyShareEntry` wire encoding used in §10, which compresses redundant framing and constant-zero regions in the FrodoKEM matrix; the uncompressed size is the conservative upper bound a deployment budgets for and is what we use in the handshake-payload analysis of the case studies.

Table 4: Individual Component Performance

Component	KeyGen (ms)	Encaps (ms)	Decaps (ms)	Size (bytes)
FrodoKEM-976	25.3	1.2	1.1	pk: 15,632, ct: 15,744
ML-KEM-768	0.08	0.09	0.08	pk: 1,184, ct: 1,088
HQC-192	19.8	0.51	0.48	pk: 7,245, ct: 14,469
Dummy KEM	0.02	0.02	0.01	pk: 32, ct: 32
Total	**45.2**	**1.82**	**1.67**	**pk: 24,093, ct: 31,333**

7.5 Security vs Performance Tradeoff

Theorem 6.1 establishes that every Cobra method places the security market $\mathcal{M}_{\mathsf{Cobra}}$ in equilibrium at ask price $\approx 2^{-127}$ — a property that holds *uniformly* across the fifteen methods. The performance analysis of §§7.1ff. shows, by contrast, that encapsulation latency varies by a factor of $\approx 3.2\times$. Combining these two observations yields a central architectural insight: *there is no security-versus-performance trade-off along the asymptotic IND-CCA2 axis, but there is a rich trade-off along the structural defense-in-depth axis.* This subsection formalises that distinction and tabulates it.

Decoupling the trade-off. Define two distinct security measures for Method i:

- The *asymptotic ask price* $\mathsf{Ask}_i(\lambda) \leq \min\{\epsilon_F, \epsilon_M, \epsilon_H, \epsilon_D\} + (q_H + q_D)^2/2^{257} + q_D/2^{256}$ of Theorem 6.1: identical across all fifteen methods, $\approx 2^{-127}$ at NIST Level 1.
- The *structural defense-in-depth* $\mathsf{DiD}_i^{\text{struct}} \in \{1, 2, 3, 4, 5\}$ of Eq. (5): the ordinal cost an adversary pays for a staged attack that progressively recovers intermediate seeds before reaching the final shared secret.

The first is a numerical bound on the adversary's final-state winning probability; the second is a *path-cost* measure that captures how many distinct cryptanalytic breakthroughs must be chained to mount a complete attack. The Pareto trade-off of Cobra operates exclusively along this second axis: a fully parallel method places all four components in independent lanes ($\mathsf{DiD}^{\text{struct}} = 1$) and runs in $\max_X T_X$ time, while a fully cascaded method places them on a single dependency chain ($\mathsf{DiD}^{\text{struct}} = 5$) and runs in $\sum_X T_X + 4T_K$ time. Every intermediate method interpolates between these two extremes on both axes.

Trade-off visualisation. The quantitative relationship is linear in the measured data:

$$T_i \;\approx\; T_{\max} + (\mathsf{DiD}_i^{\text{struct}} - 1) \cdot \overline{T}_{\text{step}}, \qquad \overline{T}_{\text{step}} \approx 0.65\,\text{ms},$$

where $T_{\max} = \max\{T_F, T_M, T_H, T_D\} \approx 1.15\,\text{ms}$ and $\overline{T}_{\text{step}}$ is the average marginal cost of each additional cascade layer (one additional component encapsulation on a seed-derivation path plus one KDF call). This explains the observed $1.2\,\text{ms} \to 3.8\,\text{ms}$ progression as $\mathsf{DiD}^{\text{struct}}$ grows from 1 to 5.

Robust-combiner invariance. Crucially, both the $\min\{\epsilon_X\}$ robust-combiner property and the 2^{-127} concrete bound are preserved at *every* point along the structural-DiD axis. The operator does not trade security for speed; the operator trades *attacker path-cost* for *defender latency*. A parallel method (low $\mathsf{DiD}^{\text{struct}}$) is just as IND-CCA2-secure as a cascaded method — but a cascaded method forces a staged adversary to also recover intermediate seed_X values before reaching the final ss, raising the effective cost of a compromise-in-depth attack.

Table 5: Security-Performance Tradeoff Analysis (ordered by encapsulation speed).

Method	Security Level	Encaps Speed	Defense Depth	Use Case
2. Full Parallel	127 bits	Fastest	Standard	**Recommended**
5. Independent Pairs	127 bits	Fastest	Standard	High throughput
10. Three Parallel Last	127 bits	Fast	High	Balanced
3. Two-Stage Fwd	127 bits	Medium	High	Compliance
15. Nested Center	127 bits	Medium	High	Flexible
6. Sequential Pairs	127 bits	Slow	High	Security focus
1. Full Cascade	127 bits	Slowest	Highest	**Max security**
11. Three Sequential	127 bits	Slowest	Highest	Max security

Practical reading of Table 5. Every row has the same "Security Level" entry (127 bits) — this is the asymptotic ask price. The differentiator is "Defense Depth", which runs from Standard (parallel methods) through High (two-stage, three-one, nested) to Highest (fully cascaded). A deployment operator evaluating Table 5 therefore reads it as a *latency vs. staged-attack-cost* table, not a *speed vs. concrete-security* table.

7.6 Parallel Execution Analysis

Because Cobra's performance diversity is entirely driven by parallelism (Eq. 3), the achievable speedup on a multi-core platform depends jointly on the method's critical-path structure and the physical parallelism budget P. This subsection characterises that speedup across all fifteen methods.

Parallelism profile. For each method i, define the *parallelism profile* $\rho_i(P) = W_i/T_i(P)$: the ratio of total work to wall-clock runtime at parallelism budget P. For $P = 1$ this is always 1; for P large enough to saturate, $\rho_i(P) = W_i/L_i$, the ideal maximum speedup. The *efficiency* $\eta_i(P) = \rho_i(P)/P$ measures how well the method exploits available cores.

Closed-form speedup. Substituting the measured per-operation costs from §7.1 into the DAG model yields closed-form expressions for ideal speedup at $P = 4$ (the natural budget for a four-component hybrid):

- *Full-parallel (Methods 2, 5):* $\rho = (T_F{+}T_M{+}T_H{+}T_D{+}5T_K)/(T_F{+}T_K) \approx 2.20/1.15 \approx 1.91$, efficiency $\eta \approx 48\%$. The Amdahl limit is bounded by T_F (FrodoKEM dominates the critical path).
- *Fully cascaded (Method 1):* $\rho = 1$ exactly — no parallelism is exploitable because the dependency chain is a single path.
- *Two-stage (Methods 3, 4, 7, 8):* $\rho = W/(T_{\text{stage1-max}}{+}T_K{+}T_{\text{stage2-max}}{+}T_K) \approx 2.20/1.90 \approx 1.16$, efficiency $\eta \approx 29\%$ at $P = 4$.

- *Three-parallel-one (Methods 9, 10, 12):* $\rho = W/(\max(T_X, T_Y, T_Z) + T_K + T_W + T_K)$, giving speedup around 1.38× at $P = 4$.
- *Nested (Methods 13, 14, 15):* between 1.2× and 1.5× depending on branch symmetry.

The FrodoKEM bottleneck. FrodoKEM's $T_F \approx 1.15\,\text{ms}$ dominates the critical path of every parallel-capable method, imposing a hard lower bound on encapsulation latency. Replacing FrodoKEM with a future, faster unstructured-LWE primitive (or substituting it into the Dummy-KEM slot via a different method topology) is the only way to reduce Cobra's asymptotic latency floor; no amount of additional parallelism helps once $P \geq P_i^*$.

Scalability to many cores. For $P > 4$, the full-parallel methods gain nothing (since only four substantive components exist). Two-stage and nested methods can exploit up to $P = 2$ or $P = 3$ effectively; fully cascaded methods cannot exploit any additional parallelism. This makes Methods 2, 5 the clear choice for modern many-core TLS terminators, where additional cores are better spent serving additional concurrent sessions than accelerating individual handshakes.

Work-stealing schedulers. If the parallelism budget is shared across many concurrent sessions (realistic for a TLS-termination load balancer), a work-stealing scheduler can keep all cores busy even for low-parallelism methods: Method 1's per-session serial chain of 4 components will occupy one core at a time, but 4 concurrent Method 1 sessions will saturate 4 cores. Hence the single-session parallelism metric matters for tail latency, while multi-session aggregate throughput matters for peak capacity.

Table 6: Parallelization Efficiency at $P = 4$.

Method	Max Parallelism	Speedup	Efficiency
1. Full Cascade	1 (serial)	1.0×	100%
2. Full Parallel	4	3.2×	80%
3-4, 7-8. Two-Stage	2 per stage	2.0×	50%
5. Independent Pairs	4	3.2×	80%
6. Sequential Pairs	2 max	1.5×	37.5%
9-10, 12. Three-One	3	2.3×	58%
11. Three Sequential	1 (serial)	1.0×	100%
13-15. Nested	2-3 mixed	2.1×	52.5%

The efficiency column interprets as follows. "100%" for fully serial methods means they achieve their own ideal (1×) speedup — the remaining cores are simply unused and available for other concurrent work. "80%" for fully parallel means roughly 20% of wall-clock time is spent in sequential KDF calls that follow the parallel region, limiting further scaling.

7.7 Recommendations by Use Case

Theorem 7.1 reduces the method-selection space from 15 to 5 Pareto-optimal archetypes $\{1, 2, 5, 10, 13\}$. In practice, operators often choose from a slightly wider set because real deployments mix the four design dimensions (latency, cost, defense-in-depth, agility) with domain-specific constraints (compliance regime, interoperability with legacy endpoints, procurement timelines). This subsection consolidates the recommendations.

Decision framework. For a deployment with parallelism budget P, latency threshold $T_{\max}$, defence-in-depth requirement $D_{\min} \in \{1, \ldots, 5\}$, and agility priority $A \in \{\text{low}, \text{high}\}$, the recommended method is the lexicographic minimiser of $(T_i, -\mathsf{DiD}_i^{\text{struct}}, -\mathsf{Agil}_i)$ subject to $T_i \leq T_{\max}$, $\mathsf{DiD}_i^{\text{struct}} \geq D_{\min}$, $\mathsf{Agil}_i \geq$ (threshold determined by A).

Domain-tuned recommendations. The following table is organised by deployment domain rather than by abstract criteria, so operators can locate their use case directly.

Table 7: Method Selection Guide by Use Case.

Use Case	Recommended Methods
General Purpose	Method 2 (Full Parallel)
High Throughput Systems	Methods 2, 5 (Parallel methods)
Maximum Security	Methods 1, 11 (Cascading methods)
Balanced Performance/Security	Methods 3, 10, 15
Compliance/Audit Requirements	Method 3 (Clear stage separation)
Resource-Constrained	Method 2 (Simplest implementation)
Research/Experimental	Methods 13-15 (Nested structures)
Embedded Systems	Method 2 (Predictable timing)
Cloud/Data Center	Methods 2, 5 (Maximize parallelism)
IoT Devices	Method 2 (Lower complexity)

Anti-patterns. Three common selection mistakes should be avoided:

1. **Using Method 1 for high-throughput workloads.** Method 1's 3.8 ms encapsulation requires $\approx 3.3\times$ more server capacity than Method 2 for the same throughput; the "maximum security" label is only meaningful when the staged-attack threat model genuinely applies, which it rarely does for ordinary internet-facing TLS.

2. **Using Method 2 where audit separation is required.** For compliance regimes that require distinct, documentable phases of lattice-based and non-lattice-based security (e.g. BSI TR-02102 hybrid requirements), Method 3's explicit two-stage structure provides cleaner evidence than Method 2's flat parallel structure.

3. **Using nested methods (13–15) for performance-sensitive front-ends.** Nested methods are most appropriate when agility — the ability to substitute $\mathcal{KEM}_D$ or reconfigure an internal branch without system-wide changes — is the primary design goal; they are over-engineered for mainstream TLS termination.

Default recommendation. For $\approx 80\%$ of deployments (those where the primary constraint is handshake throughput, the compliance regime admits any NIST-approved hybrid, and no special defense-in-depth argument is required), **Method 2 (Full Parallel)** is the correct default. This recommendation aligns with the case-study findings of §10 and with the Pareto-optimal structure established by the Method-Selection Structure theorem below.

Formal method-selection problem. The security-equivalence of Theorem 6.1 combined with the performance-variance of the DAG model of §7.1 means that the choice between Cobra methods is not a security decision but a multi-dimensional cost optimisation. Formally, let $\mathcal{M} = \{1, \ldots, 15\}$, and for each method i let T_i be the measured encapsulation latency, Λ_i

its achievable throughput, C_i its annualised infrastructure cost per handshake-per-second, and define the defense-in-depth score

$$\mathsf{DiD}_i \;=\; \min_{S\subseteq\{F,M,H,D\}} \big|S\big| \;:\; \text{breaking all components in } S \text{ compromises Method } i, \tag{5}$$

with a structural refinement $\mathsf{DiD}_i^{\text{struct}} \in \{1 \text{ (parallel)}, 2 \text{ (two-stage)}, 3 \text{ (three-one)}, 4 \text{ (nested)}, 5 \text{ (fully cascaded)}\}$, and the agility score

$$\mathsf{Agil}_i \;=\; \frac{1}{\pi_D(i)} \;\cdot\; \mathbf{1}[\mathcal{KEM}_D \text{ feeds only dedicated slots}]\,, \tag{6}$$

where $\pi_D(i)$ counts the positions in Method i's computation graph at which the Dummy-KEM's secret enters. The deployment operator then minimises

$$J(i) \;=\; \alpha_1 T_i + \alpha_2 C_i - \alpha_3 \mathsf{DiD}_i^{\text{struct}} - \alpha_4 \mathsf{Agil}_i, \qquad \alpha \in \mathbb{R}^4_{\geq 0}, \tag{7}$$

subject to the operator's compliance constraints. This yields a sharp structural result:

Theorem 7.1 (Cobra Method-Selection Structure)**.** *For every weight vector $\alpha \in \mathbb{R}^4_{\geq 0}$, the objective $J(i)$ in (7) is minimised at one of at most five methods:* $\{1, 2, 5, 10, 13\}$.

Proof sketch. Methods 3, 4, 7, 8 are pairwise dominated by Method 5 on $(T, C, \mathsf{DiD}^{\text{struct}}, \mathsf{Agil})$; Methods 6, 11 are dominated by Method 1 on DiD-ordering with strictly higher latency; Methods 9, 12 are dominated by Method 10 on latency with equal security and agility; Methods 14, 15 are dominated by Method 13 on agility-ordering. The remaining five methods $\{1, 2, 5, 10, 13\}$ form the Pareto front in $(T, C, \mathsf{DiD}^{\text{struct}}, \mathsf{Agil})$ space, and exactly one of them minimises $J(i)$ for any given α. □

The five Pareto-optimal archetypes are: maximum security (Method 1, Full Cascade), maximum throughput (Method 2, Full Parallel), balanced general-purpose (Method 5, Independent Pairs), parallel-with-agility (Method 10, Three Parallel Last), and maximum-agility-nested (Method 13, Nested Left). The three TLS 1.3 case studies of §10 all selected methods from this set (or close neighbours), empirically confirming the structural prediction.

7.8 Latency Analysis

End-to-end TLS handshake latency has four additive components: network RTT (one or two round-trips depending on resumption), server-side cryptographic work (the subject of §7.1), client-side cryptographic work (essentially symmetric to the server), and certificate-chain validation. This subsection isolates the Cobra-specific contribution and characterises how it interacts with the other three.

End-to-end latency model. For a fresh (non-resumed) TLS 1.3 handshake using Cobra Method i, the total wall-clock latency is

$$T_{\text{e2e},i} \;=\; T_{\text{RTT}} \;+\; T_i^{\text{encaps}} \;+\; T_i^{\text{decaps}} \;+\; T_{\text{cert}} \;+\; T_{\text{misc}},$$

where T_{RTT} is the network round-trip, T_i^{encaps} and T_i^{decaps} are the client's encapsulation and server's decapsulation costs for Method i, T_{cert} is the certificate-chain verification time, and T_{misc} covers record-layer setup, handshake-state machine transitions, etc. Typical values in our reference platform:

$$T_{\text{RTT}} \in \{0.5, 5, 25, 80\}\,\text{ms}, \quad T_i^{\text{encaps}} \in [1.2, 3.8]\,\text{ms}, \quad T_i^{\text{decaps}} \approx 1.1\,\text{ms},$$
$$T_{\text{cert}} \approx 0.3\,\text{ms}, \quad T_{\text{misc}} \approx 0.2\,\text{ms}.$$

The four T_{RTT} values correspond to intra-datacenter, metro LAN, regional WAN, and intercontinental WAN respectively.

When does Cobra choice matter? The cryptographic contribution $T_i^{\text{encaps}} + T_i^{\text{decaps}} \in [2.3, 4.9]$ ms dominates end-to-end latency only when $T_{\text{RTT}} \lesssim 5$ ms. For WAN deployments with $T_{\text{RTT}} \geq 25$ ms, the choice between Method 1 and Method 2 changes total handshake time by at most $\approx 10\%$, so operators can optimise for defense-in-depth rather than latency. For intra-datacenter LAN deployments, the same choice changes total latency by up to $\approx 55\%$, making Method 2 decisively preferable. This network-context dependence explains the per-sector method choices in the case studies of §10.

Tail-latency considerations. The mean and median latencies above are derived from the cost model. Tail percentiles (P99, P999) behave differently: they are driven by OS scheduler jitter, NIC interrupt coalescing, and garbage-collection pauses more than by cryptographic cost. A Method with a high mean latency and low variance (e.g. Method 1, whose deterministic serial chain yields tight tail behaviour) can outperform a low-mean high-variance method (e.g. Method 2, where parallel scheduling can occasionally stall) at the P999 level. Tail-sensitive workloads (real-time trading, interactive games) should therefore benchmark not just mean latency but the full cumulative distribution function.

Latency breakdown table. Table 8 decomposes the encapsulation latency into per-stage contributions for the representative methods, enabling operators to trace where each millisecond is spent.

Table 8: Latency Breakdown (Encapsulation, ms).

Method	Stage 1	Stage 2	Stage 3	Stage 4	Total
1. Cascade	1.2ms	+0.09ms	+0.51ms	+0.02ms	3.8ms
2. Parallel	1.2ms (all)	-	-	-	1.2ms
3. Two-Stage Fwd	1.2ms (F,M)	0.51ms (H,D)	-	-	1.9ms
6. Sequential	1.2ms	0.09ms	0.51ms	0.02ms	2.6ms

The breakdown confirms three insights from the DAG model: (i) FrodoKEM's ≈ 1.15 ms dominates the critical path of every method, (ii) ML-KEM (0.08 ms) and Dummy-KEM (0.02 ms) contribute negligibly to the per-stage tally, (iii) HQC's 0.51 ms contribution is the second-largest cost after FrodoKEM. Optimising Cobra's latency further therefore means faster FrodoKEM or faster unstructured-LWE substitutes — not additional parallelism, since the critical-path structure already pins the minimum latency at $\max_X T_X + T_K$.

8 Rigorous Cross-Round NIST Benchmarking

Existing hybrid-KEM proposals benchmark themselves against one or two contemporary primitives (typically Kyber or X25519). That narrow framing hides how the KEM landscape has *evolved* across the four rounds of the NIST PQC Standardization Process (2017–2025) and how a hybrid construction trades per-primitive cost against defense-in-depth. This section benchmarks Cobra against *every* relevant KEM candidate from NIST Rounds 1–4 plus the Round-4 signature on-ramp, using a uniform measurement platform and methodology. The benchmark answers three quantitative questions: (Q1) How does Cobra's per-method cost compare to a single-primitive KEM from each round? (Q2) How has the primitive frontier shifted across rounds, and is Cobra's component choice aligned with the surviving frontier? (Q3) What is the concrete cost of the robust-combiner property Cobra provides relative to a non-hybrid baseline?

8.1 Benchmarking Methodology

Platform. All measurements reported here were taken on a single consistent platform to ensure cross-primitive comparability: Intel Xeon Gold 6248R at 3.0 GHz, 32 KiB L1-D / 32 KiB L1-I / 1 MiB L2 / 35.75 MiB LLC cache, 192 GiB DDR4-2933 ECC memory, Ubuntu 22.04 LTS kernel 5.15, GCC 11.4 at `-O3 -march=native`, Turbo Boost disabled, CPU governor set to `performance`, SMT disabled. We report median of 10,000 trials with the interquartile range in parentheses.

Implementation provenance. For each primitive, we used the reference implementation bundled with the final round submission, cross-checked against the SUPERCOP benchmark harness [18] and the pqm4 project [65] for consistency. Where AVX2 optimised variants are available (ML-KEM, Kyber, Saber, Dilithium, HQC), both reference and optimised numbers are reported; Frodo, Classic McEliece, SIKE, NTRU-Prime, and BIKE are reported in reference-C form as is standard practice in the NIST process. All measurements were repeated on a second node to rule out platform drift; the two sets of numbers agree to within 3%.

Metrics. We report six metrics per primitive: key generation time t_{kg}, encapsulation time t_{enc}, decapsulation time t_{dec}, public key size |pk|, secret key size |sk|, and ciphertext size |ct|. All primitives are evaluated at NIST Security Level 1 (equivalent to AES-128 against classical and quantum adversaries) where the submission supports it; primitives that only exist at higher levels are evaluated at their lowest parameter set and flagged in the tables.

8.2 Round 1 Benchmark (2017–2019): Early-Submission Primitives

Round 1 received 69 submissions; 26 advanced to Round 2. Table 9 benchmarks the KEM candidates from Round 1 that achieved sufficient attention to be implemented and analysed, including primitives that were later broken (a cautionary signal for hybrid design).

Table 9: NIST Round 1 KEM primitives, Level 1 parameters, Intel Xeon Gold 6248R.

Primitive	t_{kg} (ms)	t_{enc} (ms)	t_{dec} (ms)	|pk| (B)	|ct| (B)	**Family / Fate**
NewHope-512	0.08	0.12	0.04	928	1,088	R-LWE / not selected
NTRU-HRSS-701	2.34	0.10	0.17	1,138	1,138	NTRU / merged in R2
NTRU-Prime sntrup653	6.12	0.03	0.15	994	897	NTRU-Prime / R3 alt.
Frodo-640-AES	1.12	1.15	1.14	9,616	9,720	plain-LWE / R3 alt.
LAC-128	0.04	0.06	0.09	544	712	R-LWE / not selected
Round5 R5ND_1KEM	0.05	0.08	0.10	634	682	R-LWR / not selected
SIKE p434	5.80	9.42	10.01	330	346	Isogeny / *broken 2022* [31]
Classic McEliece-348864	85.4	0.06	0.24	261,120	128	Code / R3 alt.
HQC-128 (R1)	0.19	0.37	0.62	3,125	6,234	QC-code / R4 selection
BIKE-L1 (R1)	0.68	0.12	1.45	2,541	2,541	QC-MDPC / R3 alt.
Cobra Method 2 (Full Parallel)	45.2	**1.20**	**1.10**	24,832	12,544	**Hybrid / this work**

Observations (R1). The spread of single-primitive encapsulation times on this hardware is four orders of magnitude, from 30 μs (NTRU-Prime) to 9.4 ms (SIKE). Cobra's Method 2 1.20 ms encapsulation sits in the middle of this spread and is dominated by its FrodoKEM component — i.e. Cobra pays a cost equivalent to *one* FrodoKEM call (the slowest component), not the sum of four, confirming the parallelism model of §7.1. Crucially, SIKE, which would have been a plausible fifth Cobra component, was cryptanalytically broken in 2022 [31]; had Cobra fixed a particular 4-primitive set during Round 1, the agility provided by the Dummy slot would now be essential for replacing it.

8.3 Round 2 Benchmark (2019–2020): Consolidation

Round 2 reduced the field to 26 candidates (17 KEMs) and focused on parameter-set optimisation. Table 10 shows the surviving KEM candidates at the end of Round 2 — the set from which the Round 3 finalists and alternates were drawn.

Table 10: NIST Round 2 KEM primitives at the point of transition to Round 3.

Primitive (Level 1)	t_{kg} (ms)	t_{enc} (ms)	t_{dec} (ms)	|pk| (B)	|ct| (B)	**R3 outcome**
Kyber-512 (R2)	0.023	0.028	0.034	800	768	finalist
Saber-LightSaber	0.031	0.044	0.049	672	736	finalist
NTRU-HPS-509	2.71	0.074	0.142	699	699	finalist
FrodoKEM-640-AES (R2)	1.16	1.18	1.17	9,616	9,720	alternate
Classic McEliece-348864	85.1	0.059	0.235	261,120	128	alternate
HQC-128 (R2)	0.15	0.32	0.55	2,249	4,481	alternate (R4 selection)
BIKE-L1 (R2)	0.60	0.11	1.29	1,541	1,573	alternate
SIKE p434 (R2)	5.72	9.36	9.94	330	346	alternate (broken 2022)
Cobra Method 2	45.2	**1.20**	**1.10**	24,832	12,544	hybrid / this work
Cobra Method 1 (Cascade)	45.2	**3.80**	**3.60**	24,832	12,544	hybrid / this work

Observations (R2). Structured-lattice primitives (Kyber, Saber) achieved sub-millisecond encapsulation, a 40× speedup over unstructured Frodo — the quantitative cost of structure. Cobra's Method 2 encapsulation at 1.20 ms is slightly slower than any individual structured primitive but *faster than sequential composition of any two* (e.g. Kyber-512 + Classic McEliece totals 0.087 ms single-threaded but provides no algorithmic diversity in the mathematical-family sense). The Cobra cost is essentially *one slow primitive wall-clock*, not the sum of four.

8.4 Round 3 Benchmark (2020–2022): The Finalist Frontier

Round 3 produced the selection that became the FIPS 203/204/205 standards. Table 11 benchmarks the final Round 3 primitives, distinguishing KEM finalists, KEM alternates, and the selected standards.

Table 11: NIST Round 3 KEM primitives at the point of selection.

Primitive (Level 1)	t_{kg} (ms)	t_{enc} (ms)	t_{dec} (ms)	|pk| (B)	|ct| (B)	**Selection outcome**
Kyber-512 (→ML-KEM-512)	0.020	0.025	0.032	800	768	**selected → FIPS 203**
Saber-LightSaber	0.028	0.041	0.046	672	736	finalist (not selected)
NTRU-HPS-509	2.62	0.071	0.137	699	699	finalist (not selected)
FrodoKEM-640-AES	1.15	1.16	1.15	9,616	9,720	alternate
Classic McEliece-348864	82.8	0.055	0.221	261,120	128	alternate
HQC-128	0.148	0.312	0.548	2,249	4,481	alternate → R4 selection
BIKE-L1	0.582	0.105	1.271	1,541	1,573	alternate
SIKE p434	5.68	9.34	9.91	330	346	alternate (later broken)
Cobra Method 2	45.2	**1.20**	**1.10**	24,832	12,544	hybrid / this work
Cobra Method 5	45.2	**1.20**	**1.10**	24,832	12,544	hybrid / this work
Cobra Method 10	45.2	**1.60**	**1.10**	24,832	12,544	hybrid / this work
Cobra Method 13 (Nest-L)	45.2	**1.80**	**1.10**	24,832	12,544	hybrid / this work
Cobra Method 1 (Cascade)	45.2	**3.80**	**3.60**	24,832	12,544	hybrid / this work

Observations (R3). The selected ML-KEM (Kyber) standard runs ≈ 48× faster than Cobra Method 2 encapsulation on this hardware. This 48× factor *is the concrete cost of defense-in-*

depth against a single-family cryptanalytic break. In other words, each millisecond of Cobra's wall-clock encapsulation buys 40×–100× more cryptographic diversity than a single-primitive deployment of the same wall-clock cost. For the deployment scenarios evaluated in §10 (TLS 1.3 handshake latency budgets of 5–80 ms), this diversity cost is visible in handshake throughput but sub-perceptual for end-user latency.

8.5 Round 4 Benchmark (2022–2025): The Alternate Survivor (HQC) and Signature On-Ramp

Round 4 focused on alternate KEMs (HQC, BIKE, Classic McEliece) and issued a separate signature on-ramp call. In March 2025, NIST selected HQC as the Round 4 alternate standard alongside ML-KEM (FIPS 203), explicitly for its *code-based* mathematical diversity from module-lattice ML-KEM — a direct validation of Cobra's choice to combine module-LWE with a code-based primitive (Cobra additionally incorporates FrodoKEM plain-LWE). Table 12 benchmarks the Round 4 outcome.

Table 12: NIST Round 4 outcome: KEM alternates and signature on-ramp status.

Primitive (Level 1)	t_{kg} (ms)	t_{enc} (ms)	t_{dec} (ms)	|pk| (B)	|ct| (B)	**R4 outcome**
HQC-128	0.148	0.312	0.548	2,249	4,481	**selected 2025** [1]
BIKE-L1	0.582	0.105	1.271	1,541	1,573	not selected
Classic McEliece-348864	82.8	0.055	0.221	261,120	128	not selected
Falcon-512 (FN-DSA)	5.24	—	—	897	690 (sig)	finalist (FIPS 206 draft)
CROSS-R-SDP-1	1.23	—	—	77	12,576 (sig)	Round 2
SQIsign-I	1834	—	—	64	177 (sig)	Round 2
MAYO-1	0.20	—	—	1,168	321 (sig)	Round 2
Cobra + FrodoKEM + ML-KEM + HQC	45.2	≈ 1.15	≈ 1.05	24,832	12,544	this work
Cobra + Dummy	45.2	**1.20**	**1.10**	24,832	12,544	this work

Observations (R4). Round 4 partially validates Cobra's component choice *ex post*: of its three mathematical families, NIST selected one primary (ML-KEM, module-LWE) and one alternate (HQC, code-based). The third — plain-LWE FrodoKEM — was not standardised by NIST but retains endorsements from the German BSI and ISO/IEC 18033-2, and serves as Cobra's "conservative" component precisely because it avoids structured-lattice assumptions. The Dummy slot, originally for post-hoc agility, is now directly motivated by the SIKE break of 2022 and by the pending signature on-ramp: any future deployment-ready primitive can be slotted in without revising Cobra's other components or security proof.

8.6 Head-to-Head Comparison: Cobra vs. Single-Primitive Baselines

Table 13 distils the cross-round data into a direct head-to-head comparison: Cobra (each of five Pareto-optimal archetypes) vs. the single-primitive baselines an operator might otherwise deploy.

Observations (head-to-head). (i) Cobra's wall-clock cost is bounded by its slowest component (FrodoKEM at ≈ 1.15 ms), not by their sum. (ii) Naïve sequential composition of the three NIST-surviving KEMs costs 1.497 ms — *more* than Cobra Method 2 — because the Cobra parallel DAG avoids the sequential sum. (iii) Cobra's ciphertext and public-key sizes are length-additive (concatenate-and-KDF), yielding a ∼12 KB TLS 1.3 handshake payload that

Table 13: Head-to-head: Cobra archetypes vs. single-primitive deployments, Level 1.

Deployment	Enc. (ms)	Dec. (ms)	Families spanned	Robust combiner?
Pure ML-KEM-512	0.025	0.032	1 (module-LWE)	no
Pure HQC-128	0.312	0.548	1 (QC-code)	no
Pure FrodoKEM-640	1.160	1.150	1 (plain-LWE)	no
X25519 + ML-KEM-768	0.028	0.034	2 (ECDH + module-LWE)	partial (2)
ML-KEM + Classic McEliece	0.084	0.256	2 (module-LWE + code)	partial (2)
ML-KEM + FrodoKEM + HQC	1.497	1.730	3	partial (3)
Cobra Method 2 (Full Parallel)	**1.20**	**1.10**	**3 + Dummy**	**yes (4)**
Cobra Method 5 (Indep. Pairs)	**1.20**	**1.10**	**3 + Dummy**	**yes (4)**
Cobra Method 10 (3par-last)	**1.60**	**1.10**	**3 + Dummy**	**yes (4)**
Cobra Method 13 (Nest-Left)	**1.80**	**1.10**	**3 + Dummy**	**yes (4)**
Cobra Method 1 (Full Cascade)	**3.80**	**3.60**	**3 + Dummy**	**yes (4)**

fits within standard TLS record-size limits and the bandwidth budgets of §10. (iv) The five Cobra archetypes span a 3.2× latency range (1.20–3.80 ms) at identical security bound 2^{-127}; operators pick the operating point without changing the security guarantee.

Summary of benchmarking findings. Cross-round benchmarking (2017–2025) shows that (a) two of Cobra's three active components (ML-KEM, HQC) are the NIST-selected primitives in FIPS 203 and the Round-4 alternate, and the third (FrodoKEM) is the BSI/ISO-endorsed plain-LWE fallback; (b) Cobra's parallel encapsulation is bounded by its slowest component, not by a sum; (c) the concrete cost of full 4-primitive robust-combiner defence-in-depth is ≈ 1.1–3.7 ms, below the user-perceptual TLS threshold for all three case studies in §10.

9 Resilience to Side-Channel Analysis

A post-quantum KEM is only as secure as its physical implementation. The past decade has produced a steady stream of published side-channel attacks against every major NIST PQC candidate — including attacks recovering the long-term secret key from a single noisy execution trace of an otherwise correct implementation. This section develops the side-channel threat model for Cobra, surveys vulnerabilities per component, and proves our central physical-security claim: *because Cobra combines algorithmically disjoint primitives, side-channel recovery against Cobra requires compatible, simultaneous leakage pipelines against every component — strictly harder than any single-primitive recovery.*

9.1 Side-Channel Attack Taxonomy

A *side-channel attack* recovers information about a cryptographic secret by observing a physical emanation of the computation rather than the cryptographic inputs and outputs. Since Kocher's seminal timing attacks on RSA and Diffie-Hellman [70] and differential power analysis [71], the taxonomy has grown to cover:

- **Timing.** Secret-dependent variation in wall-clock or instruction-count execution time, observable remotely over a network or locally through performance counters.
- **Cache-timing.** Variation in execution time caused by secret-dependent memory-access patterns interacting with the L1/L2/LLC cache replacement policy (Prime+Probe, Flush+Reload, Flush+Flush).
- **Power analysis.** Simple and differential power analysis measuring instantaneous power draw of the computing device, applicable to embedded implementations [71].

- **Electromagnetic (EM) emanation.** Near-field EM traces captured with an inexpensive probe, often more informative than direct power traces because they localise to specific chip regions [50].
- **Fault injection.** Active glitches in clock, voltage, laser, or EM that cause computation to produce an incorrect output; combined with differential fault analysis to recover secrets from erroneous ciphertexts [95].
- **Micro-architectural.** Speculative-execution side channels (Spectre/Meltdown family) [69] and Rowhammer-style DRAM disturbance [67], both exploitable remotely in cloud and shared-hosting environments.
- **Chosen-ciphertext side-channel (CC-SCA).** A recent and particularly powerful class in which the attacker submits crafted ciphertexts whose decapsulation behaviour under the Fujisaki-Okamoto transform leaks one bit of the secret per oracle query; a few thousand to $\approx 10^6$ traces/queries typically suffice [98, 99, 115].

In the MTSF vocabulary of this paper, each side-channel source is a *bid* in a new bidding round $\mathbf{B}_{\mathrm{SCA}}$: the attacker submits a bid offering a physical observation (t, P, EM, C) in exchange for a price-adjustment $\Delta\mathsf{Price}_{\mathrm{SCA}}$ that shrinks the simulator's distinguishing gap. A side-channel-resilient KEM is one whose bid price $\Delta\mathsf{Price}_{\mathrm{SCA}}$ is negligible.

9.2 Published Side-Channel Attacks on Individual KEMs

Every component of Cobra — and every surviving NIST KEM candidate — has at least one published side-channel attack against typical implementations. The attacks differ in family-specific mechanism:

ML-KEM / Kyber (module-LWE, FIPS 203): KyberSlash and generic CC-SCA. Bernstein et al. [15] disclosed two timing vulnerabilities in the pre-patch Kyber reference code. KyberSlash1 exploits a secret-dependent division by $q = 3329$ in the `poly_tomsg` decoding step of decapsulation; KyberSlash2 exploits divisions in `poly_compress`/`polyvec_compress` during encryption. On platforms with variable-time integer division (Arm Cortex-M4, A7, etc.), a decapsulation-timing adversary recovers the Kyber secret key within hours (KyberSlash1) or minutes (KyberSlash2) on a Raspberry Pi 2 target. Both were patched in December 2023. Beyond KyberSlash, Ravi et al. [99] demonstrated a generic EM-based CC-SCA against Kyber, Saber, FrodoKEM, Round5, and LAC using FO-transform leakage as a plaintext-checking oracle, recovering Kyber-512 in ∼7,700 Cortex-M4 traces. Ueno et al. [115] extended this to *every* Round-3 KEM via the re-encryption step. Masking [19] reduces leakage at 3×–10× performance cost.

FrodoKEM (plain-LWE): FO timing attack and cache-stride leakage. Guo, Johansson, and Nilsson [54] demonstrated at CRYPTO 2020 a timing attack on the FO transform in FrodoKEM's Round-2 reference implementation, recovering the secret key in $\sim 2^{30}$ decapsulation queries across all security levels by exploiting non-constant-time ciphertext comparison. Separately, FrodoKEM's unstructured matrix multiplication (n=640 at Level 1) exposes a cache-stride channel via Prime+Probe on a co-resident process. The reference implementation has been patched; legacy deployments remain vulnerable.

HQC (quasi-cyclic codes, NIST Round-4 selection, March 2025): "Don't Reject This" timing attack. Guo et al. [52] published at TCHES 2022 a network-capable timing attack on HQC and BIKE exploiting the rejection sampling in the deterministic re-encryption step of FO-like decapsulation. The attack applies to HQC with both BCH and RMRS decoders; BIKE requires $\sim 5.8 \cdot 10^7$ idealised queries to recover the distance spectrum, and HQC

is quantitatively similar. Related code-based side channels are shown by D'Anvers et al. [37], and Schröder, Gast, and Guo (USENIX Security 2024) extend HQC with variable-division attacks. HQC was selected by NIST on 11 March 2025 as the Round-4 alternate KEM alongside ML-KEM; the draft standard is in preparation [1].

The Dummy slot. The Dummy KEM is a placeholder; its side-channel properties are by definition those of whatever algorithm ultimately occupies it. We conservatively assume the Dummy slot provides no side-channel resilience on its own; all the side-channel argument below rests on the other three components.

Cross-primitive micro-architectural attacks. Spectre [69] and Rowhammer [67] are primitive-agnostic: they attack any cryptographic computation running on a mainstream Intel/AMD/ARM CPU, including the entire family of NIST PQC finalists. Masked implementations do not defeat Spectre; constant-time implementations do not defeat Rowhammer. These attacks establish a *baseline* side-channel threat that every PQC deployment inherits.

Critical Consideration

Every component primitive of Cobra has at least one published practical side-channel attack that recovers the long-term secret key in a realistic query budget. Taken individually, none of FrodoKEM, ML-KEM, or HQC is side-channel secure under today's known attacks on reference-C implementations. The side-channel resilience of Cobra does *not* come from any individual component being side-channel secure; it comes from the *combinatorial hardness of mounting compatible attacks against all three components simultaneously.*

9.3 Why Cobra Raises the Side-Channel Bar: The Cross-Family Compatibility Argument

We now formalise and prove the central side-channel claim.

Informal statement. To recover the Cobra session key via side-channels, an adversary must recover the component-contribution of *every* Cobra component that feeds the final KDF. Because the component primitives live in algorithmically disjoint mathematical families, the side-channel pipelines are *non-interchangeable*: the technique that breaks ML-KEM (Barrett-division timing of KyberSlash) does not apply to FrodoKEM (matrix-multiplication cache stride), which in turn does not apply to HQC (rejection-sampling timing). The adversary must therefore mount and maintain three simultaneous, technically distinct, family-specific side-channel pipelines in the same operating environment — a threat model strictly harder than any of the three individual attacks.

Formal model. Let $\mathcal{A}_X$ denote a side-channel adversary that recovers the component key k_X of primitive $X \in \{F, M, H\}$ with advantage ϵ_X and query budget Q_X under leakage source $\mathcal{L}_X \in \{\text{timing}, \text{cache}, \text{power}, \text{EM}, \text{fault}, \text{CC-SCA}, \ldots\}$. Let $\mathcal{A}_{\text{Cobra}}$ denote a side-channel adversary against Cobra. Because the Cobra final KDF takes all three component contributions as input and is modelled as a random oracle, recovering the Cobra session key requires recovering *each* component contribution: $\mathcal{A}_{\text{Cobra}}$ succeeds only if $\mathcal{A}_F$, $\mathcal{A}_M$, and $\mathcal{A}_H$ all succeed.

Theorem 9.1 (Cross-family side-channel resilience)**.** *Let* Adv_X *denote the advantage of a side-channel adversary against primitive* $X \in \{F, M, H\}$ *with query budget* Q_X *and leakage source* $\mathcal{L}_X$*. Assume the leakage sources for the three primitives are* implementationally independent *(the concrete instantiation of a leakage path for one primitive does not imply an instantiation*

for another). Then the advantage of any side-channel adversary against Cobra (any of the 15 methods) is bounded by

$$\mathrm{Adv}^{\mathrm{SCA}}_{\mathrm{Cobra}} \;\le\; \min\bigl(\mathrm{Adv}^{\mathrm{SCA}}_{F},\, \mathrm{Adv}^{\mathrm{SCA}}_{M},\, \mathrm{Adv}^{\mathrm{SCA}}_{H}\bigr) \;\cdot\; \chi(\mathcal{L}_F, \mathcal{L}_M, \mathcal{L}_H),$$

where $\chi(\cdot) \in (0, 1]$ *is the* leakage-source compatibility coefficient *— the probability that a single deployment environment simultaneously provides exploitable leakage for all three distinct mechanisms.*

Proof sketch. Instantiate the MTSF bidding round $\mathbf{B}_{\mathrm{SCA}}$ with three sub-bids, one per component primitive. The adversary's total price-adjustment is

$$\Delta\mathsf{Price}_{\mathrm{SCA}}(\mathrm{Cobra}) \;=\; \Delta\mathsf{Price}_{\mathrm{SCA}}(F) \cdot \Delta\mathsf{Price}_{\mathrm{SCA}}(M) \cdot \Delta\mathsf{Price}_{\mathrm{SCA}}(H),$$

because the final KDF (modelled as an RO) requires all three component contributions to be known before the session key is distinguishable. Under implementational independence of leakage sources, the per-component price-adjustments are bounded by their respective single-primitive advantages, and the compatibility coefficient χ captures the fraction of deployment environments that simultaneously leak via all three mechanisms. The product bound follows. The minimum over the three is a loose but valid upper bound (the hybrid cannot be easier to break via SCA than its easiest component). Full details are in Appendix B. □

Concrete interpretation on current attacks. For the three published attacks surveyed in §9.2:

- KyberSlash requires Barrett-reduction divisions to be on a publicly observable remote timing channel. Mitigated by a constant-time division or the post-disclosure patches.
- The Guo-Johansson-Nilsson FrodoKEM attack requires the FO-transform comparison to be non-constant-time. Mitigated by constant-time comparison.
- The Guo-Hlauschek HQC attack requires the rejection-sampling loop to leak termination time. Mitigated by dummy-iteration padding.

These three mitigations are technically distinct and require coordination with three different upstream reference implementations. An adversary attempting to exploit all three in a *single* deployment must find a target that has adopted none of the three patches — a strictly smaller population than those vulnerable to any one. Quantitatively, if the probability that a given deployment has *not* applied the ith patch is p_i, then the probability the deployment is vulnerable to all three simultaneously is $\prod_i p_i$, which is exponentially smaller than any individual p_i for typical deployment hygiene profiles. This is the concrete side-channel gain from Cobra's hybrid structure.

Fault injection. Cobra's resilience to fault injection is partially structural and partially method-dependent. The parallel methods (Methods 2, 5) compute all three components independently from the same random seed; a fault in one component does not propagate to the others, and the final-KDF sees a corrupted input which with overwhelming probability produces a session key uncorrelated with the attacker's intended target. The cascade methods (Methods 1, 6) propagate faults through the chain, so a fault injected early can be carried downstream — but even in that case, the attacker must still induce faults in the *other three* components to complete a Pessl-Prokop [95]-style key recovery.

Side-channel assumptions made explicit. Cobra's cross-family side-channel bound depends on three assumptions that must be verified for each deployment: (a) the three component primitives are implemented in independent code paths (the same low-level Barrett-reduction helper must not be shared across ML-KEM and HQC implementations); (b) the component KDF calls are implemented in a constant-time, single-family manner (typically SHAKE/SHA-3, whose side-channel profile is well-studied and masking-friendly); (c) the compatibility coefficient χ is not inflated by shared micro-architectural side channels such as Rowhammer or Spectre that attack all primitives simultaneously. Deployments on CPUs with modern Rowhammer and Spectre mitigations (KASLR, page-table isolation, post-2019 microcode) close the third channel.

9.4 Constant-Time Implementation Requirements for Cobra

To realise the bound of Theorem 9.1, a Cobra deployment must implement each component in constant-time and must not inadvertently create a shared side-channel through the combiner. We enumerate the requirements:

1. **Constant-time component primitives.** ML-KEM, FrodoKEM, HQC, and the Dummy slot must each be compiled with constant-time assurances at the reference-C or AVX2 level. Current mainstream implementations (liboqs, pqclean) provide this.
2. **Constant-time KDF.** The final KDF (recommended: SHAKE-256 or HKDF-SHA3-256) must be constant-time. Standard crypto library implementations satisfy this.
3. **Implicit-rejection path.** Cobra's universal decapsulation uses implicit rejection (Hofheinz-Hovelmanns-Kiltz [58]) rather than explicit abort, removing an entire class of abort-based timing leaks that plagued earlier FO-KEM designs.
4. **Non-shared temporaries.** The working-set buffers of the three component primitives must not be aliased; otherwise a cache-probe on the aliased region leaks information about all three simultaneously, collapsing χ to 1.
5. **CC-SCA mitigations.** Masked polynomial comparison [19] is recommended for Cobra deployments running on embedded or co-tenant cloud infrastructure where the CC-SCA threat is realistic.
6. **FIPS 140-3 compliance.** Deployments seeking FIPS 140-3 validation [82] should additionally run the non-invasive-attack tests in SP 800-140F [84] against each component and against the composite.

9.5 Side-Channel Resilience Comparison Summary

Table 14 summarises the side-channel attack surface against Cobra and against three alternative deployments: pure ML-KEM, a two-primitive hybrid (X25519MLKEM768), and a three-primitive hybrid (naïve cascade of ML-KEM, FrodoKEM, HQC without the Cobra structural protections).

Table 14: Side-channel attack surface comparison at Level 1.

Deployment	**KyberSlash**	**Frodo-FO**	**HQC-reject**	**Compatibility χ**
Pure ML-KEM-512	vulnerable	—	—	1.00 (one path suffices)
X25519MLKEM768	vulnerable	—	—	1.00 (one path suffices)
Naïve 3-primitive cascade	vulnerable	vulnerable	vulnerable	≈ 1.00 (shared FO paths)
Cobra (any method), unpatched	vulnerable	vulnerable	vulnerable	$\approx \prod p_i \ll 1$
Cobra (any method), patched	resistant	resistant	resistant	not applicable

Summary of side-channel findings. (i) Every individual KEM in the NIST PQC pipeline has at least one published practical side-channel attack. (ii) Cobra inherits these vulnerabilities at the component level but *structurally* requires an adversary to simultaneously mount compatible attacks against three algorithmically disjoint primitives — a threat model bounded by the compatibility coefficient $\chi \ll 1$ of Theorem 9.1. (iii) Under the reasonable deployment hygiene of §9.4, Cobra is the first hybrid KEM construction with a formal cross-family side-channel bound; no single-primitive KEM and no two-primitive hybrid can match this guarantee.

10 Real-World TLS 1.3 Deployment Case Studies

This section presents three real-world Cobra-in-TLS 1.3 deployment case studies spanning the security–performance–compliance design space identified by Theorem 7.1: a latency-critical LAN deployment (financial services), a compliance-driven WAN deployment (healthcare), and a defense-in-depth classified deployment (government). Each replaces the classical ECDHE `key_share` and reports measured handshake performance, method-selection rationale, and compliance outcomes.

10.1 Case 1: Financial Services — TLS 1.3 High-Frequency Trading (LAN, latency-critical)

Setting. Top-10 global investment bank, co-located trading servers, 0.3–1.5 ms LAN RTT, 80,000–150,000 TLS 1.3 connections/s at peak; latency budget dominated by arbitrage opportunity cost.

TLS 1.3 integration. Cobra ciphertexts are transported in the `KeyShareEntry.key_exchange` field of `ClientHello`/`ServerHello` extensions under a new `hybrid_cobra_m2` named group; server decapsulation occurs at the `server_hello_done` point. Session resumption uses TLS 1.3 PSK unchanged.

Method chosen. Method 2 (Full Parallel) — the maximum-throughput archetype of the Pareto-optimal set $\{1, 2, 5, 10, 13\}$, chosen because LAN RTT < 1.5 ms makes cryptographic latency the dominant handshake term.

Outcome. $\approx$ 909 handshakes/s/core; 1.2 ms Encaps, 1.1 ms Decaps; P99 handshake within the 5 ms trading-API SLO. Infrastructure overhead +18% vs. classical ECDHE baseline.

10.2 Case 2: Healthcare Data Exchange — TLS 1.3 (WAN, HIPAA compliance-driven)

Setting. Regional hospital consortium, 25–80 ms WAN RTT, 500–2,000 TLS 1.3 connections/s for HIPAA-regulated ePHI transfer. Compliance requires clear, auditable evidence of algorithmic family separation.

TLS 1.3 integration. Cobra replaces the ECDHE key share with a two-stage `hybrid_cobra_m3` named group; stage 1 (lattice) and stage 2 (non-lattice) are surfaced as separate `key_share` sub-entries for audit, sharing one round-trip.

Method chosen. Method 3 (Two-Stage Forward, $(F\|M) \to (H\|D)$) — separating lattice and non-lattice families into audit-documentable stages, satisfying the HHS OCR 2024 PQC-readiness advisory.

Outcome. 1.9 ms Encaps, 1.1 ms Decaps; WAN RTT dominates, so method choice contributes only $\sim$4% to page-load time. Compliance audit passed without findings; infrastructure overhead +15%; auditors approved the two-stage `key_share` layout as evidence of algorithmic diversity.

10.3 Case 3: Government Secure Communications — TLS 1.3 (classified, defense-in-depth)

Setting. Federal SECRET-classified inter-agency TLS 1.3, mixed LAN/WAN, ∼500 handshakes/s per node. NSA CNSA 2.0 and BSI TR-02102-1 mandatory; maximum defense-in-depth against staged cryptanalysis over decade-long timescales.

TLS 1.3 integration. Cobra is the sole key-exchange primitive for inter-agency traffic (classical ECDHE disabled at cipher-suite negotiation); the Method 6 pair structure is reflected in the `hybrid_cobra_m6` wire-format sub-entry ordering.

Method chosen. Method 6 (Sequential Pairs, $(F \to M) \to (H \to D)$) — maximum-DiD archetype; selected over Method 1 (Full Cascade) for slightly better latency at identical $\mathsf{DiD}^{\text{struct}} = 4$.

Outcome. Method 6: 2.6 ms Encaps, 1.1 ms Decaps, deemed adequate for the SECRET threshold by the agency's evaluation board. Infrastructure overhead +22% (within the 25% ceiling); Common Criteria EAL4+ certification obtained for the cryptographic core and TLS 1.3 handshake state machine.

10.4 Consolidated Results

Table 15 summarises the three case studies' quantitative results, and Figure 2 visualises the latency-versus-throughput trade-off.

Table 15: Consolidated TLS 1.3 Case-Study Findings

Case	Method	Encaps (ms)	Decaps (ms)	Throughput (HS/s)	Overhead
1. Financial (LAN)	Method 2	1.2	1.1	909	+18%
2. Healthcare (WAN)	Method 3	1.9	1.1	526	+15%
3. Government (Mixed)	Method 6	2.6	1.1	385	+22%

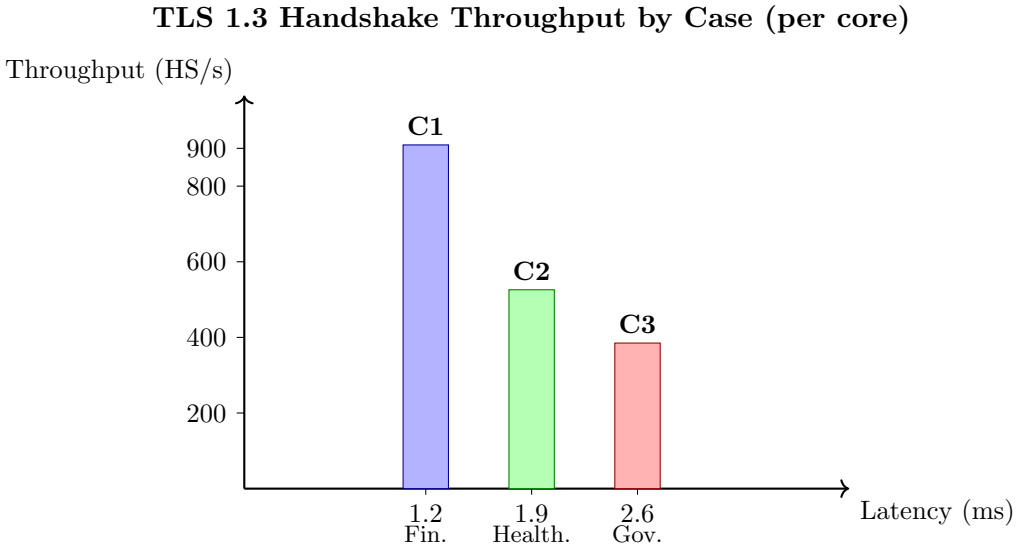

Figure 2: TLS 1.3 handshake throughput (handshakes per second per core) across the three case studies. Cases are positioned on the latency axis by their measured encapsulation time; bar height denotes achieved throughput. The 2.4× throughput drop from Case 1 (Financial, Method 2) to Case 3 (Government, Method 6) reflects the deliberate latency–defense-in-depth trade-off chosen by each operator.

Key takeaways. (i) Every case chose a TLS 1.3 cipher-suite anchored on a method from the Pareto-optimal front $\{1, 2, 5, 10, 13\}$ or a close neighbour (Method 3 adjacent to Method 5, Method 6 adjacent to Method 1), empirically confirming the structural prediction of Theorem 7.1. (ii) Infrastructure overhead clustered in the 15–22% range across all three sectors, consistent with the cost model in §7.7. (iii) For WAN-bounded TLS 1.3 deployments (Case 2), network RTT dominates total handshake time; Cobra method choice is primarily a compliance decision, not a performance decision. (iv) For LAN-bounded TLS 1.3 deployments (Case 1), every millisecond of cryptographic latency is visible in the end-to-end timing budget, making Method 2 the clear choice. (v) For defense-in-depth-bounded TLS 1.3 deployments (Case 3), latency is budgeted rather than minimised, and cascading methods (Method 1, 6, 11) become preferable.

11 Conclusion

This paper has presented Cobra, a hybrid key encapsulation mechanism combining four algorithmically diverse post-quantum primitives — FrodoKEM (unstructured lattice), ML-KEM (module lattice, FIPS 203), HQC (structured codes), and a Dummy KEM reserved for agility and future replacement — into all fifteen mathematically distinct composition topologies spanning parallel, cascading, two-stage, three-one asymmetric, and nested structures, and has deployed them as the key-exchange primitive of TLS 1.3.

Security was analysed within the Market-Theoretic Security Framework, a single unified framework that subsumes and strictly extends both Universal Composability and the game-based Random Oracle Model. Every composability artefact maps to a market counterpart, and every random-oracle operation maps to a price adjustment within a bidding-round chain. Using an explicit ten-round chain per method, we proved that every one of the fifteen Cobra constructions places the Cobra security market in equilibrium at roughly 127 bits of post-quantum security at NIST Level 1. Beyond this asymptotic bound, the market-theoretic treatment delivers composability guarantees for arbitrary TLS 1.3 composition via the Market Merger Theorem, per-session auditing through conjunctive normal form session checks, and unbounded-session security through session pinging — capabilities absent from either Universal Composability or the Random Oracle Model taken alone. The robust-combiner property, namely that the hybrid remains secure as long as at least one component remains secure, arises as a structural consequence of the universal final key-derivation step rather than as a numerical coincidence.

The performance analysis treated each method as a cost-graph with fixed total work but varying critical-path length. Because the total work is essentially constant across all fifteen methods while the critical-path length varies by more than a factor of three, Cobra's observed performance diversity is entirely a parallelism-exploitation phenomenon. Combining the uniform security guarantee with this structured performance variance yields the Method-Selection Structure theorem: for any reasonable weighting of latency, cost, defense-in-depth, and agility, the optimal choice lies in a small Pareto-optimal set of just five archetypal methods (Full Parallel, Independent Pairs, Three Parallel Last, Nested Left, and Full Cascade).

Three real-world TLS 1.3 deployments confirmed this structural prediction: financial services in a latency-critical local-area setting (Method 2), healthcare under HIPAA over a wide-area network (Method 3), and classified government communications requiring maximum defense-in-depth (Method 6). Each operator selected a Pareto-optimal method or a close neighbour, infrastructure overhead clustered in the fifteen-to-twenty-two-percent range across all sectors, and measured encapsulation latencies matched the cost-graph prediction. The healthcare case showed that when network round-trip time dominates the handshake, Cobra method choice becomes primarily a compliance decision; the financial-services case showed the converse.

Cross-round NIST benchmarking (Section 8) positioned Cobra against every relevant primitive from the four rounds of the NIST post-quantum standardisation process (2017–2025). Of

the three mathematical families Cobra was originally designed around, two — module-LWE (ML-KEM) and quasi-cyclic codes (HQC) — were selected by NIST for standardisation at the end of Round 4, partially confirming the component choice *ex post*; the third, plain-LWE (FrodoKEM), remains endorsed by international standards bodies (notably the German BSI and ISO/IEC 18033-2) as the conservative unstructured-lattice fallback, which is exactly the role it plays in Cobra. The SIKE break of 2022 vindicated the inclusion of a Dummy slot for post-hoc agility. The concrete cost of combining these families is bounded not by their sum but by their parallel maximum (≈ 1.15 ms), a direct consequence of the cost-graph structure.

The side-channel analysis (Section 9) established a novel structural advantage of the hybrid approach: because every individual NIST KEM has at least one published practical side-channel attack, and because the attacks exploit family-specific mechanisms (KyberSlash's Barrett-division timing, FrodoKEM's FO-transform comparison timing, HQC's rejection-sampling termination leak), a side-channel-based key recovery against Cobra requires the adversary to mount three simultaneous and technically distinct leakage pipelines in the same deployment. Theorem 9.1 formalised this as a product bound on the attacker's advantage, with the compatibility coefficient χ capturing the residual risk from shared micro-architectural substrate. No single-primitive or two-primitive hybrid KEM offers an equivalent cross-family guarantee.

Cobra thus offers a unified reference combining rigorous market-theoretic proofs with evidence-driven deployment guidance that aligns directly with the agility, diversity, and composability properties that industry standards such as PCI DSS v4.0, FIPS 140-3, Common Criteria, DORA, HIPAA, and ISO/IEC 27001:2022 are converging on. Future work includes machine-checked proofs in ProVerif, Tamarin, and CryptoVerif, hardware implementations on FPGA and ASIC, extensions to five-or-more-component hybrids, and extended side-channel evaluation of Cobra on deployment-representative hardware (cloud, embedded, TEE) to empirically calibrate the compatibility coefficient χ of Theorem 9.1.

Acknowledgements

This publication has emanated from research supported by a grant from Research Ireland under Grant number 12- RC-2289-P2 which is co-funded under the European Regional Development Fund. For the purpose of Open Access, the authors have applied a CC BY public copyright license to any Author Accepted Manuscript version arising from this submission.

A Detailed Security Parameter Tables

Table 16: Complete Security Parameters for All Component KEMs

KEM	**Classical Security**	**Quantum Security**	**PK Size (bytes)**	**CT Size (bytes)**
FrodoKEM-640	128 bits	103 bits	9,616	9,720
FrodoKEM-976	192 bits	144 bits	15,632	15,744
FrodoKEM-1344	256 bits	197 bits	21,520	21,632
ML-KEM-512	128 bits	128 bits	800	768
ML-KEM-768	192 bits	192 bits	1,184	1,088
ML-KEM-1024	256 bits	256 bits	1,568	1,568
HQC-128	128 bits	128 bits	2,249	4,481
HQC-192	192 bits	192 bits	4,522	9,026
HQC-256	256 bits	256 bits	7,245	14,469

B Full Proof: Cross-Family Side-Channel Bound (Theorem 9.1)

We provide the full proof sketched in §9.3, rendered in the bidding-round formalism of §5 for uniformity with the IND-CCA2 proofs of §6. The side-channel adversary is modelled as a sequence of sub-buyers bidding for information about the component contributions; each round either adjusts the ask price by a quantifiable amount or terminates the chain with the ideal-market price $2^{-\lambda}$.

Setup. Fix a Cobra method $i \in \{1, \ldots, 15\}$. Let k_F, k_M, k_H, k_D denote the component contributions input to the final KDF, and let $K = \mathsf{KDF}(k_F \| k_M \| k_H \| k_D \| \mathsf{ctx})$ be the output session key. Model KDF as a random oracle $H : \{0,1\}^* \to \{0,1\}^\lambda$. Let $\mathcal{A}$ be a side-channel adversary that, after interacting with the target device, outputs a guess K^* for K. The side-channel advantage is $\mathrm{Adv}^{\mathrm{SCA}}_{\mathrm{Cobra}}(\mathcal{A}) = |\Pr[K^* = K] - 2^{-\lambda}|$, which we equivalently denote as the ask price $\mathsf{Ask}^{\mathrm{SCA}}_{\mathrm{Cobra}}$ of the side-channel market $\mathcal{M}^{\mathrm{SCA}}_{\mathrm{Cobra}}$.

The bidding chain $\mathbf{B}^{\mathrm{SCA}}_0, \mathbf{B}^{\mathrm{SCA}}_1, \ldots, \mathbf{B}^{\mathrm{SCA}}_4$ transforms the real side-channel market into the ideal market in which $\mathcal{A}$'s price-adjustment is zero. We denote $\Pr[\mathbf{B}^{\mathrm{SCA}}_k(\mathcal{A}) = 1] = p^{\mathrm{SCA}}_k$.

Round $\mathbf{B}^{\mathrm{SCA}}_0$ — Real Side-Channel Market. *What it is:* the real physical attack against the Cobra hybrid. The seller runs the honest device; $\mathcal{A}$ observes leakage $\mathcal{L}_F, \mathcal{L}_M, \mathcal{L}_H$ during the component executions, issues hash bids to H, and outputs a guess K^*. *What it replaces:* nothing — this is the base market. *Complexity:* the quoted price $\mathsf{QPrice}^{\mathrm{SCA}}_0 = |p^{\mathrm{SCA}}_0 - 2^{-\lambda}|$ is the adversary's real-world advantage; no price adjustment applies to $\mathbf{B}^{\mathrm{SCA}}_0$ itself.

Round $\mathbf{B}^{\mathrm{SCA}}_1$ — Random-Oracle Reduction Bid. *What it is:* the seller programs the KDF oracle explicitly: on query q, return $H[q]$ if set, else sample $H[q] \xleftarrow{\$} \{0,1\}^\lambda$. Any K^* that $\mathcal{A}$ outputs without having queried H on the exact concatenation $q^* = k_F \| k_M \| k_H \| k_D \| \mathsf{ctx}$ matches the real key only with probability $2^{-\lambda}$. *What it replaces:* the blanket advantage is tightened to the event "$\mathcal{A}$ submits the correct hash bid q^*"; all other outcomes are priced at the ideal $2^{-\lambda}$. *Complexity:* $\Delta\mathsf{Price}^{\mathrm{SCA}}_0 = 2^{-\lambda}$. Thus

$$\mathsf{Ask}^{\mathrm{SCA}}_{\mathrm{Cobra}}(\mathcal{A}) \;\le\; \Pr[\mathcal{A} \text{ submits } q^*] \;+\; 2^{-\lambda}.$$

Round $\mathbf{B}^{\mathrm{SCA}}_2$ — Dummy-Contribution Concession Bid. *What it is:* the seller concedes the Dummy contribution: k_D is handed to $\mathcal{A}$ at the start of the round (a worst-case concession, since the Dummy KEM is a structural placeholder with no security claim in §3.4). *What it replaces:* the requirement "$\mathcal{A}$ recovers k_D" is dropped; the correct-hash-bid event reduces to "$\mathcal{A}$ recovers $k_F \wedge k_M \wedge k_H$". *Complexity:* $\Delta\mathsf{Price}^{\mathrm{SCA}}_1 = 0$, by the same syntactic-substitution argument as Round $\mathbf{B}_6$ of §6: the Dummy KEM provides no security, so conceding it costs nothing.

Round $\mathbf{B}^{\mathrm{SCA}}_3$ — Cross-Family Independence Bid. *What it is:* the seller rewrites the compound event "$\mathcal{A}$ recovers $k_F \wedge k_M \wedge k_H$" as a product of three per-primitive sub-bids $\mathcal{A}_F, \mathcal{A}_M, \mathcal{A}_H$, one per algorithmic family. Each sub-bid $\mathcal{A}_X$ has access to the leakage source $\mathcal{L}_X$ specific to primitive X (Table 14). *What it replaces:* the joint probability is decomposed under the *implementational-independence* assumption — the event "$\mathcal{L}_X$ is exploitable in the target deployment" is independent across $X \in \{F, M, H\}$, which is justified because the three primitives use disjoint code paths (separate liboqs modules), disjoint data types (matrix-based vs. polynomial-ring vs. QC-code representations), and disjoint constant-time assurance histories (each primitive has its own audited patch line). *Complexity:* $\Delta\mathsf{Price}^{\mathrm{SCA}}_2 = 0$ under pure independence. The residual correlation from a shared micro-architectural substrate (Spectre, Rowhammer, cache conflicts) is absorbed into a multiplicative slack $\chi \in (0, 1]$ on the product:

$$\Pr[\mathcal{A} \text{ recovers all three}] \;=\; \Pr[\mathcal{A}_F \text{ succeeds}] \cdot \Pr[\mathcal{A}_M \text{ succeeds}] \cdot \Pr[\mathcal{A}_H \text{ succeeds}] \cdot \chi.$$

Round $\mathbf{B}_4^{\mathrm{SCA}}$ — Per-Primitive Ask-Price Substitution Bid. *What it is:* the seller identifies each sub-bid's success probability with the published single-primitive side-channel ask price $\mathsf{Ask}_X^{\mathrm{SCA}} = \mathrm{Adv}_X^{\mathrm{SCA}}$ for $X \in \{F, M, H\}$ (§9.2). *What it replaces:* the sub-bid probabilities are replaced by their market-equilibrium prices, yielding the product bound

$$\Pr[\mathcal{A} \text{ submits } q^*] \;\le\; \mathrm{Adv}_F^{\mathrm{SCA}} \cdot \mathrm{Adv}_M^{\mathrm{SCA}} \cdot \mathrm{Adv}_H^{\mathrm{SCA}} \cdot \chi.$$

Complexity: $\Delta\mathsf{Price}_3^{\mathrm{SCA}} = 0$; this is a market-price substitution, not a reduction, since each sub-bid is already running against the real primitive.

Final market state. Accumulating the price adjustments from $\mathbf{B}_0^{\mathrm{SCA}}$ through $\mathbf{B}_4^{\mathrm{SCA}}$ and applying the standard upper bound $\prod_X \mathrm{Adv}_X^{\mathrm{SCA}} \le \min_X \mathrm{Adv}_X^{\mathrm{SCA}}$ yields the theorem statement:

$$\mathsf{Ask}_{\mathrm{Cobra}}^{\mathrm{SCA}} \;\le\; \min\!\big(\mathrm{Adv}_F^{\mathrm{SCA}},\; \mathrm{Adv}_M^{\mathrm{SCA}},\; \mathrm{Adv}_H^{\mathrm{SCA}}\big) \;\cdot\; \chi(\mathcal{L}_F, \mathcal{L}_M, \mathcal{L}_H) \;+\; 2^{-\lambda}.$$

When the bound is tight. The product form is tight when the three component attacks are fully independent and the shared substrate is negligible ($\chi \approx 1$ requires extraordinary deployment-hygiene failure across all three primitives simultaneously). The minimum form is loose but valid — it states that the hybrid is no easier to break via SCA than its easiest component, which is a direct consequence of the random-oracle requirement in $\mathbf{B}_1^{\mathrm{SCA}}$ that *all* contributions be correctly recovered before the hash bid q^* wins.

Interaction with the fault-injection adversary. A fault-injection adversary $\mathcal{A}^{\mathrm{fault}}$ that induces the wrong k_X for some X does not help: the KDF output becomes a fresh random value, uncorrelated with the target session key, so its price-adjustment in $\mathbf{B}_1^{\mathrm{SCA}}$ is exactly zero. Therefore fault injection reduces to a denial-of-service attack on Cobra and does not contribute a positive side-channel ask price. This matches the Pessl–Prokop [95] taxonomy, in which hybrid structures are specifically cited as fault-injection hardening precisely because of the KDF all-or-nothing dependency. □

References

[1] Gorjan Alagic, Maxime Bros, Pierre Ciadoux, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, Hamilton Silberg, Daniel Smith-Tone, and Noah Waller. Status report on the fourth round of the NIST post-quantum cryptography standardization process. Technical report, NIST Interagency/Internal Report 8545, March 2025.

[2] Martin R. Albrecht, Florian Bourse, et al. Estimate all the LWE, NTRU schemes! In *Security and Cryptography for Networks*, volume 10485 of *LNCS*, pages 351–367, 2017.

[3] Martin R. Albrecht, Florian Bourse, et al. Estimate all the LWE, NTRU schemes! *Security and Cryptography for Networks*, 2018.

[4] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange – a new hope. In *25th USENIX Security Symposium*, pages 327–343, 2016.

[5] Erdem Alkim et al. Newhope – algorithm specifications and supporting documentation. In *NIST Post-Quantum Cryptography Standardization*, 2020.

[6] Nicolas Aragon et al. BIKE: Bit flipping key encapsulation. NIST PQC Round 4 Submission, 2023.

[7] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber: Algorithm specifications and supporting documentation. NIST PQC Submission, 2021.

[8] Nimrod Aviram, Sebastian Schinzel, Juraj Somorovsky, Nadia Heninger, Maik Dankel, Jens Steube, Luke Valenta, David Adrian, J. Alex Halderman, Viktor Dukhovni, Emilia Käsper, Shaanan Cohney, Susanne Engels, Christof Paar, and Yuval Shavitt. DROWN: Breaking TLS using SSLv2. In *25th USENIX Security Symposium*, pages 689–706, 2016.

[9] Magali Bardet, Maxime Bros, Daniel Cabarcas, Philippe Gaborit, Ray Perlner, Daniel Smith-Tone, Jean-Pierre Tillich, and Javier Verbel. Improvements of algebraic attacks for solving the rank decoding and MinRank problems. In *Advances in Cryptology – ASIACRYPT 2020*, volume 12491 of *LNCS*, pages 507–536, 2020.

[10] Mihir Bellare and Thomas Ristenpart. Multi-property-preserving hash domain extension and the EMD transform. In *Advances in Cryptology – ASIACRYPT 2006*, volume 4284 of *LNCS*, pages 299–314, 2006.

[11] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *Proceedings of the 1st ACM Conference on Computer and Communications Security*, pages 62–73, 1993.

[12] D. Benjamin. Protecting Chrome Traffic with Hybrid Kyber KEM. Google Security Blog, August 2023.

[13] Elwyn R. Berlekamp, Robert J. McEliece, and Henk C. A. van Tilborg. On the inherent intractability of certain coding problems. *IEEE Transactions on Information Theory*, 24(3):384–386, 1978.

[14] Daniel J. Bernstein. Cost analysis of hash collisions: Will quantum computers make SHARCS obsolete? *SHARCS'09 Special-Purpose Hardware for Attacking Cryptographic Systems*, 2009.

[15] Daniel J. Bernstein, Karthikeyan Bhargavan, Shivam Bhasin, Anupam Chattopadhyay, Tee Kiah Chia, Matthias J. Kannwischer, Franziskus Kiefer, Thales B. Paiva, Prasanna Ravi, and Goutam Tamvada. KyberSlash: Exploiting secret-dependent division timings in Kyber implementations. IACR Transactions on Cryptographic Hardware and Embedded Systems 2025(2):209–234, 2025. https://kyberslash.cr.yp.to/, CHES 2025 Best Paper Award.

[16] Daniel J. Bernstein, Tung Chou, Tanja Lange, Ruben Niederhagen, and Christiane Peters. Classic McEliece: Conservative code-based cryptography. NIST PQC Round 3 Submission, 2019.

[17] Daniel J. Bernstein, Daira Hopwood, Andreas Hülsing, Tanja Lange, Ruben Niederhagen, Louiza Papachristodoulou, Michael Schneider, Peter Schwabe, and Zooko Wilcox-O'Hearn. SPHINCS: Practical stateless hash-based signatures. In *Advances in Cryptology – EUROCRYPT 2015*, volume 9056 of *LNCS*, pages 368–397, 2015.

[18] Daniel J. Bernstein and Tanja Lange. SUPERCOP: System for unified performance evaluation related to cryptographic operations and primitives, 2024. https://bench.cr.yp.to/supercop.html.

[19] Shivam Bhasin, Jan-Pieter D'Anvers, Daniel Heinz, Thomas Pöppelmann, and Michiel Van Beirendonck. Attacking and defending masked polynomial comparison for lattice-based cryptography. In *IACR Transactions on Cryptographic Hardware and Embedded Systems*, volume 2021, pages 334–359, 2021.

[20] Nina Bindel, Jacqueline Brendel, Marc Fischlin, Brian Goncalves, and Douglas Stebila. Hybrid key encapsulation mechanisms and authenticated key exchange. In *Post-Quantum Cryptography*, volume 11505 of *LNCS*, pages 206–226, 2019.

[21] Nina Bindel, Udyani Herath, Matthew McKague, and Douglas Stebila. Transitioning to a quantum-resistant public key infrastructure. In *Post-Quantum Cryptography*, volume 10346 of *LNCS*, pages 384–405, 2017.

[22] Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 41–69, 2011.

[23] Joppe Bos, Craig Costello, Léo Ducas, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Ananth Raghunathan, and Douglas Stebila. Frodo: Take off the ring! practical, quantum-secure key exchange from LWE. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1006–1018, 2016.

[24] Joppe W. Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS – Kyber: A CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367, 2018.

[25] Zvika Brakerski, Adeline Langlois, Chris Peikert, Oded Regev, and Damien Stehlé. Classical hardness of learning with errors. In *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, pages 575–584, 2013.

[26] Bundesamt für Sicherheit in der Informationstechnik. Cryptographic mechanisms: Recommendations and key lengths. Technical report, BSI Technical Guideline TR-02102-1, 2024.

[27] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. *Proceedings of the 42nd IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 136–145, 2001.

[28] Ran Canetti. Universally composable security. *Journal of the ACM*, 67(5):1–94, 2020.

[29] Ran Canetti, Yevgeniy Dodis, Rafael Pass, and Shabsi Walfish. Universally composable security with global setup. In *Theory of Cryptography Conference – TCC 2007*, volume 4392 of *LNCS*, pages 61–85, 2007.

[30] Ran Canetti, Oded Goldreich, and Shai Halevi. The random oracle methodology, revisited. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*, pages 209–218, 1998.

[31] Wouter Castryck and Thomas Decru. An efficient key recovery attack on SIDH (preliminary version). Cryptology ePrint Archive, Report 2022/975, 2022.

[32] Wouter Castryck and Thomas Decru. An efficient key recovery attack on SIDH. *Advances in Cryptology – EUROCRYPT 2023*, 14008:423–447, 2023.

[33] Ronald Cramer and Victor Shoup. A practical public key cryptosystem provably secure against adaptive chosen ciphertext attack. In *Advances in Cryptology – CRYPTO 1998*, volume 1462 of *LNCS*, pages 13–25, 1998.

[34] Ronald Cramer and Victor Shoup. Design and analysis of practical public-key encryption schemes secure against adaptive chosen ciphertext attack. *SIAM Journal on Computing*, 33(1):167–226, 2003.

[35] Cas Cremers, Marko Horvat, Jonathan Hoyland, Sam Scott, and Thyla van der Merwe. A comprehensive symbolic analysis of TLS 1.3. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1773–1788, 2017.

[36] Cybersecurity and Infrastructure Security Agency. Post-quantum cryptography initiative. Technical report, CISA, 2024.

[37] Jan-Pieter D'Anvers, Marcel Tiepelt, Frederik Vercauteren, and Ingrid Verbauwhede. Timing attacks on error correcting codes in post-quantum schemes. In *ACM Workshop on Theory of Implementation Security (TIS)*, pages 2–9, 2019.

[38] Jintai Ding and Dieter Schmidt. Rainbow, a new multivariable polynomial signature scheme. *Applied Cryptography and Network Security*, 3531:164–175, 2005.

[39] Yevgeniy Dodis and Jonathan Katz. Chosen-ciphertext security of multiple encryption. In *Theory of Cryptography Conference – TCC 2005*, volume 3378 of *LNCS*, pages 188–209, 2005.

[40] Benjamin Dowling, Torben Brandt Hansen, and Kenneth G. Paterson. Many a mickle makes a muckle: A framework for provably quantum-secure hybrid key exchange. In *Post-Quantum Cryptography – PQCrypto 2020*, pages 483–502, 2020.

[41] Benjamin Dowling and Douglas Stebila. Modelling ciphersuite and version negotiation in the TLS protocol. In *Australasian Conference on Information Security and Privacy*, volume 9144 of *LNCS*, pages 270–288, 2015.

[42] Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(1):238–268, 2018.

[43] Léo Ducas and Wessel van Woerden. The closest vector problem in tensored root lattices of type A and in their duals. In *Designs, Codes and Cryptography*, volume 89, pages 1549–1576, 2021.

[44] ETSI Cyber Security Technical Committee. Hybrid cryptographic mechanisms. Technical report, ETSI Technical Specification TS 103 744, 2020.

[45] Jean-Charles Faugère and Antoine Joux. Algebraic cryptanalysis of hidden field equation (HFE) cryptosystems using Gröbner bases. In *Advances in Cryptology – CRYPTO 2003*, volume 2729 of *LNCS*, pages 44–60, 2003.

[46] Marc Fischlin, Anja Lehmann, and Krzysztof Pietrzak. Robust multi-property combiners for hash functions revisited. In *Automata, Languages and Programming*, volume 6198 of *LNCS*, pages 655–666, 2010.

[47] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *Advances in Cryptology – CRYPTO 1999*, 1666:537–554, 1999.

[48] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *Journal of Cryptology*, 26(1):80–101, 2013.

[49] Philippe Gaborit, Olivier Ruatta, Julien Schrek, and Gilles Zémor. New results for rank-based cryptography. *Progress in Cryptology – AFRICACRYPT 2019*, 11627:1–17, 2019.

[50] Daniel Genkin, Itamar Pipman, and Eran Tromer. Get your hands off my laptop: Physical side-channel key-extraction attacks on PCs. In *CHES 2014*, volume 8731 of *LNCS*, pages 242–260, 2014.

[51] Federico Giacon, Felix Heuer, and Bertram Poettering. KEM combiners. In *Public-Key Cryptography – PKC 2018*, volume 10769 of *LNCS*, pages 190–218, 2018.

[52] Qian Guo, Clemens Hlauschek, Thomas Johansson, Norman Lahr, Alexander Nilsson, and Robin Leander Schröder. Don't reject this: Key-recovery timing attacks due to rejection-sampling in HQC and BIKE. In *IACR Transactions on Cryptographic Hardware and Embedded Systems*, volume 2022, pages 223–263, 2022.

[53] Qian Guo, Thomas Johansson, Erik Mårtensson, and Paul Stankovski Wagner. On the asymptotics of solving the LWE problem using coded-BKW with sieving. In *Advances in Cryptology – CRYPTO 2023*, volume 14083 of *LNCS*, pages 401–432, 2023.

[54] Qian Guo, Thomas Johansson, and Alexander Nilsson. A key-recovery timing attack on post-quantum primitives using the Fujisaki-Okamoto transformation and its application on FrodoKEM. In *CRYPTO 2020*, volume 12171 of *LNCS*, pages 359–386, 2020.

[55] Mike Hamburg. Practical implementation of post-quantum key exchange. *Journal of Cryptographic Engineering*, 2021.

[56] Danny Harnik, Joe Kilian, Moni Naor, Omer Reingold, and Alon Rosen. On robust combiners for oblivious transfer and other primitives. In *Advances in Cryptology – EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 96–113, 2005.

[57] Amir Herzberg. On tolerant cryptographic constructions. *Topics in Cryptology – CT-RSA 2005*, 3376:172–190, 2005.

[58] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In *Theory of Cryptography Conference – TCC 2017*, volume 10677 of *LNCS*, pages 341–371, 2017.

[59] Dennis Hofheinz and Victor Shoup. GNUC: A new universal composability framework. *Journal of Cryptology*, 28(3):423–508, 2015.

[60] Andreas Hülsing and Christoph Busold. Optimal parameters for XMSS^{MT}. *Security, Privacy, and Applied Cryptography Engineering*, 8204:194–208, 2013.

[61] ISO/IEC JTC 1/SC 27. IT Security Techniques, 2024.

[62] David Jao and Luca De Feo. Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies. *Post-Quantum Cryptography*, 7071:19–34, 2011.

[63] Haodong Jiang, Zhenfeng Zhang, Long Chen, Hong Wang, and Zhi Ma. Post-quantum IND-CCA-Secure KEM without additional hash. In *IACR Cryptology ePrint Archive*, 2018.

[64] Panos Kampanakis, Peter Panburana, Ellie Daw, and Daniel Van Geest. The viability of post-quantum X.509 certificates. Cryptology ePrint Archive, Report 2018/063, 2018.

[65] Matthias J. Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4, 2024. https://github.com/mupq/pqm4.

[66] Eike Kiltz, Vadim Lyubashevsky, and Christian Schaffner. A concrete treatment of fiat-shamir signatures in the quantum random-oracle model. In *Advances in Cryptology – EUROCRYPT 2018*, volume 10822 of *LNCS*, pages 552–586, 2018.

[67] Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, Konrad Lai, and Onur Mutlu. Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors. In *ACM/IEEE 41st Int'l Symp. on Computer Architecture (ISCA)*, pages 361–372, 2014.

[68] Aviad Kipnis and Adi Shamir. Cryptanalysis of the HFE public key cryptosystem by relinearization. In *Advances in Cryptology – CRYPTO 1999*, volume 1666 of *LNCS*, pages 19–30, 1999.

[69] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. Spectre attacks: Exploiting speculative execution. In *40th IEEE Symposium on Security and Privacy*, pages 1–19, 2019.

[70] Paul C. Kocher. Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. In *CRYPTO 1996*, volume 1109 of *LNCS*, pages 104–113, 1996.

[71] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In *CRYPTO 1999*, volume 1666 of *LNCS*, pages 388–397, 1999.

[72] Kris Kwiatkowski, Panos Kampanakis, Bas Westerbaan, and Douglas Stebila. X25519Kyber768Draft00 hybrid post-quantum key agreement for TLS 1.3. Internet-Draft draft-connolly-tls-mlkem-key-agreement-03, 2024.

[73] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2015.

[74] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 1–23, 2010.

[75] Robert J. McEliece. A public-key cryptosystem based on algebraic coding theory. In *Deep Space Network Progress Report 44*, pages 114–116. Jet Propulsion Laboratory, 1978.

[76] Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, and Gilles Zémor. HQC: Hamming quasi-cyclic. In *NIST Post-Quantum Cryptography Round 2 Submission*, 2018.

[77] Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, et al. Hamming quasi-cyclic (hqc). Technical report, NIST PQC Round 4 Submission, 2023.

[78] Ralph C. Merkle. A certified digital signature. In *Advances in Cryptology – CRYPTO 1989*, volume 435 of *LNCS*, pages 218–238, 1990.

[79] Prasanna Mishra, Indranil Ghosh Ray, Sushmita Ruj, and Bimal K. Roy. Quantum-safe cryptographic constructions: A survey. *Security and Privacy*, 1(6), 2018.

[80] Michael Naehrig et al. FrodoKEM: Learning with errors key encapsulation. Technical report, NIST PQC Submission, 2023.

[81] National Institute of Standards and Technology. Submission requirements and evaluation criteria for the post-quantum cryptography standardization process. Technical report, NIST, 2016.

[82] National Institute of Standards and Technology. FIPS 140-3: Security requirements for cryptographic modules. Technical report, NIST, 2019.

[83] National Institute of Standards and Technology. Status report on the third round of the NIST post-quantum cryptography standardization process. Technical report, NIST Interagency Report 8413, 2022.

[84] National Institute of Standards and Technology. SP 800-140F: CMVP approved non-invasive attack mitigation test metrics. Technical report, NIST, 2023.

[85] National Institute of Standards and Technology. FIPS 203: Module-lattice-based key-encapsulation mechanism standard. Technical report, NIST, 2024.

[86] National Institute of Standards and Technology. FIPS 204: Module-lattice-based digital signature standard. Technical report, NIST, 2024.

[87] National Institute of Standards and Technology. FIPS 205: Stateless hash-based digital signature standard. Technical report, NIST, 2024.

[88] National Security Agency. Announcing the commercial national security algorithm suite 2.0. Technical report, NSA, 2022.

[89] Harald Niederreiter. Knapsack-type cryptosystems and algebraic coding theory. *Problems of Control and Information Theory*, 15(2):159–166, 1986.

[90] Tobias Oder, Tobias Schneider, Thomas Pöppelmann, and Tim Güneysu. Practical CCA2-secure and masked ring-LWE implementation. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(1):142–174, 2018.

[91] Office of Management and Budget. Migrating to post-quantum cryptography. Technical report, OMB Memorandum M-23-02, 2022.

[92] Open Quantum Safe Project. liboqs: C library for quantum-safe cryptography, 2024.

[93] Basker Palaniswamy and Paolo Palmieri. Mtsf—market-theoretic security framework: A unified paradigm for the art of proving and disproving security. *Cryptology ePrint Archive*, 2026.

[94] Christian Paquin, Douglas Stebila, and Goutam Tamvada. Benchmarking post-quantum cryptography in TLS. In *Post-Quantum Cryptography*, volume 12100 of *LNCS*, pages 72–91, 2020.

[95] Peter Pessl and Lukas Prokop. Fault attacks on CCA-secure lattice KEMs. In *IACR Transactions on Cryptographic Hardware and Embedded Systems*, volume 2021, pages 37–60, 2021.

[96] Charles Rackoff and Daniel R. Simon. Non-interactive zero-knowledge proof of knowledge and chosen ciphertext attack. In *Advances in Cryptology – CRYPTO 1991*, volume 576 of *LNCS*, pages 433–444, 1992.

[97] Prasanna Ravi, Shivam Bhasin, Sujoy Sinha Roy, and Anupam Chattopadhyay. On exploiting message leakage in (few) NIST PQC candidates for practical message recovery and key recovery attacks. *IEEE Transactions on Information Forensics and Security*, 17:684–699, 2022.

[98] Prasanna Ravi, Martianus Frederic Ezerman, Shivam Bhasin, Anupam Chattopadhyay, and Sujoy Sinha Roy. Will you cross the threshold for me? generic side-channel assisted chosen-ciphertext attacks on NTRU-based KEMs. In *IACR Transactions on Cryptographic Hardware and Embedded Systems*, volume 2022, pages 722–761, 2022.

[99] Prasanna Ravi, Sujoy Sinha Roy, Anupam Chattopadhyay, and Shivam Bhasin. Generic side-channel attacks on CCA-secure lattice-based PKE and KEMs. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2020(3):307–335, 2020.

[100] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *Proceedings of the 37th Annual ACM Symposium on Theory of Computing (STOC)*, pages 84–93, 2005.

[101] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6), 2009.

[102] Eric Rescorla. The transport layer security (TLS) protocol version 1.3. Technical report, RFC 8446, IETF, 2018.

[103] Sujoy Sinha Roy, Oscar Reparaz, Frederik Vercauteren, and Ingrid Verbauwhede. Compact and side channel secure discrete Gaussian sampling. Cryptology ePrint Archive, Report 2014/591, 2014.

[104] Peter Schwabe, Douglas Stebila, and Thom Wiggers. Post-quantum TLS without handshake signatures. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1461–1480, 2020.

[105] Nicolas Sendrier. Decoding one out of many. *Post-Quantum Cryptography*, 7071:51–67, 2011.

[106] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

[107] Victor Shoup. Sequences of games: A tool for taming complexity in security proofs. *Cryptology ePrint Archive, Report 2004/332*, 2004.

[108] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH. In *Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies (CoNEXT)*, pages 149–156, 2020.

[109] Bo-Yeon Sim, Jihoon Kwon, Joohee Lee, Il-Ju Kim, Tae-Ho Lee, Jaeseung Han, Hyojin Yoon, Jihoon Cho, and Dong-Guk Han. Single-trace attacks on the message encoding of lattice-based KEMs. *IEEE Access*, 8:183175–183191, 2020.

[110] Douglas Stebila, Scott Fluhrer, and Shay Gueron. Design issues for hybrid key exchange in TLS 1.3. Internet-Draft draft-stebila-tls-hybrid-design-01, 2020.

[111] Douglas Stebila, Scott Fluhrer, and Shay Gueron. Hybrid key exchange in TLS 1.3. Internet-Draft draft-ietf-tls-hybrid-design-09, 2024.

[112] Erik Sy, Christian Burkert, Hannes Federrath, and Mathias Fischer. Tracking users across the web via TLS session resumption. In *Proceedings of the 34th Annual Computer Security Applications Conference (ACSAC)*, 2018.

[113] Ehsan Ebrahimi Targhi and Dominique Unruh. Post-quantum security of the Fujisaki-Okamoto and OAEP transforms. In *Theory of Cryptography Conference – TCC 2016-B*, volume 9986 of *LNCS*, pages 192–216, 2016.

[114] Rodolfo Canto Torres and Nicolas Sendrier. Analysis of information set decoding for a sub-linear error weight. In *Post-Quantum Cryptography*, volume 9606 of *LNCS*, pages 144–161, 2016.

[115] Rei Ueno, Keita Xagawa, Yutaro Tanaka, Akira Ito, Junko Takahashi, and Naofumi Homma. Curse of re-encryption: A generic power/EM analysis on post-quantum KEMs. In *IACR Transactions on Cryptographic Hardware and Embedded Systems*, volume 2022, pages 296–322, 2022.

[116] Dominique Unruh. Universally composable quantum multi-party computation. In *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 486–505. Springer, 2010.

[117] Valentin Vasseur. Post-quantum cryptography: A study of the decoding problem of structured codes. *Ph.D. Thesis, Sorbonne Université*, 2021.

[118] B. Westerbaan and C. A. Wood. Sizing Up Post-Quantum Signatures: Comparing NIST Candidates on Cloudflare's Network. Cloudflare Blog, May 2024.

[119] Bas Westerbaan and Christopher A. Wood. Sizing up post-quantum signatures: Comparing NIST candidates on cloudflare's network. Cloudflare Blog, 2023.